



МИНОБРНАУКИ РОССИИ

федеральное государственное автономное образовательное учреждение

высшего образования

«Московский государственный технологический университет «СТАНКИН»

(ФГАОУ ВО МГТУ «СТАНКИН»)

Институт	Кафедра
информационных технологий	информационных систем

Проект по дисциплине «**Управление данными**»

на тему: Проектирование БД торгового центра

Студент группа ИДБ–23-06	 _____	Баранов В. Г.
	подпись	
Руководитель старший преподаватель	 _____	Быстрикова В. А.
	подпись	

Москва 2025 г.

Оглавление

Введение.....	3
Глава 1. Проектирование структуры базы данных.....	4
1.1. Анализ предметной области	4
1.2. Концептуальное проектирование.....	12
1.3. Логическое проектирование	16
1.4. Физическое проектирование.....	23
Глава 2. Описание функционирования базы данных	30
2.1 Назначение и перечень функций базы данных.....	30
2.2 Описание работы с базой данных	31
Заключение	62
Список используемых источников.....	64
Приложение А. Примеры заполнения таблиц.....	65
Приложение Б. SQL-операторы создания основных объектов БД	70
Приложение В. Код программы	73

ВВЕДЕНИЕ

Целью проекта является проектирование и реализация базы данных для системы управления арендой помещений торгового центра. Современные торговые комплексы требуют эффективного управления арендными отношениями, контроля договоров, учета платежей и оперативного мониторинга состояния помещений. Ручное ведение этих процессов приводит к значительным временным затратам, ошибкам в расчетах и снижению качества обслуживания как арендаторов, так и посетителей. Автоматизация данных операций позволит оптимизировать управленческие процессы, повысить прозрачность финансовых расчетов и улучшить общую эффективность работы торгового центра.

В ходе работы последовательно решаются задачи анализа предметной области, концептуального, логического и физического проектирования базы данных. На первом этапе изучаются существующие программные решения для управления торговыми центрами, что позволяет выделить ключевые функциональные требования к разрабатываемой системе. На этапе концептуального проектирования определяются основные роли пользователей (покупатель, арендатор, менеджер по аренде) и строится диаграмма вариантов использования. При логическом проектировании разрабатывается ER-модель, отражающая взаимосвязи между сущностями, выполняется нормализация отношений для обеспечения целостности данных.

Результатом работы станет полноценная модель базы данных с веб-интерфейсом для взаимодействия пользователей с системой. Разрабатываемое решение обеспечит надежное хранение информации об аренде, автоматизацию административных процессов, повышение точности финансовых расчетов и формирование аналитической отчетности для оценки эффективности использования площадей торгового центра. Реализация системы позволит значительно повысить качество управления арендными отношениями и создаст основу для дальнейшего развития информационной инфраструктуры торгового центра.

ГЛАВА 1. ПРОЕКТИРОВАНИЕ СТРУКТУРЫ БАЗЫ ДАННЫХ

1.1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Предметная область представляет собой сферу человеческой деятельности, которая выделена и описана в соответствии с определенными критериями. В это понятие включаются данные о ее элементах, явлениях, отношениях и процессах, которые отражают различные аспекты данной деятельности.

Одной из первоначальных задач, с которой сталкивается разработчик программной системы, является изучение, осмысление и анализ предметной области. Это важно, поскольку предметная область оказывает значительное влияние на все аспекты проекта: требования к системе, взаимодействие с пользователем, модель хранения данных, реализацию и другие.

Анализ предметной области помогает выделить ее ключевые сущности, определить первоначальные функциональные требования и установить границы проекта.

В данной работе в качестве предметной области рассматривается деятельность торгового центра.

Торговый центр (ТЦ) представляет собой крупный комплекс, объединяющий множество магазинов, ресторанов, развлекательных и сервисных зон. В такой структуре задействовано большое количество различных процессов и участников, включая арендаторов, посетителей, персонал и управляющую компанию.

Актуальность управления торговым центром (ТЦ) обусловлена рядом ключевых факторов, которые определяют его успешное функционирование и конкурентоспособность на рынке. Вот основные аспекты, подчеркивающие важность эффективного управления ТЦ:

1. Изменение потребительского поведения: Современные покупатели все больше ценят удобство, разнообразие и персонализированный подход. Управление ТЦ должно учитывать эти тенденции, чтобы привлекать и удерживать посетителей.

2. Конкуренция: Рынок торговых центров насыщен, и конкуренция высока. Эффективное управление позволяет выделяться среди конкурентов, предлагая уникальные торговые предложения и высококачественный сервис.

3. Технологические инновации: Использование современных технологий, таких как Big Data, искусственный интеллект и системы автоматизации, помогает оптимизировать операционные процессы, улучшать управление арендаторами и анализировать поведение посетителей.

4. Финансовая устойчивость: Эффективное управление финансами и ресурсами ТЦ обеспечивает его экономическую устойчивость, позволяя инвестировать в развитие и модернизацию.

5. Безопасность и комфорт: в условиях пандемии и роста требований к безопасности, управление ТЦ должно включать в себя меры по обеспечению безопасности посетителей и персонала, а также создание комфортной среды для покупок и отдыха.

6. Устойчивое развитие: Современные ТЦ должны соответствовать принципам устойчивого развития, включая энергоэффективность, экологическую ответственность и социальную вовлеченность.

7. Адаптация к изменениям: Рынок постоянно меняется, и управление ТЦ должно быть гибким и адаптивным, чтобы быстро реагировать на новые вызовы и возможности.

Основные процессы и задачи, решаемые в данной предметной области:

1. Управление арендаторами:

- Ведение данных арендаторов.
- Контроль договоров аренды и их сроков.
- Учет арендных платежей и коммунальных услуг.
- Анализ эффективности арендаторов.

2. Анализ посещаемости и предпочтений посетителей:

- Сбор данных о посещаемости.
- Анализ данных для оптимизации ассортимента и услуг.
- Разработка и реализация маркетинговых кампаний.

3. Управление персоналом

- Ведение данных сотрудников.
- Автоматизация процессов найма и обучения.
- Учет рабочего времени и распределение обязанностей.
- Контроль выполнения задач и оценка эффективности работы.

4. Финансовый учет и отчетность

- Ведение бухгалтерского учета.
- Формирование финансовых отчетов.
- Анализ доходов и расходов.
- Планирование бюджета и контроль его исполнения.

5. Безопасность и контроль доступа

- Управление системами видеонаблюдения.
- Контроль доступа в различные зоны ТЦ.
- Учет и анализ инцидентов безопасности.
- Обеспечение соблюдения норм и правил безопасности.

Проектирование базы данных позволит интегрировать все эти процессы в единую систему, что значительно повысит эффективность управления торговым центром.

Существует множество программных продуктов, решающих задачи в области управления торговыми центрами:

- 1С: Управление торговым центром (приложение). [1]
- «ArendaSoft» (приложение или сайт). [2]
- «Retail Pro Prism» от компании Retail Pro International (приложение).
- «Mall Manager» от компании Yardi (приложение).
- «Oracle Retail» (приложение и сайт).
- «Rentrabbit» (приложение и сайт) и другие.

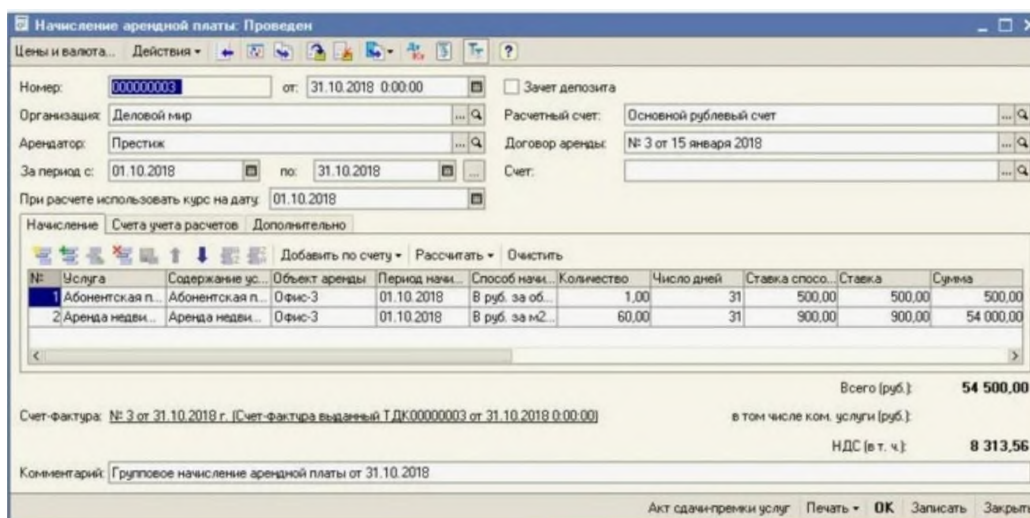
Эти системы предназначены для автоматизации и оптимизации процессов управления торговыми центрами, начиная от управления арендой и заканчивая анализом посещаемости и финансовым учетом. Далее рассмотрим

более подробно несколько из них.

В качестве первого программного продукта рассмотрим 1С: Управление торговым центром [1] — это специализированное решение на платформе 1С:Предприятие, предназначенное для автоматизации управления торговыми центрами. Оно позволяет эффективно управлять арендой, финансами, персоналом и маркетинговыми кампаниями.

Среди основных функций стоит выделить:

- Управление арендой: Ведение данных арендаторов, контроль договоров аренды, учет арендных платежей и коммунальных услуг (рис.1.1).



№	Услуга	Содержание ус...	Объект аренды	Период нач.	Способ нач.	Количество	Число дней	Ставка спосо...	Ставка	Сумма
1	Абонентская п...	Абонентская п...	Офис-3	01.10.2018	В руб. за об...	1,00	31	500,00	500,00	500,00
2	Аренда недви...	Аренда недви...	Офис-3	01.10.2018	В руб. за м2...	60,00	31	900,00	900,00	54 000,00

Всего (руб.): 54 500,00
в том числе ком. услуги (руб.):
НДС (в т. ч.): 8 313,56

Рис.1.1 Управление арендой в 1С

- Анализ посещаемости: Сбор и анализ данных о посещаемости, разработка маркетинговых кампаний на основе этих данных (рис.1.2).

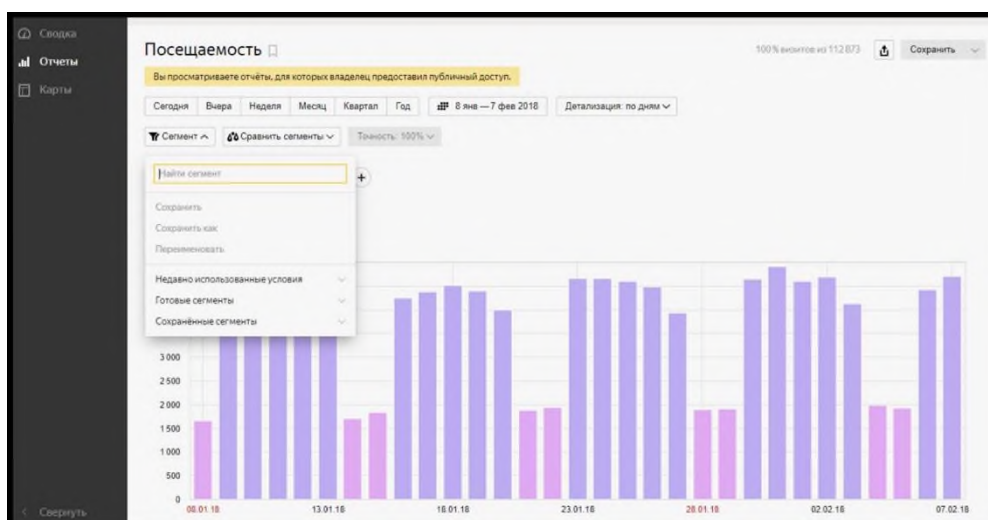


Рис.1.2 Анализ посещаемости

- Финансовый учет: Ведение бухгалтерского учета, формирование финансовых отчетов, анализ доходов и расходов.
- Управление эксплуатационными расходами: Контроль и распределение затрат на содержание и обслуживание торгового центра, включая коммунальные услуги и ремонтные работы.
- Управление персоналом: Ведение данных сотрудников, учет рабочего времени, распределение обязанностей (рис.1.3).

N	Сотрудник	Фамилия	Имя	Отчество	ИНН	СНИЛС
1	Базин Антон Владим...	Базин	Антон	Владимирович	772620227070	322-620-227 19
2	Бальцер Герман Эдуа...	Бальцер	Герман	Эдуардович	776937625207	416-937-625 08
3	Булатов Игорь Виле...	Булатов	Игорь	Виленович	775245067541	505-245-067 50
4	Ваньков Александр М...	Ваньков	Александр	Матвеевич	509557755558	699-557-755 05
5	Войцехович Игорь Бо...	Войцехович	Игорь	Борисович	500865692144	780-865-692 61
6	Волков Марат Савель...	Волков	Марат	Савельевич	778172302637	878-172-302 42
7	Габровская Светлана ...	Габровская	Светлана	Марковна	772482788663	962-482-788 58
8	Громова Надежда Пет...	Громова	Надежда	Петровна	776798023080	056-798-023 06
9	Захарюин Савелий Пе...	Захарюин	Савелий	Петрович	502003164445	142-003-164 86
10	Козьмин Глеб Матвее...	Козьмин	Глеб	Матвеевич	505314926980	235-314-926 52

Руководитель: Булатов Игорь Виленович Должность: Генеральный директор

Комментарий: Ответственный: Савинская З.Ю. (Системный программист)

Рис.1.3 Управление персоналом

- Маркетинг и реклама: Планирование и анализ рекламных кампаний, управление акциями и скидками.
- Управление техническим обслуживанием: Планирование и контроль технического обслуживания инфраструктуры торгового центра, включая системы безопасности, вентиляции, освещения и другие инженерные системы.

Программа "1С: Управление торговым центром" предоставляет комплексное решение для эффективного управления всеми аспектами работы торгового центра. Она позволяет автоматизировать процессы учета и анализа финансовых потоков, что значительно упрощает контроль над доходами и расходами. Кроме того, система поддерживает интеграцию с другими продуктами "1С", что делает её адаптируемой и удобной для использования в различных бизнес-сценариях.

Рассмотрим второй программный продукт ArendaSoft [2] — это многофункциональное программное обеспечение для управления арендным бизнесом, которое автоматизирует и упрощает процессы от создания и ведения договоров аренды до учета платежей и генерации отчетов. Благодаря интуитивно понятному интерфейсу и широкому спектру функций, включая отслеживание состояния объектов, контроль своевременности платежей и эффективное взаимодействие с арендаторами, Программа также предлагает интеграцию с другими системами, мобильное приложение для управления из любой точки мира и функции безопасности и резервного копирования для сохранности данных, что делает её универсальным решением для бизнеса любого масштаба.

Среди основных функций стоит выделить:

- Управление договорами аренды: быстрый и удобный поиск по клиентам, объектам аренды и договорам (рис. 1.4).

Боксы

Главная - Боксы

Этаж А

A001 0.00	A002 1.50	A003 1.00	A004 11.00	A005 9.00	A006 3.50	A007 7.00	A008 9.00	A009 6.00	A010 8.00	A011 8.00	A012 7.50	A013 4.00	A014 4.50	A015 5.00	A016 4.00	A017 7.00	A018 2.50	A019 4.50
A020 4.50	A021 1.00	A022 1.00	A023 1.00	A024 0.50	A025 0.50	A026 0.50	A027 0.50	A028 0.50	A029 1.00	A030 1.00	A031 1.00	A032 5.00	A033 1.50	A034 4.00	A035 5.00	A036 5.50	A037 5.50	A038 0.00
A039 4.50																		

План этажа

Этаж В

B201 0.00	B202 4.50	B203 4.50	B204 0.50	B205 0.50	B206 0.50	B207 0.50	B208 0.50	B209 0.50	B210 1.50	B211 0.50	B212 2.50	B213 2.50	B214 4.50	B215 5.50	B216 2.00	B217 0.00	B218 4.50	B219 1.00
B220 1.00	B221 4.50	B222 0.50	B223 5.50	B224 1.50	B225 3.00	B226 1.00	B227 1.00	B228 5.00	B229 0.00	B230 0.00	B231 1.00	B232 1.00	B233 1.00	B234 1.00	B235 1.00	B236 0.50	B237 0.00	B238 0.50
B239 0.50	B240 0.50	B241 4.50	B242 0.50	B243 4.50	B244 4.50	B245 4.50	B246 1.00	B247 5.00	B248 1.50	B249 1.50	B250 1.50	B251 1.50	B252 1.50	B253 5.50	B254 1.50	B255 1.50	B256 1.00	B257 1.00
B258 1.00	B259 5.00	B260 0.00	B261 5.50	B262 0.00	B263 0.00	B264 5.00	B265 2.50	B266 2.50	B267 2.50	B268 2.50	B269 0.50	B270 0.50	B271 0.50	B272 0.50	B273 0.50	B274 0.50	B275 0.50	B276 0.50
B277 1.50	B278 7.50	B279 4.50	B280 7.50	B281 0.50	B282 0.50	B283 0.50	B284 1.50	B285 5.50	B286 4.50	B287 0.00	B288 4.50	B289 7.00	B290 0.00	B291 0.00	B292 2.00	B293 2.00	B294 7.00	B295 1.00
B296 1.00	B297 4.00	B298 3.50	B299 0.00	B300 5.00	B301 4.50	B302 4.50	B303 1.00	B304 1.00	B305 1.00	B306 1.00	B307 1.00	B308 1.00	B309 1.00	B310 4.00	B311 1.00	B312 1.00	B313 1.00	B314 1.00
B315 1.00	B316 5.00	B317 5.00	B318 4.50	B319 5.00	B320 0.50	B321 0.00	B322 4.00	B323 4.00	B324 5.00	B325 1.00	B326 5.00	B327 5.00	B328 4.50	B329 4.00	B330 3.00	B331 1.00	B332 1.00	B333 5.50
B334 1.00	B335 1.00	B336 1.00	B337 1.00	B338 2.50	B339 3.50	B340 1.00	B341 1.00	B342 1.00	B343 1.00	B344 1.00	B345 0.00	B346 0.00	B347 1.50	B348 0.50	B349 4.00	B350 1.00	B351 1.00	B352 1.00
B353 1.00	B354 4.00	B355 0.00	B356 1.50	B357 1.50	B358 2.50	B359 0.50	B360 1.00	B361 0.50	B362 1.00	B363 0.50	B364 1.00	B365 0.50	B366 1.00	B367 0.50	B368 1.00	B369 0.50	B370 1.00	B371 1.50
B372 2.00	B373 0.50	B374 1.00	B375 0.50	B376 1.00	B377 0.50	B378 1.00	B379 0.50	B380 1.00	B381 0.50	B382 1.00	B383 0.50	B384 1.00	B385 0.50	B386 1.00	B387 0.50	B388 0.50	B389 0.50	B390 0.50
B391 0.50	B392 0.50	B393 0.50	B394 0.50	B395 0.50	B396 0.50	B397 0.50	B398 0.50	B399 0.50	B400 0.50	B401 0.50	B402 0.50	B403 0.50	B404 0.50	B405 0.50	B406 0.50	B407 0.50	B408 1.50	B409 1.50
B410 0.50	B411 0.50	B412 0.50	B413 0.50	B414 0.50	B415 0.50	B416 0.50	B417 0.50	B418 0.50	B419 0.50	B420 0.50	B421 0.50	B422 0.50	B423 0.50	B424 0.50	B425 0.50	B426 0.50	B427 0.50	B428 0.50
B429 0.50	B430 0.50	B431 0.50	B432 0.50	B433 0.50	B434 0.50	B435 0.50	B436 0.50	B437 0.50	B438 0.50	B439 0.50	B440 0.50	B441 0.50	B442 0.50	B443 0.50	B444 0.50	B445 0.50	B446 0.50	B447 0.50
B448 0.50																		

Рис.1.4 Управление договорами аренды

- Учет платежей: контроль арендных и прочих платежей в различных представлениях (рис. 1.5).

Список оплат

Показаны записи 1-30 из 1706.

Номер	Договор	Бокс	Клиент	Период оплаты	Сумма оплаты	Тип оплаты	Статус	Оплата с	Оплата по	Действие
1803	№743/17 07.09.17 - 16.09.18	С013 1.50	Клиент	1 месяц	2 050 Р	расчетный счет Альфа	Получено	17.03.2018	16.04.2018	2
1802	№643/17 03.10.17 - 02.10.18	С142 1.50	Клиент	7 месяцев	43 200 Р	Расчетный счет	Получено	04.04.2018	03.11.2018	2
1801	№114 15.03.18 - 14.03.19	С034 2.00 С035 2.00	Клиент	12 месяцев	44 500 Р	Касса М (чек)	Получено	15.03.2018	14.03.2019	1
1800	№213 27.03.18 - 26.07.18	С210 1.50	Клиент	1 месяц	2 525 Р	Касса Н	Получено	27.03.2018	26.04.2018	2
1799	№169 15.03.18 - 14.03.19	Б205 1.50	Клиент	6 месяцев	29 700 Р	Касса Н	Получено	15.03.2018	14.09.2018	1
1798	№127 19.03.18 - 18.04.18	Б302 0.50 Б426 0.50	Клиент	1 месяц	1 472 Р	расчетный счет Альфа	Получено	19.03.2018	18.04.2018	2

Рис.1.5 Учет платежей

- Отчетность: автоматическая подготовка и печать договоров, счетов, актов и пр. (рис. 1.6).

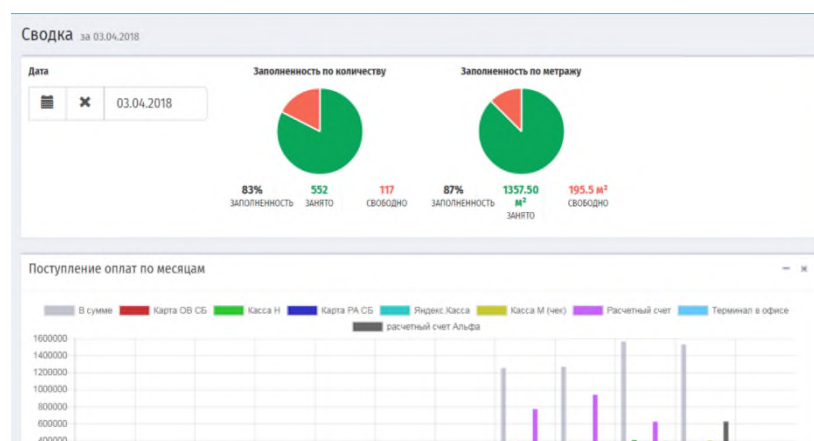


Рис.1.6 Отчетность

- Учет задолженностей: удобство отслеживания дебетовой задолженности и конкретных должников (рис. 1.7).

Очередь оплат

Всего 150 записей.

Боксы	Договор	За месяц	Долг	Получено	До
№127 1.50	№2709 01.10.18 - 30.09.19	2 300 Р	-	27 600 Р	01.10.18
Б257 5.00	№2733 01.10.18 - 30.09.19	1 450 Р	-	17 400 Р	01.10.18
Б446 0.50	№2763 01.10.18 - 30.09.19	750 Р	-	4 500 Р	01.10.18
№25 3.50	№2737 01.10.18 - 30.09.19	5 200 Р	-	31 200 Р	01.10.18
№13 3.00	№2764 01.10.18 - 30.09.19	1 550 Р	-	1 550 Р	01.10.18
Б230 3.00	№2739 01.10.18 - 30.09.19	4 250 Р	-	51 000 Р	01.10.18
Б427 0.50	№2739 01.10.18 - 30.09.19	800 Р	-	1 600 Р	01.10.18
С102 1.50	№2764 01.10.18 - 30.09.19	6 350 Р	-	76 200 Р	01.10.18
Б127 5.00	№2709 01.10.18 - 30.09.19	5 500 Р	13 960 Р	31 000 Р	16.07.18

Рис. 1.7 Учет задолженностей

- Статистика: сводные графики по показателям выручки, заполненности, просроченных платежей и т.д. (рис.1.8).

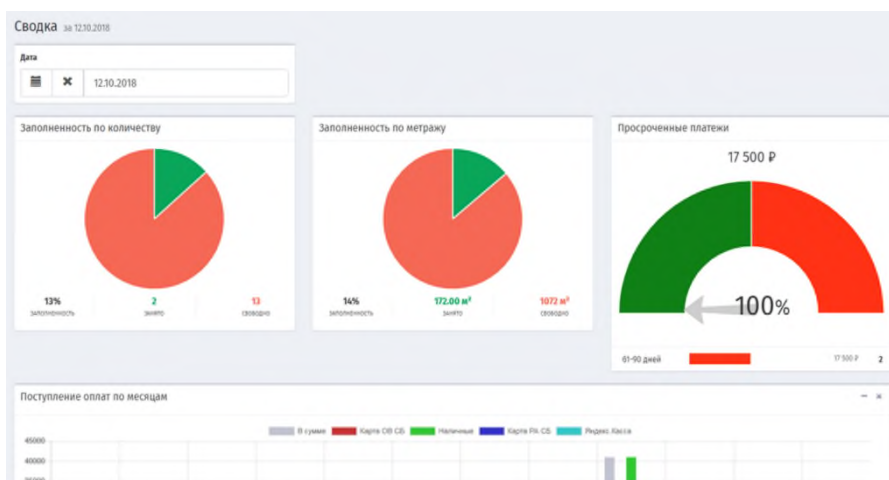


Рис. 1.8 Статистика

- Управление объектами: планы арендуемых площадей с визуализацией статусов объектов аренды в реальном времени (рис.1.9).

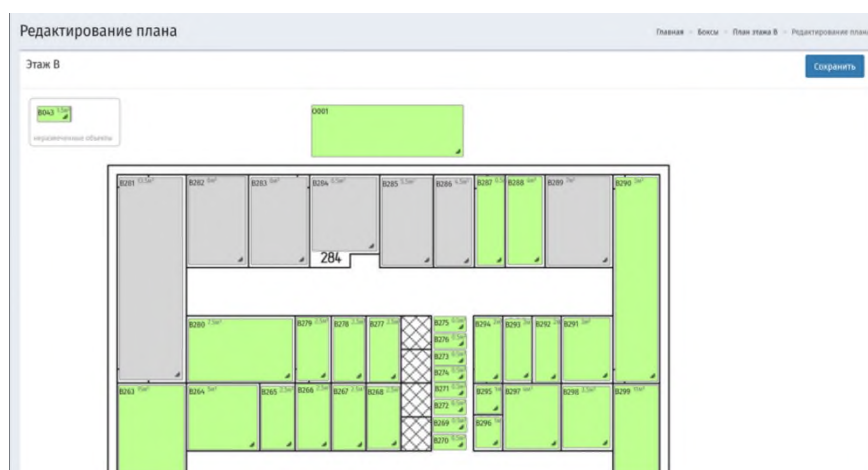


Рис. 1.9 Управление объектами

Главное преимущество ArendaSoft заключается в комплексном управлении арендным бизнесом, автоматизируя процессы от создания договоров до учета платежей. Программа обеспечивает удобное взаимодействие с арендаторами и позволяет генерировать детальные отчеты для аналитики. Интеграция с другими системами и мобильное приложение делают управление арендой гибким и доступным из любой точки мира. Дополнительно, ArendaSoft гарантирует безопасность данных и их резервное копирование.

После проведения анализа предметной области был выделен перечень функций, которые будут реализованы в данной работе:

- Ведение данных арендаторов (сбор контактных данных).
- Контроль договоров аренды и их сроков.
- Мониторинг статуса объекта.
- Учет арендных платежей и коммунальных услуг.
- Ведение данных сотрудников.
- Учет и обработка заявок на ремонт.
- Формирование отчетности.

1.2. КОНЦЕПТУАЛЬНОЕ ПРОЕКТИРОВАНИЕ

Концептуальное проектирование — это процесс создания формализованного описания предметной области, который позволяет оценить корректность разрабатываемого проекта базы данных и при этом не зависит от конкретной системы управления базами данных (СУБД).

Любая база данных создаётся для решения определённых практических задач. В простейшем случае она предназначена для хранения и выдачи данных, а в более сложных случаях — для решения задач, использующих данные из базы в качестве исходных.

Цель концептуального проектирования — определить содержание базы данных.

Диаграмма вариантов использования (ДВИ) или диаграмма прецедентов является исходной концептуальной моделью системы в процессе её проектирования и разработки. Она описывает функции, доступные каждой группе пользователей в разрабатываемой программной системе.

ДВИ используется для формализации функциональных требований к системе с помощью действующих лиц и вариантов использования. Суть диаграммы заключается в представлении проектируемой системы в виде множества объектов, взаимодействующих с системой через варианты использования.

Объектом в ДВИ может быть любой объект, субъект или система, взаимодействующая с разрабатываемой системой извне. Вариант использования — это спецификация функций, которые система предоставляет объекту. Каждый вариант использования определяет набор действий, выполняемых системой при взаимодействии с объектом. При этом в модели не указывается, как именно будет реализован этот набор действий.

Проектируемая база данных (БД) предназначена для управления арендой торговых помещений в торговом центре. Взаимодействие с БД осуществляется тремя категориями пользователей: Покупатель, Арендатором и Менеджером по аренде.

Клиент, заходя в торговый центр и открывая Web- приложение, получает доступ к информации о доступных магазинах и заведениях. Пользователь может просматривать список торговых точек, читать оставленные отзывы и оставлять свои отзывы о посещениях.

Арендатор через специализированный интерфейс вносит ежемесячные платежи за аренду и коммунальные услуги, а также просматривает информацию о своих торговых точках и отзывы на них. Для оформления договора он может подать заявку на аренду помещения через систему, после чего выбранный объект за ним временно бронируется. Окончательное оформление и подписание договора с менеджером, который обеспечивает соблюдение прав и обязательств обеих сторон, происходят во время личной встречи после подтверждения арендатором своего решения, и затем помещение официально закрепляется за ним. Все его действия фиксируются в системе, что обеспечивает целостность данных и актуальность отчетности. Также арендатор может расторгнуть договор по собственной инициативе, уведомив об этом менеджера. Процедуры оформления и расторжения договора происходят в соответствии с четкими правилами, что поддерживает взаимовыгодные и прозрачные отношения между арендаторами и управляющей компанией торгового центра.

Менеджер по аренде имеет доступ к функциям БД, включающие в себя

ведение отчетности, настройку тарифов и управление персоналом, а также ему доступны функции по обновлению каталога помещений, контролю за оплатой, ремонтными работами и заключением договоров. В случае расторжения договора предусмотрены временные ограничения, прописанные в условиях контракта.

Все данные в БД обрабатываются в реальном времени, что позволяет менеджеру эффективно управлять торговым центром и поддерживать высокое качество обслуживания арендаторов и клиентов.

С проектируемой системой будут взаимодействовать следующие действующие лица:

- Клиент (Покупатель).
- Арендатор.
- Менеджер по аренде.

Покупателю доступны следующие функции:

- Оставить отзыв.
- Просмотр отзывов.
- Просмотреть список торговых точек.

Арендатору доступны следующие функции:

- Просмотр отзывов о своих торговых точках.
- Просмотр заключенных договоров аренды.
- Внесение платежей (ежемесячно).
- Получение информации о помещении.
- Бронирование помещения.
- Расторжение договора.

Менеджеру по аренде доступны следующие функции:

- Учет арендаторов
- Ведение отчетности.
- Изменение стоимости аренды.
- Найм и увольнение персонала.

- Обновление каталога помещений.
- Просмотр и оформление договора аренды.
- Подача заявки на ремонт.
- Расторжение договора.
- Бронирование помещений.
- Контроль оплаты и возможность вносить платежи.

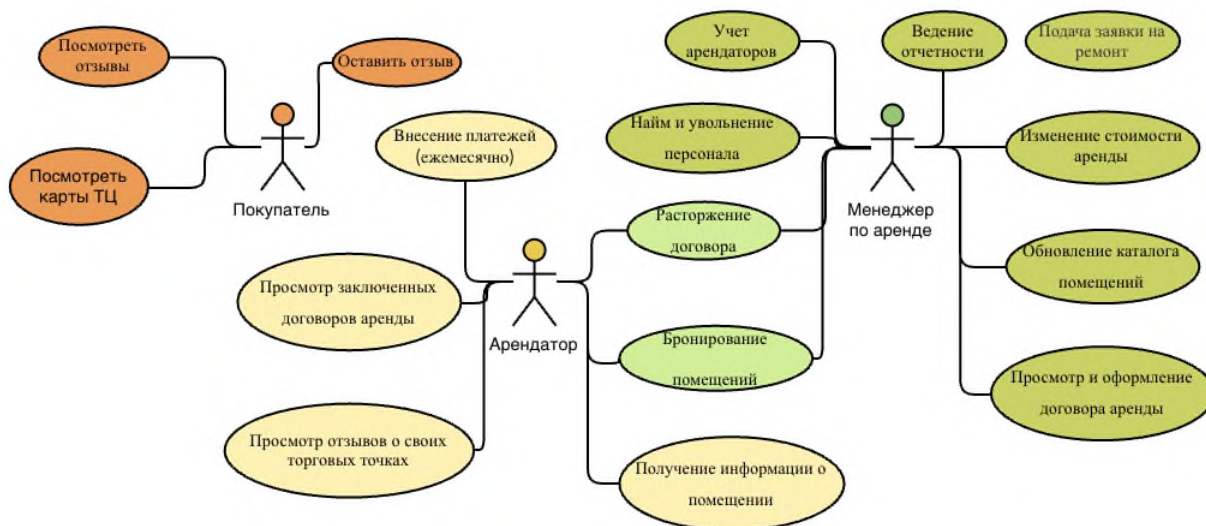


Рис. 1.10 Диаграмма вариантов использования базы данных аренды ТЦ

Для решения выделенных задач (рис. 1.10) необходимо хранение данных о следующих объектах:

- Информация об арендаторе (ID арендатора, ФИО, номер телефона)
- Договор аренды (№ договора, ID арендатора, ID сотрудника, дата подписания, дата окончания, № помещения, тариф).
- Информация о сотрудниках (ID сотрудника, ФИО, должность, номер телефона, статус (работает или уволен)).
- Оплата аренды и коммунальных услуг (№ операции, № договора, сумма, дата оплаты).
- Торговая точка (№ помещения, категория торговой точки, название магазина, статус (свободная, занятая, забронированная и на ремонте), информация о текущем арендаторе, статус договора (активен, истек или расторгнут)).
- Помещения (№ помещения, тип помещения, площадь, расположение,

стоимость аренды, условия аренды).

- Бронирования (ID брони, № помещения, ID арендатора, дата бронирования, дата истечения, статус брони)
- Заявки на ремонт (№ помещения, № заявки, описание проблемы, дата подачи, статус выполнения (выполнено или не выполнено)).
- Отзыв (номер отзыва, номер торговой точки, описание, дата отзыва, автор)

Необходимые данные можно классифицировать по частоте их изменения:

- Условно-постоянные данные (договоры аренды, список помещений, информация о тех. персонале).
- Данные, которые оперативно обновляются при каждом решении задачи (история платежей, бронирование, информация о ремонте и обслуживании помещений, информация об арендаторах, заявки на ремонт, штрафы).

1.3. ЛОГИЧЕСКОЕ ПРОЕКТИРОВАНИЕ

Вторым этапом проектирования БД является логическое проектирование. Оно заключается в создании схемы базы данных на основе конкретной модели данных, например, реляционной модели данных. Для реляционной модели данных логическая модель — набор схем отношений, обычно с указанием первичных ключей, а также «связей» между отношениями, представляющих собой внешние ключи.

При создании моделей данных используется метод семантического моделирования. Семантическое моделирование основывается на значении структурных компонентов или характеристик данных, что способствует правильности их интерпретации. В качестве инструмента семантического моделирования используются различные варианты диаграмм «сущность-связь».

Целью логического проектирования является определение состава и структуры таблицы БД на основе результатов концептуального

проектирования и проверка полученной модели с помощью методов нормализации.

Создание ER-диаграмм

ER-модель — представляет собой формальную конструкцию, которая сама по себе не предписывает графических средств ее визуализации.

Основные элементы ER-моделей:

- объекты (сущности);
- связи между объектами.

Сущность — это реальный или представляемый объект, информация о котором должна сохраняться и быть доступна. В диаграммах ER-модели сущность представляется в виде прямоугольника, содержащего имя сущности.

Связь — это графически изображаемая ассоциация, устанавливаемая между двумя сущностями. Эта ассоциация всегда является бинарной и может существовать между двумя разными сущностями или между сущностью и ей же самой. Связь между сущностями характеризуется типом связи (1:1, 1:N, N:M) и обязательностью (обязательные и необязательные).

На этапе концептуального проектирования были определены ключевые объекты, информацию о которых необходимо хранить в базе данных: арендатор, договор аренды, технический персонал, операции по оплате аренды и коммунальных услуг, помещения, торговые точки, задолженности, штрафы, заявки на ремонт и отзывы.

Они станут основой для ER-моделирования и последующего процесса формирования отношений.

Построим ER-диаграммы всех сущностей и связей между ними.



Рис. 1.11 ER-диаграмма "Менеджер по аренде оформляет договор"

Связь «Оформляет» (рис. 1.11) представляет собой отношение типа 1:M, поскольку один сотрудник (менеджер по аренде) может оформлять несколько договоров с арендатором для различных объектов. Сущность «Договор» имеет обязательный класс принадлежности (КП), так как каждый договор должен быть оформлен сотрудником (менеджером по аренде). В то же время сущность «Сотрудник» может иметь необязательный класс принадлежности, поскольку не все менеджеры могут участвовать в оформлении.

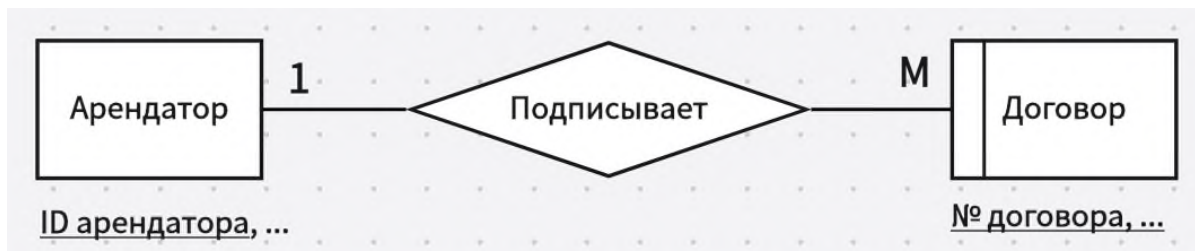


Рис. 1.12 ER-диаграмма "Арендатор подписывает договор"

Связь «Подписывает» (рис. 1.12) представляет собой отношение типа 1:M, так как потенциальный арендатор может подписать несколько договоров с арендодателем по множеству торговых точек. Сущность «Арендатор» имеет необязательный класс принадлежности, поскольку арендатор не обязательно должен заключать договор. В то же время сущность «Договор» обладает обязательным классом принадлежности, так как каждый договор должен быть подписан арендатором.



Рис. 1.13 ER-диаграмма "Договор заключается на помещение"

Связь «Заключается» (рис. 1.13) представляет собой отношение типа M:1, поскольку каждый договор заключается для одного помещения. Сущность «Договор» является обязательной, поскольку он служит для закрепления помещения за арендатором. В свою очередь, сущность «Помещение» может быть необязательной, так как помещения могут оставаться незанятыми.

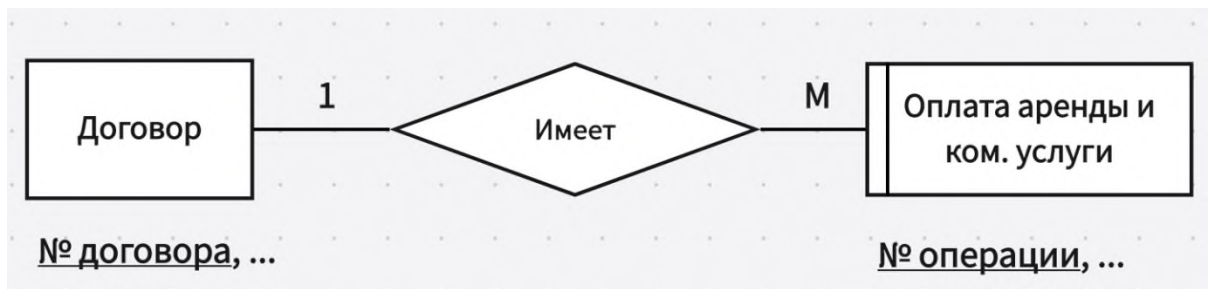


Рис. 1.14 ER-диаграмма “Договор имеет оплату аренды и коммунальных услуг”

Связь «Имеет» (рис. 1.14) характеризуется типом 1:M, поскольку может существовать только один договор, который может быть оплачен несколько раз, например, ежемесячно. Сущность «Договор» имеет необязательный КП, поскольку арендатор может не производить оплату аренды. В то же время сущность «Оплата аренды и ком. услуг» обладает обязательным КП, поскольку оплата аренды всегда связана с определённым договором.



Рис. 1.15 ER-диаграмма “Помещение арендуется для торговой точки”

Связь “Арендуется” (рис. 1.15) представляет собой отношение типа 1:M, поскольку за много лет в одном помещении могут находиться разные торговые точки. Сущность “Помещение” может содержать необязательный класс принадлежности, потому что помещение может быть незанято арендой торговой точки. А сущность “Торговая точка” должна содержать обязательный класс принадлежности, потому что каждая торговая точка обязательно должна занимать одно помещение



Рис. 1.16 ER-диаграмма “Сотрудник обрабатывает заявку на ремонт”

Связь «Обрабатывает» (рис. 1.16) относится к типу 1:M, где множество заявок на ремонт может обрабатываться одним сотрудником технического персонала. Сущность «Заявка на ремонт» содержит обязательный класс принадлежности, так как каждая заявка требует обработки сотрудником. В то же время сущность «Сотрудник» имеет необязательный класс принадлежности, поскольку не все сотрудники могут обрабатывать заявки на ремонт.

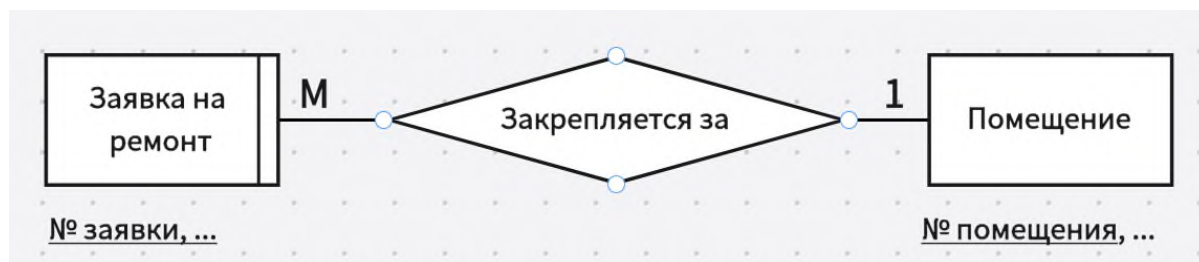


Рис. 1.17 ER-диаграмма «Заявка на ремонт закрепляется за помещением»

Связь «Закрепляется за» (рис. 1.17) относится к типу M:1, где множество заявок на ремонт могут относиться к одному помещению. Сущность «Заявка на ремонт» содержит обязательный класс принадлежности, так как каждая заявка относится к конкретному помещению. В то же время сущность «Помещение» имеет необязательный класс принадлежности, поскольку не каждое помещение должно иметь активные заявки на ремонт.

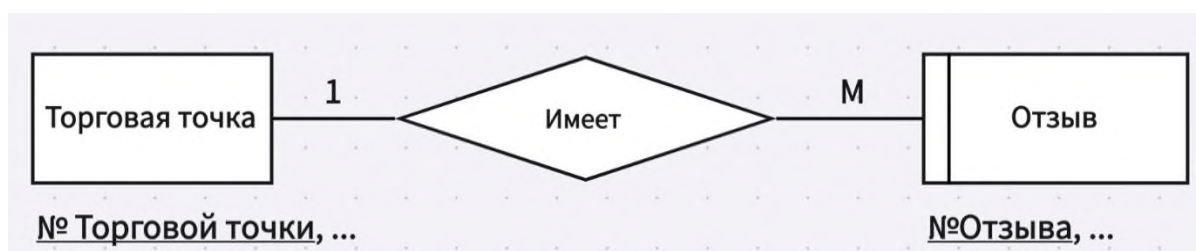


Рис. 1.18 ER-диаграмма «Посетитель оставляет отзыв»

Связь «Имеет» (рис. 1.18) относится к типу 1:M, где одна торговая точка может иметь множество отзывов. Сущность «Торговая точка» содержит необязательный класс принадлежности, потому что точка может существовать без отзывов. Сущность «Отзыв» имеет обязательный класс принадлежности, так как каждый отзыв должен быть привязан к конкретной точке.

Формирование отношений на основе анализа ER-диаграмм

Отношение «Оформляется» (рис. 1.11) удовлетворяет условиям правила 4, в соответствии с которым получаем отношения:

1. Сотрудник (ID сотрудника, ...)
2. Договор (№ договора, ID сотрудника, ...)

Отношение «Подписывает» (рис. 1.12) удовлетворяет условиям правила 4, в соответствии с которым получаем отношения:

1. Арендатор (ID арендатора, ...)
2. Договор (№ договора, ID арендатор, ...)

Отношение «Заключается» (рис. 1.13) удовлетворяет условиям правила 4, в соответствии с которым получаем отношения:

1. Помещение (№ помещения, ...)
2. Договор (№ договора, № помещения, ...)

Отношение «Имеет» (рис. 1.14) удовлетворяет условиям правила 4, в соответствии с которым получаем отношения:

1. Договор (№ договора, ...)
2. Оплата аренды и ком. услуг (№ операции, № договора, ...)

Отношение «Арендуется под» (рис. 1.15) удовлетворяет условиям правила 2, в соответствии с которым получаем отношения:

1. Помещение (№ помещения, ...)
2. Торговая точка (№ торговой точки, № помещения ...)

Отношение «Обрабатывает» (рис. 1.16) удовлетворяет условиям правила 4, в соответствии с которым получаем отношения:

1. Сотрудник (ID сотрудника, ...)
2. Заявка на ремонт (№ заявки, ID сотрудника, ...)

Отношение «Закрепляется за» (рис. 1.17) удовлетворяет условиям правила 4, в соответствии с которым получаем отношения:

1. Помещение (№ помещения, ...)
2. Заявка на ремонт (№ заявки, № помещения, ...)

Отношение «Имеет» (рис. 1.18) удовлетворяет условиям правила 4, в

соответствии с которым получаем отношения:

1. Торговая точка (№ торговой точки, ...)
2. Отзыв (№ отзыва, № торговой точки, ...)

Добавим неключевые атрибуты в каждое из предварительных отношений с условием, чтобы отношения отвечали требованиям третьей нормальной формы:

1. Арендатор (ID арендатора, ФИО, Номер телефона)
2. Договор (№ договора, ID арендатора, № помещения, ID сотрудника, Дата оформления, Дата окончания, Тариф)
3. Сотрудник (ID сотрудника, ФИО, Должность, Номер телефона, Статус)
4. Торговая точка (№ торговой точки, № помещения, Категория торговой точки, Стоимость аренды, Название магазина, Статус договора)
5. Оплата аренды и ком. услуг (№ операции, №договора, Дата оплаты, Сумма)
6. Заявка на ремонт (№ заявки, № помещения, Описание проблемы, Дата подачи, Статус)
7. Бронирование (ID брони, № помещения, ID арендатора, Дата бронирования, Дата истечения, Статус брони)
8. Помещение (№ помещения, Этаж, Площадь, Стоимость аренды, Статус)
9. Отзыв (№ отзыва, № торговой точки, Описание, Дата отзыва, Автор, Оценка)

Определение ограничений предметной области

1. Оформлять договоры аренды и бронировать может только сотрудник с должностью «Менеджер по аренде».
2. При подписании договора аренды статус помещения должен автоматически изменяться со «Свободно» на «Занято».
3. При подтверждении факта окончания договора или разрыва, с любой стороны, статус должен автоматически изменяться с «Занято» на «Свободно».
4. Договор не может быть подписан, если статус помещения «Занято».
5. Статус «Забронированное» действует только в течение суток.
6. Нельзя забронировать помещение, которое имеет статус «Занято».

7. Сотрудник торгового центра должен быть зарегистрирован под уникальным номером и иметь индивидуальный номер телефона.
8. Уволенный сотрудник не может быть задействован при подписании новых договоров и назначении на проведение ремонтов.
9. Производит ремонт помещений может только сотрудник с должностью «Инженер».
10. Стоимость аренды рассчитывается относительно площади помещения, а именно 1200 руб. за м2.
11. По умолчанию текущими являются дата оформления договора, дата оплаты, дата подачи заявки и дата написания отзыва.
12. По умолчанию значения тарифов, стоимости аренды, суммы оплаты, задолженности, штрафов и площади помещения должны быть больше нуля.
13. Длительность договора не меньше 1 месяца и не больше 12 месяцев.
14. В таблице бронирование автоматически меняется статус брони с «Активно» на «Неактивно» при истечении времени бронирования.
15. Если заявка на ремонт активна, то статус помещения меняется на «Ремонт».
16. Подписать договор в помещении, где стоит статус «Ремонт» невозможно.

1.4. ФИЗИЧЕСКОЕ ПРОЕКТИРОВАНИЕ

Этап физического проектирования – это заключительный этап в последовательности построения базы данных. На предыдущем этапе проектирования были получены отношения, которые в данном разделе будут описаны в виде таблиц, представляющие собой модель физической реализации базы данных.

Физическое проектирование базы данных – это реализация базы данных на определённой СУБД (система управления базами данных – комплекс языковых и программных средств, предназначенных для создания, ведения, и совместного ведения БД многими пользователями) и обеспечение всех требований к ней.

На данном этапе создания проекта базы данных принимается решение о

способах реализации разрабатываемой базы данных. Физическое проектирование заключается в объединении логической структуры базы данных и физической среды.

В качестве используемой СУБД была выбрана Microsoft SQL Server, а в качестве среды - Microsoft SQL Management Studio, пригодная для разработки и физического проектирования выбранной в работе базы данных торгового центра. SQL Server Management Studio (SSMS) — это интегрированная среда для управления любой инфраструктурой SQL, от SQL Server до баз данных SQL Azure. SSMS предоставляет средства для настройки, наблюдения и администрирования экземпляров SQL Server и баз данных. Данная СУБД предоставляет возможность с помощью языка SQL создавать, редактировать и заполнять таблицы, выполнять применяемые к созданным таблицам запросы. Также возможна работа с базой данных и через графический интерфейс программы.

Данная среда разработки базы данных была выбрана так как именно она больше всего подходит и позволяет реализовать задуманные в прошлой главе отношения в виде таблиц и заполнить их данными. К тому же является наиболее удобной СУБД с точки зрения уже присутствующего опыта обращения с ней.

Создадим таблицы, на которых будет основана база данных химчистки (табл. 1.1-1.9). После, в приложении А будут указаны операторы на языке SQL для создания таблиц. В приложении Б будут показаны таблицы с заполненными в них данными.

Таблица 1.1

Структура таблицы «Арендатор»

Столбец	Тип данных	Нуль?	Ключ	Значение по умолчанию	Ограничение
ID_арендатора	int	Not null	Первичный		
ФИО	nvarchar (50)	Not null			
Номер_телефона	nvarchar (20)	Not null			UNIQUE

Таблица 1.2

Структура таблицы «Договор»

Столбец	Тип данных	Нуль?	Ключ	Значение по умолчанию	Ограничение	Ссылка
№_договора	int	Not null	Первичный			
ID_арендатора	int	Not null	Внешний			Арендатор (ID_арендатора)
№_помещения	int	Not null	Внешний			Помещение (№_помещения)
ID_сотрудника	int	Not null	Внешний			Сотрудник (ID_сотрудника)
Дата_оформления	date	Not null		Текущая дата	Дата_оформления - Дата_окончания = от 1 до 12 месяцев	
Дата_окончания	date	Not null				
Тариф	Decimal (18, 0)	Not null			>0	

Таблица 1.3

Структура таблицы «Сотрудник»

Столбец	Тип данных	Нуль?	Ключ	Ограничение
ID_сотрудника	int	Not null	Первичный	
ФИО	nvarchar (50)	Not null		
Должность	nvarchar (20)	Not null		Менеджер по аренде, инженер, охранник, клининговый работник
Номер_телефона	nvarchar (20)	Not null		UNIQUE
Статус				Уволен, Активен

Таблица 1.4

Структура таблицы «Торговая точка»

Столбец	Тип данных	Нуль?	Ключ	Ограничение	Ссылка
№_торговой точки	int	Not null	Первичный		
№_помещения	int	Not null	Внешний		Помещение (№_помещения)

Столбец	Тип данных	Ноль?	Ключ	Ограничение	Ссылка
Категория_торговой_точки	nvarchar (50)	Not null			
Стоимость_аренды	Decimal (18, 0)	Not null		>0	
Название_магазина	nvarchar (50)	Not null			
Статус_договора	nvarchar (50)	Not null			Активен,Истек, Расторгнут

Таблица 1.5

Структура таблицы «Оплата аренды и ком. услуг»

Столбец	Тип данных	Ноль?	Ключ	Значение по умолчанию	Ограничение	Ссылка
№_операции	int	Not null	Первичный			
№_договора	int	Not null	Внешний			Договор (№_договора)
Дата_оплаты	date	Not null		Текущая дата		
Сумма	Decimal (18, 0)	Not null			>0	

Таблица 1.6

Структура таблицы «Бронирование»

Столбец	Тип данных	Ноль?	Ключ	Ограничение	Ссылка
ID_брони	int	Not null	Первичный		
№_помещения	int	Not null	Внешний		Помещение (№_помещения)
ID_арендатора	int	Not null			Арендатор (ID_арендатора)
Дата_бронирования	datetime	Not null		<=Текущей даты	
Дата_истечения	datetime	Not null		Дата_истечения - Дата_бронирования = 1 день	
Статус_брони	nvarchar(20)	Not null			

Таблица 1.7

Структура таблицы «Заявка на ремонт»

Столбец	Тип данных	Нуль?	Ключ	Значение по умолчанию	Ссылка
№_заявки	int	Not null	Первичный		
№_помещения	int	Not null	Внешний		Помещение (№_помещения)
Описание_проблемы	nvarchar (500)	Not null			
Дата_подачи	date	Not null		Текущая дата	
Статус	nvarchar (50)	Not null			Неактивно, Активно

Таблица 1.8

Структура таблицы «Помещение»

Столбец	Тип данных	Нуль?	Ключ	Значение по умолчанию	Ограничение
№_помещения	int	Not null	Первичный		
Этаж	int	Not null			
Площадь	int	Not null			>0
Стоимость_аренды	Decimal (18, 0)	Not null			>0
Статус	nvarchar (500)	Not null		Свободное	Свободное, Занятое, Забронированное

Таблица 1.9

Структура таблицы «Отзыв»

Столбец	Тип данных	Нуль?	Ключ	Значение по умолчанию	Ссылка
№_отзыва	int	Not null	Первичный		
№_торговой_точки	int	Not null	Внешний		Торговая точка (№_торговой_точки)
Описание	nvarchar (500)	Not null			
Дата_отзыва	date	Not null		Текущая дата	
Автор	nvarchar (50)	Not null		Анонимный пользователь	
Оценка	int	Not null			1, 2, 3, 4, 5

После того как таблицы созданы и связаны между собой, получается следующая схема базы данных (рис 1.19).

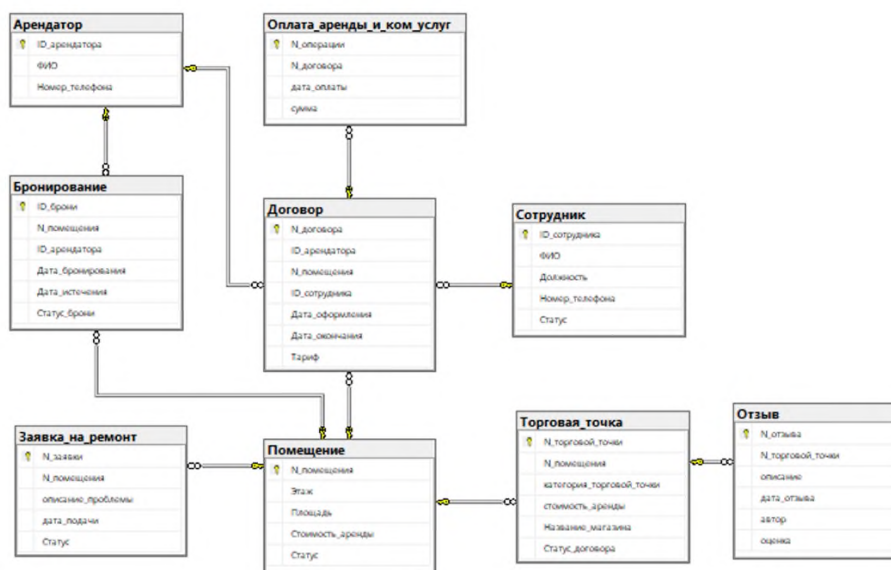


Рис. 1.19 Схема базы торгового центра

На этапе физического проектирования базы данных, помимо таблиц, были разработаны триггеры и хранимая процедура. Их SQL-операторы представлены в Приложении Б.

Триггер «Обновить_Статус_Бронь» автоматически обновляет статусы всех помещений при любых изменениях в таблице бронирований. Срабатывает после операций вставки, обновления или удаления записей в таблице Бронь и вызывает хранимую процедуру для пересчета текущих статусов помещений.

Триггер «Бронь_Проверка_Занятости» проверяет возможность бронирования помещений на основе их текущего статуса. Срабатывает при изменениях в таблице Бронирование и предотвращает бронирование помещений со статусами "Занятое" или "Ремонт", а также автоматически обновляет статусы помещений.

Триггер «Договор_аренды_Проверка_сотрудника» контролирует процесс оформления договоров аренды. Проверяет, что договор может быть оформлен только сотрудником с должностью "Менеджер по аренде", и предотвращает создание договоров другими сотрудниками.

Триггер «Договор_Проверка_Статуса_Помещения_Ремонт» обеспечивает невозможность подписания договоров аренды на помещения, находящиеся на ремонте. Проверяет статус помещения при создании нового договора и отменяет операцию, если помещение имеет статус "Ремонт".

Триггер «Запрет_Уволенного_Сотрудника» предотвращает участие уволенных сотрудников в процессе оформления договоров аренды. Проверяет статус сотрудника при создании договора и отменяет операцию, если сотрудник имеет статус "Уволен".

Триггер «Обновить_Статус_Договор» автоматически пересчитывает статусы помещений при любых изменениях в договорах аренды. Срабатывает после операций с таблицей Договор и вызывает процедуру обновления статусов всех помещений.

Триггер «Проверка_Статуса_Помещения_При_Договоре» проверяет доступность помещения для заключения договора аренды. Предотвращает подписание договора на помещение, которое уже занято, что исключает двойное использование одного помещения.

Триггер «Обновить_Статус_Ремонт» автоматически обновляет статусы помещений при изменениях в заявках на ремонт. Срабатывает при операциях с таблицей Заявки_на_ремонт и вызывает процедуру пересчета всех статусов помещений.

Триггер «Расчет_Аренды» автоматически рассчитывает стоимость аренды помещения на основе его площади. Срабатывает при вставке или обновлении записей в таблице Помещение и устанавливает стоимость как произведение площади на базовый тариф.

Процедура «Пересчитать_Статус_Помещений» пересчитывает текущие статусы всех помещений на основе активных договоров, бронирований и ремонтных работ. Определяет статус как "Ремонт", "Занято", "Забронировано" или "Свободно" в зависимости от текущих активных записей в системе.

УЛАВА 2. ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ БД

2.1 НАЗНАЧЕНИЕ И ПЕРЕЧЕНЬ ФУНКЦИЙ БАЗЫ ДАННЫХ

Взаимодействие пользователей с системой осуществляется через веб-интерфейс, предназначенный для использования на компьютерных устройствах. Для его разработки применялись следующие технологии: интегрированная среда разработки Visual Studio Code [8], язык программирования Python [3], веб-фреймворк Streamlit [9], язык разметки HTML [5], каскадные таблицы стилей CSS [6], а также СУБД Microsoft SQL Server [7].

Как следует из результатов концептуального проектирования, выделены три категории пользователей, взаимодействующих с базой данных аренды торгового центра: клиент (покупатель), арендатор и менеджер по аренде.

Клиенту (Покупателю) доступны следующие функции:

1. Просмотр отзывов – изучение оценок и комментариев других посетителей.
2. Оставление отзывов – возможность оставить свой отзыв с оценкой для любой торговой точки.
3. Просмотр карты торгового центра – интерактивная карта с расположением магазинов по этажам.

Арендатору доступны следующие функции:

1. Личный кабинет – доступ к персональной статистике, активным договорам и торговым точкам.
2. Управление торговыми точками – просмотр списка своих магазинов, их статусов и деталей договоров.
3. Просмотр отзывов на свои точки – мониторинг обратной связи от покупателей.
4. Оплата аренды и коммунальных услуг – внесение платежей через систему, просмотр истории оплат.
5. Расторжение договора – подача заявки на расторжение действующего договора аренды.

6. Бронирование помещения – временная бронь свободного помещения перед заключением договора.

Менеджеру по аренде доступны следующие функции:

1. Управление помещениями – добавление, редактирование, удаление помещений, управление их статусами.
2. Оформление договоров аренды – создание и подписание договоров с арендаторами.
3. Управление бронированиями – создание, продление, отмена броней на помещения.
4. Ведение арендаторов – добавление, редактирование данных арендаторов.
5. Обработка заявок на ремонт – регистрация и контроль выполнения ремонтных работ.
6. Управление персоналом – добавление, увольнение сотрудников, изменение их статусов.
7. Финансовый учёт – просмотр истории оплат, формирование отчётности.
8. Формирование отчётов – экспорт данных по договорам, оплатам и статистике в Excel и PDF.

Код программы представлен в приложении В.

2.2 ОПИСАНИЕ РАБОТЫ С БАЗОЙ ДАННЫХ

При входе в систему управления торговым центром пользователь попадает на страницу авторизации (рис. 2.1). Одновременно с формой входа в боковой панели навигации доступны общедоступные разделы: «Карта ТЦ», «Отзывы» и «Оставить отзыв», позволяющие ознакомиться с инфраструктурой центра и отзывами на торговые точки до входа в систему.

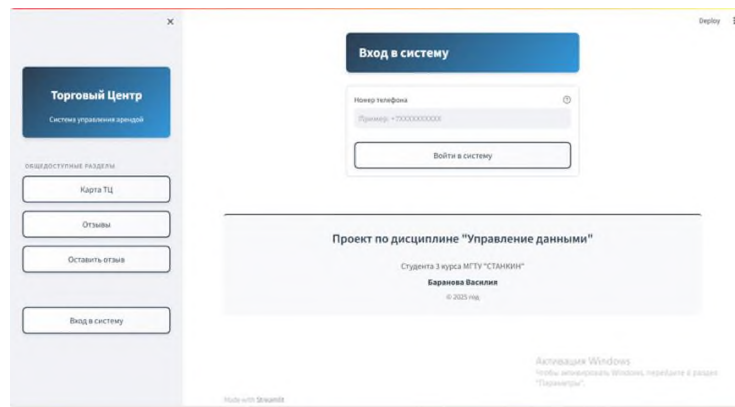


Рис. 2.1 Страница входа в систему

До входа в систему пользователи могут полноценно работать с информацией о торговых точках: просматривать их расположение на интерактивной карте (рис. 2.2), изучать отзывы других посетителей (рис. 2.3) и оставлять собственные отзывы с оценками (рис. 2.4).

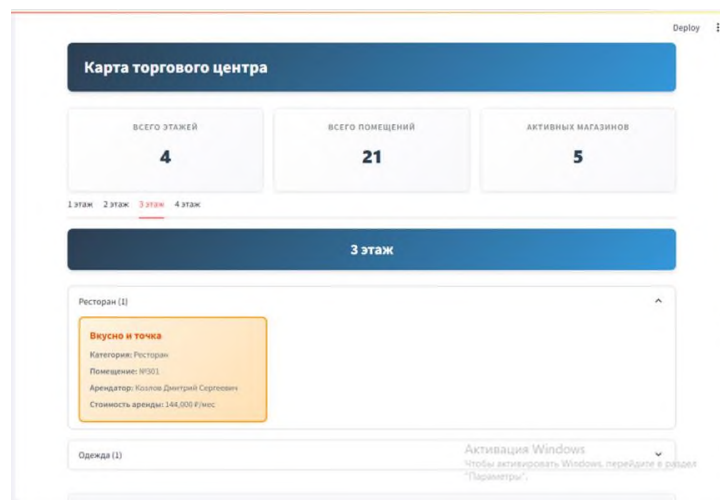


Рис 2.2 Общедоступный раздел карты торгового центра

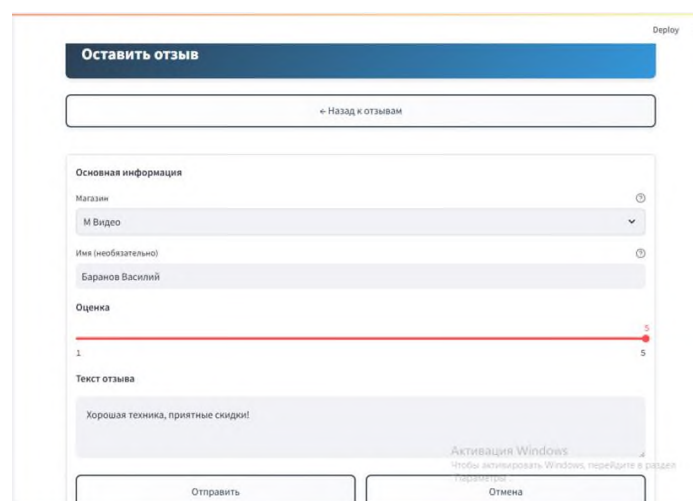


Рис 2.3 Общедоступный раздел добавления отзыва

Раздел «Отзывы» оснащен системой фильтров для удобного поиска, включая фильтрацию по рейтингу (от 1 до 5 звезд), конкретным магазинам и сортировке по дате или оценке. Это позволяет потенциальным арендаторам эффективно анализировать обратную связь и принимать обоснованные решения об аренде.

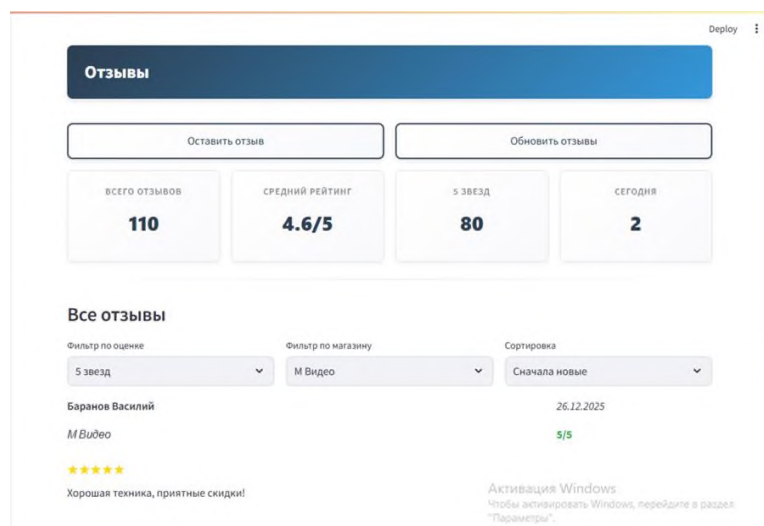


Рис 2.4 Общедоступный раздел взаимодействия с отзывами

После заполнения номера телефона в форме авторизации система проводит уникальную идентификацию пользователя (рис. 2.5). Каждый номер телефона в системе уникален и зарегистрирован исключительно для одного типа пользователя — либо сотрудника, либо арендатора. Система проверяет номер в соответствующих таблицах базы данных и предоставляет доступ в соответствии с определенной ролью.

The form consists of a blue header button labeled 'Вход в систему'. Below it is a white box containing a label 'Номер телефона' and a text input field with the value '+79161234508'. A red border highlights the input field, and a small red box contains the text 'Press Enter to submit form'. At the bottom of the white box is a button labeled 'Войти в систему'.

Рис 2.5 Авторизация пользователя

При попытке входа в систему уволенного сотрудника происходит автоматическая проверка статуса учетной записи (рис. 2.6). Система

идентифицирует номер телефона как принадлежащий сотруднику, но при проверке поля «Статус» в таблице «Сотрудник» обнаруживает значение «Уволен». В этом случае система блокирует вход и отображает соответствующее сообщение об ошибке, информирующее пользователя о невозможности доступа к системе.

Вход в систему

Номер телефона

+79188878363

Войти в систему

Сотрудник Мартынюк Галина Ивановна не активен

Рис 2.6 Авторизация уволенного сотрудника

После успешной авторизации арендатор попадает в личный кабинет (рис. 2.7), который предоставляет полный доступ к управлению его арендной деятельностью. Интерфейс включает панель быстрой статистики с информацией о количестве активных договоров, торговых точках и финансовых обязательствах. Личный кабинет позволяет арендатору оперативно отслеживать состояние всех арендуемых помещений и управлять взаимоотношениями с торговым центром.

Личный кабинет арендатора

активных договоров: 11

торговых точек: 42

долг (₽): 0

Доступные функции

Мои торговые точки

Мои договоры

Отзывы на мои точки

Оплата услуг

Расторжение договора

Ваши данные

Контактная информация

ФИО: Аксенова Мария Сергеевна

Телефон: +79161234508

ID арендатора: 20

Статус

Статус: Активный арендатор

Активация Windows

Вход выполнен: 26.12.2025 23:46

Добро пожаловать в личный кабинет арендатора

Рис 2.7 Личный кабинет арендатора

Также слева расположено боковое меню навигации (рис. 2.8) представляет собой структурированный список доступных функций,

сгруппированных по категориям: «Личный кабинет» для основной информации, «Мои объекты» для управления торговыми точками и договорами, и «Платежи и управление» для финансовых операций и административных действий. Каждый пункт меню ведет к соответствующему разделу системы, обеспечивая быстрый и логичный доступ ко всем возможностям личного кабинета арендатора.

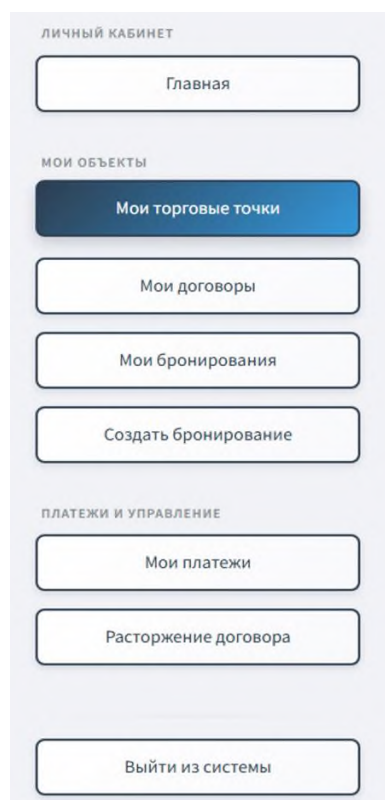


Рис. 2.8 Боковое меню навигации

Раздел «Мои торговые точки» (рис. 2.9) позволяет арендатору управлять своими объектами. С помощью фильтров и сортировки можно быстро найти нужную точку, а в её карточке — увидеть все детали договора и выполнить основные действия: посмотреть отзывы и платежи, а также расторгнуть договор.

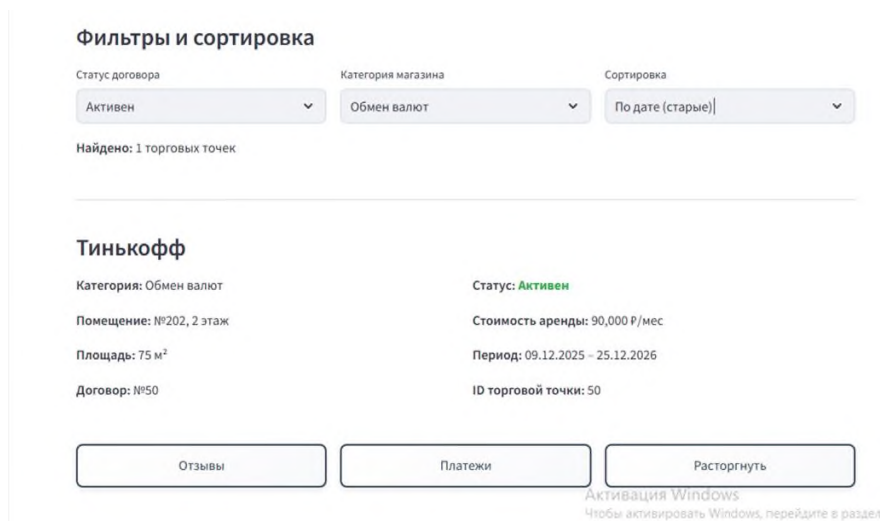


Рис. 2.9 Страница раздела «Мои торговые точки»

При переходе по кнопке «Отзывы» на странице «Мои торговые точки» открывается отдельная страница (рис. 2.10), где представлена сводная статистика и список всех отзывов на выбранную торговую точку. Здесь можно оценить общий рейтинг, количество отзывов и ознакомиться с подробными оценками и комментариями клиентов. В верхней части экрана находится кнопка «Назад к торговым точкам» для быстрого возврата к списку объектов.

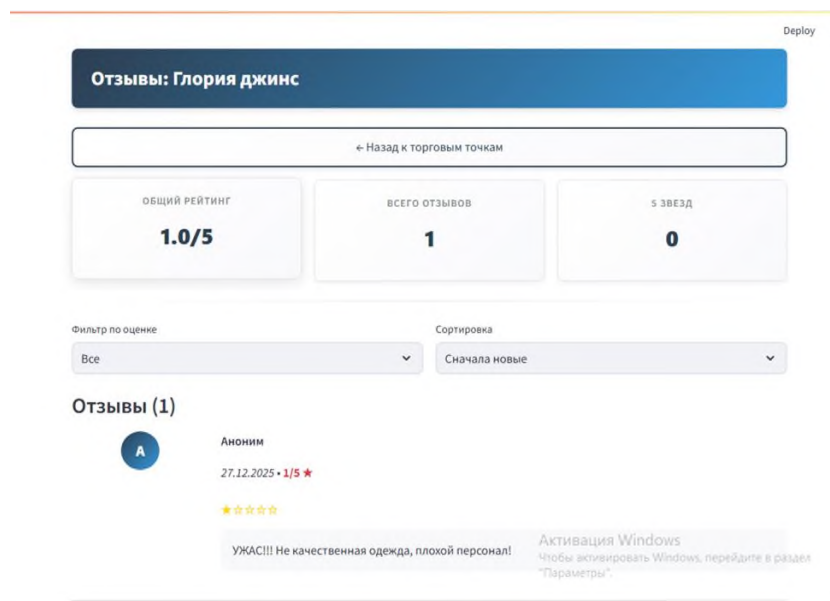


Рис.2.10 Страница раздела «Отзывы» на выбранную торговую точку

При переходе по кнопке «Платежи» на странице «Мои торговые точки» открывается отдельный раздел (рис. 2.11), где отображается история финансовых операций для выбранной точки. Пользователь может увидеть общее количество платежей, их детализацию и статусы. В верхней части

экрана находится кнопка «Назад к торговым точкам» для возврата к списку объектов.

№ операции	Дата оплаты	Сумма
178	25.12.2025	50,000 ₽

Рис 2.11 Страница раздела «Платежи» на выбранную торговую точку

Для инициации новой оплаты внизу страницы «Платежи» на выбранную торговую точку находится кнопка «Совершить новый платеж», которая открывает форму (рис. 2.12) для выбора договора, указания суммы, а дата ставится текущая. Чтобы произвести оплаты нужно нажать на кнопку «Оплатить».

Дата оплаты: 27.12.2025

Рис 2.12 Страница раздела «Совершить новый платеж»

После подтверждения оплаты в форме система отображает страницу с подтверждением успешного проведения платежа (рис. 2.13). А также автоматически генерирует электронный чек с деталями транзакции, который можно сохранить.

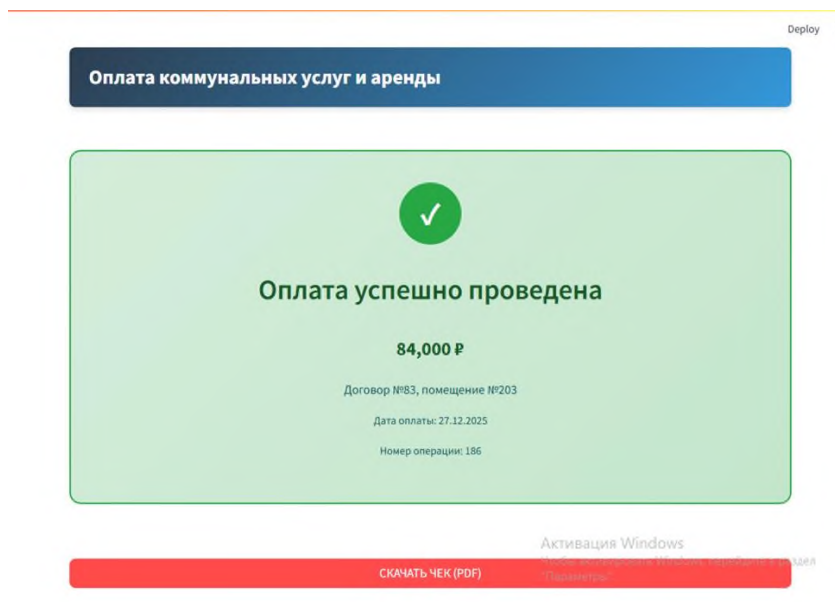


Рис. 2.12 Страница успешной оплаты

При нажатии на кнопку «Скачать чек» скачивается квитанция об оплате в формате PDF, содержащая полную детализацию транзакции: сумму, назначение, реквизиты и уникальный номер.



Рис. 2.13 Электронный чек

При переходе по кнопке «Расторгнуть» на странице «Мои торговые точки» или при выборе раздела «Расторжение» из бокового меню откроется страница подтверждения расторжения договора (рис. 2.14), где нужно выбрать договор и подтвердить необратимое действие.

Выберите договор для расторжения

Договор

Договор №50 - Тинькофф (помещение №202)

Договор №50

Магазин: Тинькофф

Помещение: №202, 2 этаж

Площадь: 75 м²

Стоимость: 90,000 Р/мес

Период: 09.12.2025 - 25.12.2026

Внимание: Договор будет немедленно расторгнут. Это действие необратимо.

Да, расторгнуть договор

Отмена

Рис. 2.14 Страница раздела «Расторжение»

После успешного расторжения система отображает страницу с подтверждением (рис. 2.15), где статус договора изменён на «Расторгнут», и дата окончания договора изменилась на вчерашнюю, а также представлена его финальная информация.

Договор №50 • Расторгнут

Магазин: Тинькофф

Помещение: №202, 2 этаж • Площадь: 75 м²

Период: 09.12.2025 - 26.12.2025

Стоимость: 90,000 Р/мес

Оплатить

Платежи

Рис. 2.15 Страница с подтверждением расторжения договора

Раздел «Мои договоры» (рис. 2.16) предоставляет сводный список всех договоров арендатора. Пользователь может фильтровать договоры по статусу (например, «Истек») и для каждого просмотреть ключевую информацию: данные о магазине, помещении, сроке действия и стоимости, а также выполнить действия по оплате или просмотру платежей. Если пользователю необходимо вернуться в главное меню личного кабинета, для этого в верхней части экрана раздела предусмотрена кнопка «Вернуться в личный кабинет», обеспечивающая быстрый переход.

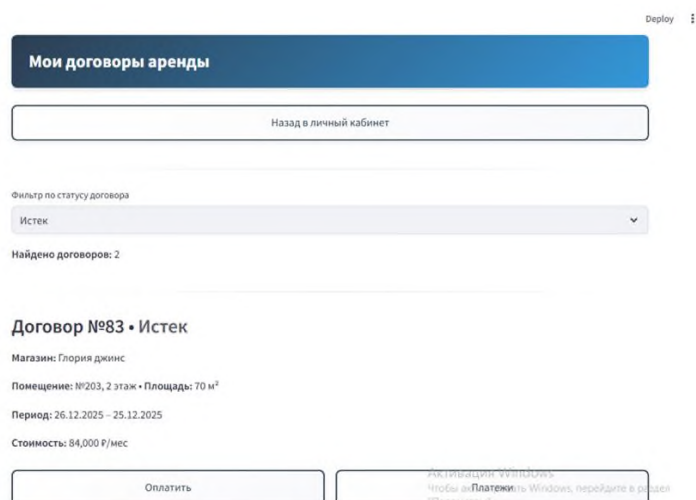


Рис. 2.16 Страница раздела «Мои договоры аренды»

Раздел «Мои бронирования» (рис. 2.17) отображает список всех текущих и завершённых бронирований торговых площадей или услуг. Пользователь может отслеживать статус каждой брони, её период и детали, а также при необходимости создавать новые. Если пользователю необходимо вернуться в главное меню личного кабинета, для этого в верхней части экрана раздела предусмотрена кнопка «Вернуться в личный кабинет», обеспечивающая быстрый переход.

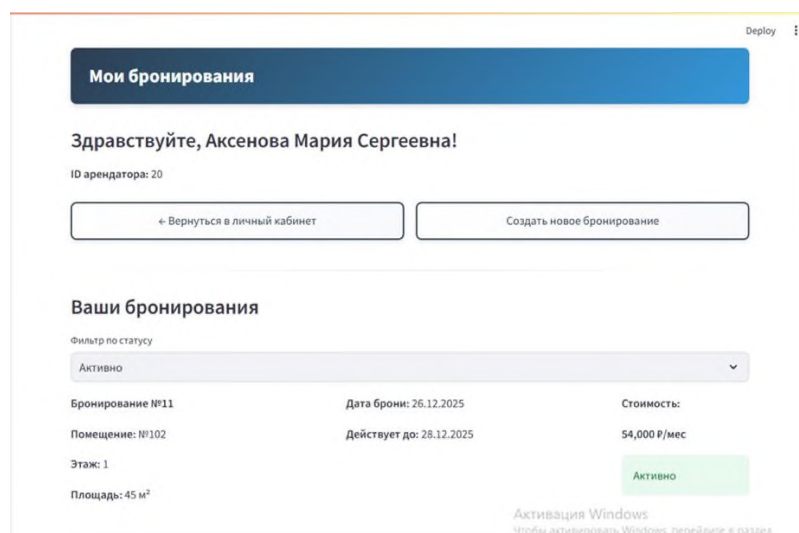
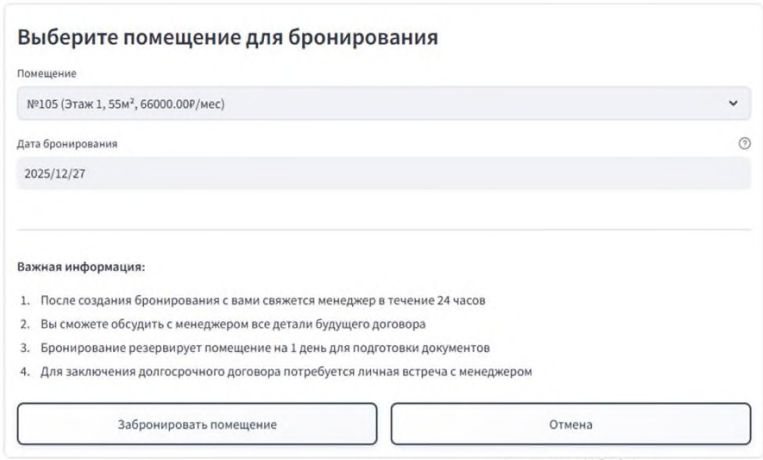


Рис. 2.17 Страница раздела «Мои бронирования»

Для создания нового бронирования необходимо нажать на кнопку «Создать новое бронирование» в верхней части экрана страницы «Мои бронирования» или перейти в раздел «Создать бронирование» из бокового

меню. На открывшейся странице (рис. 2.18) нужно выбрать помещение, указать дату бронирования и ознакомиться с важными условиями, после чего подтвердить действие кнопкой «Забронировать помещение».



Выберите помещение для бронирования

Помещение
№105 (Этаж 1, 55м², 66000.00Р/мес)

Дата бронирования
2025/12/27

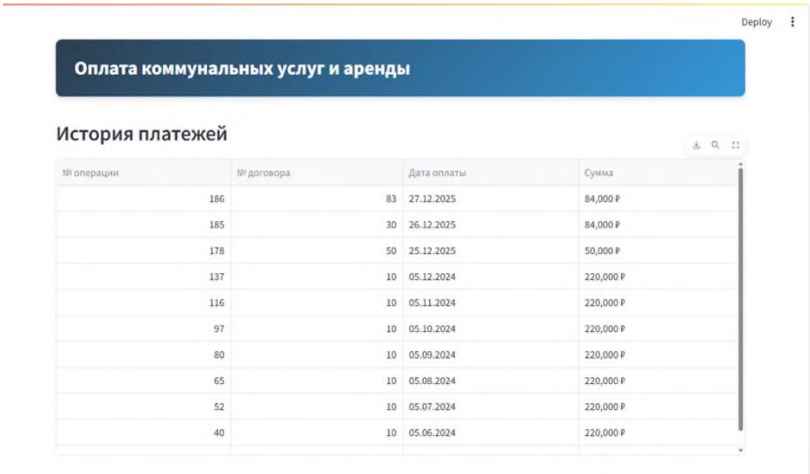
Важная информация:

1. После создания бронирования с вами свяжется менеджер в течение 24 часов
2. Вы сможете обсудить с менеджером все детали будущего договора
3. Бронирование резервирует помещение на 1 день для подготовки документов
4. Для заключения долгосрочного договора потребуются личная встреча с менеджером

Забронировать помещение Отмена

Рис. 2.18 Страница раздела «Создать бронирование»

Раздел «Оплата коммунальных услуг и аренды» (рис. 2.19) предоставляет полную историю всех проведённых платежей в виде таблицы с детализацией по номеру операции, договора, дате и сумме. Также для совершения нового платежа в системе доступна функция «Совершить новый платеж», интерфейс которой представлен на рис. 2.12.



Оплата коммунальных услуг и аренды

История платежей

№ операции	№ договора	Дата оплаты	Сумма
186	83	27.12.2025	84,000 Р
185	30	26.12.2025	84,000 Р
178	50	25.12.2025	50,000 Р
137	10	05.12.2024	220,000 Р
116	10	05.11.2024	220,000 Р
97	10	05.10.2024	220,000 Р
80	10	05.09.2024	220,000 Р
65	10	05.08.2024	220,000 Р
52	10	05.07.2024	220,000 Р
40	10	05.06.2024	220,000 Р

Рис. 2.19 Страница истории оплат

Для завершения работы в личном кабинете необходимо нажать кнопку «Выйти из системы» (рис 2.20), расположенную в боковом меню

навигации. Это действие безопасно завершает текущий сеанс и возвращает пользователя на страницу входа (рис. 2.1).

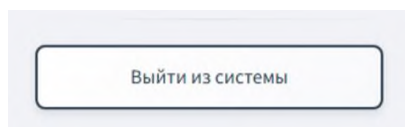


Рис. 2.20 Кнопка выхода из системы

Главная страница личного кабинета сотрудника (рис. 2.21) отображает персональную информацию и права доступа. В левой части представлены основные данные: ФИО, должность, контактные данные, статус и время последнего входа. В правой части перечислены должностные обязанности и права сотрудника, включая управление помещениями, бронированиями, заявками на ремонт, работу с договорами и арендаторами.

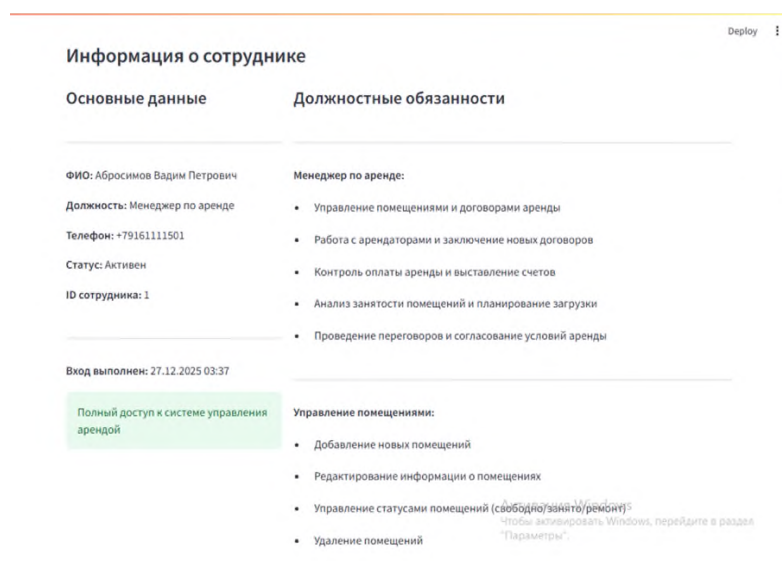


Рис. 2.21 Главная страница личного кабинета сотрудника

Боковое меню сотрудника (рис. 2.22) предоставляет структурированный доступ к разделам системы управления арендой, сгруппированным по трём категориям. «Основное меню» включает функции работы с помещениями, договорами, бронированиями и заявками на ремонт. Раздел «Контрагенты» предназначен для управления арендаторами и сотрудниками, а «Торговля» — для контроля торговых точек и платёжных операций.

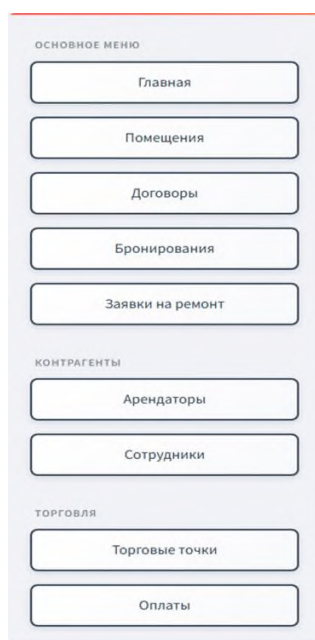


Рис. 2.22 Боковое меню сотрудника

Страница «Управление помещениями» (рис. 2.23) является основным интерфейсом для сотрудника по контролю всех помещений. В верхней части отображается общая статистика (общее количество, свободные, занятые, забронированные помещения и находящиеся в ремонте), а также кнопки для добавления нового помещения и обновления данных. Ниже представлен фильтруемый и сортируемый список всех помещений с ключевой информацией: номер, этаж, площадь, стоимость аренды, статус и данные арендатора.

№ помещения	Этаж	Площадь	Стоимость аренды	Статус	Арендатор (ФИО)
103	1	40,0 м²	48,000 ₽/мес	Занято	Орлов Сергей Викторович

Рис. 2.23 Страница управления помещениями

Ниже на этой же странице, после списка помещений, располагается блок «Детальная информация о помещении» (рис. 2.24), который отображает полные сведения о выбранном в таблице объекте: основные параметры, финансовые условия и детали текущей аренды, включая срок действия договора.

The screenshot shows a web interface titled "Детальная информация о помещении" (Detailed information about the room). Below the title is a dropdown menu labeled "Выберите помещение для просмотра деталей" (Select room for details view) with the value "103" selected. The main content is divided into three sections: "Основная информация" (Basic information) with fields for "Номер помещения: 103", "Этаж: 1", "Площадь: 40 м²", and "Статус: • Занято" (Occupied); "Финансовая информация" (Financial information) with fields for "Стоимость аренды: 48,000 Р/мес", "Стоимость за год: 576,000 Р", and "Стоимость за м²: 1,200 Р/м²"; and "Информация об арендаторе" (Tenant information) with fields for "ФИО арендатора: Орлов Сергей Викторович", "Номер договора: 95", "Дата начала: 26.12.2025", "Дата окончания: 26.12.2026", and "Осталось дней: 364".

Рис. 2.24 Блок детальной информации о помещении на странице управления

При нажатии кнопки «Добавить новое помещение» на странице управления открывается форма (рис. 2.25). В ней сотрудник заполняет обязательные данные о новом объекте: номер помещения, этаж, площадь и стоимость аренды, после чего сохраняет его в системе.

The screenshot shows a web interface titled "Добавление нового помещения" (Add new room). The form contains several input fields: "Номер помещения" (Room number) with the value "501", "Этаж" (Floor) with the value "5", and "Площадь (м²)" (Area in m²) with the value "100,00". There are also dropdown menus for "Статус" (Status) with the value "Свободно" (Free) and "Стоимость аренды (Р/мес)" (Rent cost in RUB/month) with the value "120000". At the bottom of the form are two buttons: "Сохранить помещение" (Save room) and "Отмена" (Cancel).

Рис. 2.25 Форма добавления нового помещения

После заполнения всех полей в форме добавления нового помещения (рис. 2.25) и нажатия кнопки «Сохранить помещение» объект сохраняется в

системе и автоматически отображается в общем списке помещений на странице управления.

Список помещений

Фильтр по статусу: Все | Фильтр по этажу: 5 | Сортировка: По номеру (возр.)

Показано помещений: 1

№ помещения	Этаж	Площадь	Стоимость аренды	Статус	Арендатор (ФИО)
501	5	100.0 м²	120,000 Р/мес	• Свободно	—

Рис. 2.26 Обновлённый список помещений после добавления

Страница «Управление договорами аренды» (рис. 2.27) позволяет сотруднику просматривать, фильтровать и сортировать все договоры. Интерфейс включает панель фильтров (по статусу, арендатору, номеру договора, помещению и дате) и таблицу с ключевыми данными: номер договора, помещение, арендатор, сроки действия, стоимость и статус. Также доступна функция экспорта данных в Excel.

Управление договорами аренды

Все договоры | Новый договор | Расторжение

Список всех договоров

Фильтры договоров

Фильтр по статусу: Активен | Фильтр по арендатору: Аксенова Мария Сергеевна | Фильтр по помещению: Все помещения

Фильтр по дате: Все | Сортировка: По номеру договора

Найдено договоров: 1 из 96

№ договора	№ помещения	Арендатор	Дата начала	Дата окончания	Стоимость аренды	Статус договора
№83	№203	Аксенова Мария Сергеевна	26.12.2025	25.12.2025	84,000 Р/мес	Активен

Экспорт в Excel

Рис 2.27 Страница управления договорами аренды

Для детального анализа и формирования отчётов данные со страницы управления договорами можно выгрузить во внешний файл. При нажатии кнопки «Экспорт в Excel» система формирует и загружает файл (рис. 2.28), содержащий выборку договоров, отфильтрованную по заданным параметрам.

завершить процесс, нажав кнопку «Создать договор». Это действие сохранит новый договор в системе.

The screenshot shows a web application titled 'Управление договорами аренды' (Lease Management). It has three tabs: 'Все договоры' (All contracts), 'Новый договор' (New contract), and 'Расторжение' (Termination). The 'Новый договор' tab is active. Below the tabs is a form titled 'Добавление нового договора' (Add new contract). The form contains several fields: 'Арендатор' (Tenant) with a dropdown menu showing 'Дружко Альбина Васильевна (Тел: +79151234567)'; 'Помещение' (Room) with a dropdown menu showing '№101 (Этаж 1, 50м², 60000.00Р/мес)'; 'Дата начала' (Start date) with a date picker set to '2025/12/27'; 'Дата окончания' (End date) with a date picker set to '2026/12/27'; 'Категория магазина' (Store category) with a dropdown menu showing 'Одежда'; 'Название магазина' (Store name) with a text input field containing 'СекендХенд'. There are two buttons at the bottom: 'Создать договор' (Create contract) and 'Отмена' (Cancel). A watermark 'Активация Windows' is visible in the background.

Рис. 2.31 Заполненная форма создания нового договора

После нажатия кнопки «Создать договор» система обрабатывает данные и, в случае успеха, отображает страницу подтверждения (рис. 2.32). На ней указаны номер нового договора, название магазина, помещение и период его действия. Также предоставляются кнопки для создания следующего договора или возврата к общему списку.

The screenshot shows the confirmation page in the 'Управление договорами аренды' (Lease Management) system. It has the same tabs as the previous screenshot. Below the tabs is a confirmation message: 'Договор успешно оформлен! Создана торговая точка.' (Contract successfully created! Store created.). Below the message are four blue boxes containing the contract details: 'Номер договора: 97' (Contract number: 97), 'Магазин: СекендХенд' (Store: СекендХенд), 'Помещение: №101' (Room: №101), and 'Период: 27.12.2025 - 27.12.2026' (Period: 27.12.2025 - 27.12.2026). At the bottom are two buttons: 'Создать еще один договор' (Create another contract) and 'Вернуться к списку договоров' (Return to contract list). A watermark 'Активация Windows' is visible in the background.

Рис. 2.32 Подтверждение успешного создания договора

После успешного создания договора (рис. 2.32) его данные автоматически обновляются в смежных разделах системы. Новый договор с номером 97 и статусом «Активен» появляется в отфильтрованном списке договоров (рис. 2.33), а соответствующая ему торговая точка «СекендХенд» — в списке торговых точек (рис. 2.34). Одновременно статус помещения №101

на странице управления помещениями изменяется на «Занято», а в графе «Арендатор» указывается новый контрагент (рис. 2.35).

Фильтры договоров

Фильтр по статусу: Активен | Фильтр по арендатору: Все арендаторы | Фильтр по помещению: 101

Фильтр по дате: Все | Сортировка: По номеру договора

Найдено договоров: 1 из 97

№ договора	№ помещения	Арендатор	Дата начала	Дата окончания	Стоимость аренды	Статус договора
№97	№101	Дружко Альбина Васильевна	27.12.2025	27.12.2026	60,000 Р/мес	Активен

Рис. 2.33 Список договоров с новым договором №97

Список торговых точек

Фильтр по категории: Все | Фильтр по номеру точки: 97

№ точки	Помещение	Категория	Стоимость аренды	Название магазина	Статус_договора
97	101	Одежда	60,000 Р/мес	СекендХенд	Активен

Рис. 2.34 Список торговых точек с новой точкой

Список помещений

Фильтр по статусу: Занятое | Фильтр по этажу: 1 | Сортировка: По номеру (возр.)

Показано помещений: 2

№ помещения	Этаж	Площадь	Стоимость аренды	Статус	Арендатор (ФИО)
101	1	50.0 м²	60,000 Р/мес	• Занято	Дружко Альбина Васильевна

Рис. 2.35 Обновлённый статус помещения №101

Для расторжения действующего договора сотрудник переходит на вкладку «Расторжение» в разделе управления договорами. На открывшейся странице (рис. 2.36) необходимо выбрать договор через фильтры (по арендатору или магазину), после чего система отобразит его детальную информацию. Процесс завершается нажатием кнопки «Расторгнуть договор», после чего помещение станет свободным, а статусу договора присвоится значение «Расторгнут».

Выберите договор для расторжения

Фильтр по арендатору

Дружко Альбина Васильевна

Фильтр по магазину

Все магазины

Выберите договор

Договор №97 - Дружко Альбина Васильевна (помещ. №101, СекендХенд) до 27.12.2026

Информация о выбранном договоре

Номер договора: 97

Помещение: №101

Арендатор: Дружко Альбина Васильевна

Этаж: 1

Магазин: СекендХенд

Площадь: 50 м²

Стоимость: 60,000 Р/мес

Внимание: расторжение договора освободит помещение!

Расторгнуть договор

Активация Windows

Параметры

Рис. 2.36 Страница расторжения договора аренды

После успешного расторжения договора система автоматически обновляет все связанные данные. Статус договора №97 в общем списке (рис. 2.37) изменяется на «Расторгнут». Соответствующая торговая точка «СекендХенд» (рис. 2.39) также получает статус «Расторгнут» в своём списке. Помещение №101 на странице управления помещениями (рис. 2.38) освобождается, его статус становится «Свободно», а поле «Арендатор» очищается.

Фильтры договоров

Фильтр по статусу

Расторгнут

Фильтр по арендатору

Все арендаторы

Фильтр по помещению

101

Фильтр по дате

Истекие

Сортировка

По дате начала

Найдено договоров: 2 из 97

№ договора	№ помещения	Арендатор	Дата начала	Дата окончания	Стоимость аренды	Статус договора
№97	№101	Дружко Альбина Васильевна	27.12.2025	26.12.2025	60,000 Р/мес	Расторгнут

Рис. 2.37 Список договоров с расторгнутым договором

Список помещений

Фильтр по статусу: Свободно | Фильтр по этажу: 1 | Сортировка: По номеру (возр.)

Показано помещений: 3

№ помещения	Этаж	Площадь	Стоимость аренды	Статус	Арендатор (ФИО)
101	1	50,0 м²	60,000 Р/мес	● Свободно	—

Рис. 2.38 Помещение №101 со статусом «Свободно»

Список торговых точек

Фильтр по категории: Все | Фильтр по номеру точки: 97

№ точки	Помещение	Категория	Стоимость аренды	Название магазина	Статус_договора
97	101	Одежда	60,000 Р/мес	СекендХенд	Расторгнут

Рис. 2.39 Обновлённый статус торговой точки

Страница «Управление бронированиями» (рис. 2.40) предоставляет сотруднику обзор всех текущих и завершённых бронирований. В верхней части отображается статистика: общее количество, активные и неактивные брони, а также истекающие сегодня. Ниже расположен фильтруемый список с деталями каждого бронирования: номер, помещение, даты, статус и арендатор. Кнопки «Создать новое бронирование» и «Обновить список» обеспечивают управление данными.

Рис. 2.40 Страница управления бронированиями

При нажатии кнопки «Создать новое бронирование» открывается форма (рис. 2.41). В ней сотрудник выбирает арендатора и указывает дату начала

бронирования, которая не может превышать максимальную дату — 2 дня от текущего числа. Система автоматически рассчитывает продолжительность и показывает доступные даты в календаре.

Рис. 2.41 Форма создания бронирования с ограничением в 2 дня

В форме создания бронирования (рис. 2.42) сотрудник дополнительно выбирает конкретное помещение из доступных. Система контролирует, чтобы дата окончания не превышала максимальный срок бронирования (2 дня) и автоматически рассчитывает продолжительность. После заполнения всех обязательных полей и нажатия кнопки «Создать бронирование», данные сохраняются.

Рис. 2.42 Форма создания бронирования с выбором помещения

После успешного создания бронирования статус выбранного помещения (№101) на странице управления помещениями автоматически изменяется на «Забронировано», а в графе «Арендатор» указывается соответствующий контрагент (рис. 2.43).

Список помещений

Фильтр по статусу: Забронировано | Фильтр по этажу: 1 | Сортировка: По номеру (возр.)

Показано помещений: 1

№ помещения	Этаж	Площадь	Стоимость аренды	Статус	Арендатор (ФИО)
101	1	50.0 м²	60,000 Р/мес	● Забронировано	Аксенова Мария Сергеевна

Рис. 2.43 Обновлённый статус помещения после бронирования

Страница «Заявки на ремонт» (рис. 2.44) позволяет сотруднику отслеживать все обращения по техническим неисправностям. Верхний блок содержит общую статистику: общее количество заявок, активных и новых на сегодня. Ниже расположен фильтруемый список всех заявок с информацией о номере, помещении, описании проблемы, дате подачи и текущем статусе. Кнопки «Создать новую заявку» и «Обновить список» обеспечивают управление процессом.

Deploy

Заявки на ремонт

Создать новую заявку Обновить список

ВСЕГО ЗАЯВОК
109

АКТИВНЫХ
1

НОВЫХ СЕГОДНЯ
1

Список заявок на ремонт

Фильтр по статусу: Активна

№ заявки	№ помещения	Описание проблемы	Дата подачи	Статус
109	401	Течет кран	27.12.2025	Активна

Рис. 2.44 Страница управления заявками на ремонт

Для регистрации новой проблемы сотрудник нажимает кнопку «Создать новую заявку», после чего открывается форма (рис. 2.45). В ней необходимо выбрать помещение, подробно описать проблему (например, «Не работает свет») и установить начальный статус (например, «Активна»). После заполнения данных заявка создаётся нажатием кнопки «Создать заявку» и добавляется в общий список.

Создание новой заявки на ремонт

Помещение

№105 (Этаж 1, 55м²) ▾

Статус

Активна ▾

Описание проблемы

Не работает свет

Press Ctrl+Enter to submit form

Создать заявку

Отмена

Рис. 2.45 Форма создания новой заявки на ремонт

После создания новая заявка на ремонт сразу появляется в отфильтрованном списке активных обращений (рис. 2.46). В таблице отображаются все ключевые детали: номер заявки и помещения, описание проблемы, дата подачи и статус.

Список заявок на ремонт

Фильтр по статусу

Активна ▾

↑ № заявки	№ помещения	Описание проблемы	Дата подачи	Статус
109	401	Течет кран	27.12.2025	Активна

Рис. 2.46 Заявка на ремонт добавлена в список

Одновременно с созданием заявки статус соответствующего помещения на странице управления помещениями автоматически изменяется на «Ремонт» (рис. 2.47). Это указывает на то, что объект временно не доступен для аренды или бронирования.

Список помещений

Фильтр по статусу: Ремонт | Фильтр по этажу: 4 | Сортировка: По номеру (возр.)

Показано помещений: 1

№ помещения	Этаж	Площадь	Стоимость аренды	Статус	Арендатор (ФИО)
401	4	150.0 м²	180,000 Р/мес	Ремонт	—

Рис. 2.47 Помещение со статусом «Ремонт»

Страница «Управление арендаторами» (рис. 2.48) отображает полный список всех контрагентов. В верхней части представлена статистика: общее количество арендаторов и число тех, у кого есть действующие договоры. Ниже размещена таблица с данными: ID, ФИО и номер телефона каждого арендатора. Интерфейс позволяет сотруднику быстро находить нужного контрагента и управлять их данными.

Deploy

Управление арендаторов

Добавить нового арендатора | Обновить список

ВСЕГО АРЕНДАТОРОВ

20

С ДОГОВОРАМИ

20

Список арендаторов

ID	ФИО	Телефон
20	Аксенова Мария Сергеевна	+79161234508
5	Баранов Семен Семенович	+79595678901
11	Дружко Альбина Васильевна	+79151234567
4	Дударь Семен Олегович	+79484567890
10	Захаров Игорь Дмитриевич	+79040123456
19	Калинина Ульяна Михайловна	+79161234507

Рис. 2.48 Страница управления арендаторами

При попытке добавить нового арендатора система проверяет уникальность номера телефона в базе данных. Если арендатор с таким номером уже существует, операция прерывается, и под формой появляется сообщение об ошибке (рис. 2.49). Это предотвращает создание дубликатов и сохраняет целостность данных.

Добавление нового арендатора

ФИО: Мхеева София Сергеевна

Номер телефона: +79373456789

Добавить арендатора

Отмена

Ошибка при добавлении арендатора

Рис. 2.49 Ошибка при добавлении из-за неуникального номера телефона

После исправления данных — ввода нового, уникального номера телефона — форма становится готовой к успешному завершению операции (рис. 2.50). При нажатии кнопки «Добавить арендатора» система сохранит нового контрагента в базу данных.

Добавление нового арендатора

ФИО: Мхеева София Сергеевна

Номер телефона: +79373456700

Добавить арендатора

Отмена

Рис. 2.50 Форма с исправленными данными для добавления арендатора

После успешного добавления нового арендатора общая статистика в разделе «Управление арендаторами» обновляется, а сам арендатор появляется в общем списке контрагентов (рис. 2.51). В таблице отображаются его ФИО и номер телефона, что подтверждает корректное сохранение данных в системе.

Управление арендаторов

Добавить нового арендатора

Обновить список

ВСЕГО АРЕНДАТОРОВ: 21

С ДОГОВОРАМИ: 20

Список арендаторов

ID	ФИО	Телефон
21	Мхеева София Сергеевна	+79373456700

2.51 Обновлённый список арендаторов после добавления

Раздел «Управление сотрудниками» (рис. 2.52) позволяет администратору просматривать и управлять персоналом системы. Интерфейс включает кнопки для добавления нового сотрудника и обновления списка, а также панель фильтров (по статусу и должности). В таблице представлена информация о каждом сотруднике: ID, ФИО, должность, контактный телефон и текущий статус активности.

Управление сотрудниками

Добавить нового сотрудника Обновить список

ВСЕГО СОТРУДНИКОВ: 7 АКТИВНЫХ: 6 НЕАКТИВНЫХ: 0

Список сотрудников

Фильтр по статусу: Активен Фильтр по должности: Менеджер по аренде Сортировка: По ФИО (А-Я)

Показано сотрудников: 4

ID_сотрудника	ФИО	Должность	Номер_телефона	Статус
1	Абросимов Вадим Петрович	Менеджер по аренде	+79161111501	Активен
2	Богатырев Глеб Игоревич	Менеджер по аренде	+79161111502	Активен

Рис. 2.52 Страница управления сотрудниками

Для регистрации нового сотрудника администратор заполняет форму (рис. 2.53), указывая ФИО, должность, номер телефона и статус. Аналогично добавлению арендатора, система проверяет уникальность номера телефона в базе данных. При попытке ввести уже существующий номер операция будет прервана с выводом соответствующей ошибки.

Добавление нового сотрудника

ФИО * Номер телефона *

Яшкин Александр Михайлович +79876567887

Должность * Статус

Администратор Активен

Добавить сотрудника Отмена

Рис. 2.53 Форма добавления нового сотрудника

После успешного добавления сотрудника с уникальными данными, он появляется в общем списке персонала (рис. 2.54). Таблица автоматически обновляется, отражая нового сотрудника с указанными ФИО, должностью, номером телефона и статусом.

Список сотрудников

Фильтр по статусу: Активен | ▼ Фильтр по должности: Администратор | ▼ Сортировка: По ФИО (А-Я) | ▼

Показано сотрудников: 1

ID_сотрудника	ФИО	Должность	Номер_телефона	Статус
8	Яшкин Александр Михайлович	Администратор	+79876567887	● Активен

Рис. 2.54 Обновлённый список сотрудников после добавления

Раздел «Управление торговыми точками» (рис. 2.55) предоставляет сотруднику обзор всех зарегистрированных объектов торговли. В верхней части отображается статистика: общее количество точек и категорий. Ниже расположена таблица со списком точек, которую можно фильтровать по категории и номеру. Для каждой точки указаны ключевые параметры: номер, помещение, категория, стоимость аренды, название магазина и статус договора. Кнопки «Добавить новую торговую точку» и «Обновить список» обеспечивают управление данными.

Deploy ⋮

Управление торговыми точками

Добавить новую торговую точку Обновить список

ВСЕГО ТОЧЕК

97

КАТЕГОРИЙ

82

Список торговых точек

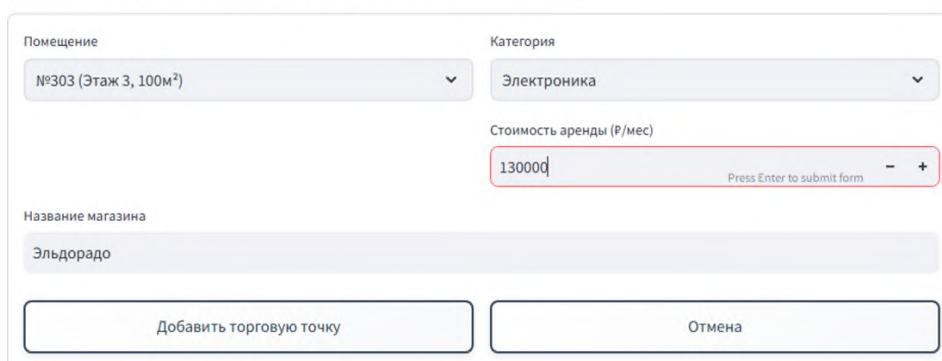
Фильтр по категории: Продуктовый магазин | ▼ Фильтр по номеру точки: Все | ▼

№ точки	Помещение	Категория	Стоимость аренды	Название магазина	Статус_договора
1	401	Продуктовый магазин	180,000 Р/мес	Пятёрочка	Расторгнут

Рис. 2.55 Страница управления торговыми точками

Раздел «Добавление новой торговой точки» (рис. 2.56) позволяет сотруднику зарегистрировать новый объект торговли. В форме необходимо выбрать помещение, указать его категорию и задать название магазина. Поле «Стоимость аренды» является редактируемым — сотрудник может изменить предложенную системой базовую стоимость в соответствии с условиями договора. После заполнения данных торговая точка создаётся нажатием кнопки «Добавить торговую точку».

Добавление новой торговой точки



Помещение: №303 (Этаж 3, 100м²)

Категория: Электроника

Стоимость аренды (Р/мес): 130000

Название магазина: Эльдорадо

Добавить торговую точку

Отмена

Рис. 2.56 Форма добавления новой торговой точки

После успешного добавления новой торговой точки система автоматически обновляет данные в разделе управления. Пользователь может убедиться в корректности операции, применив фильтр по соответствующей категории. В отфильтрованном списке будет отображена новая точка с указанием её номера, помещения, стоимости аренды, названия и актуального статуса договора.

Список торговых точек

Фильтр по категории		Фильтр по номеру точки			
Электроника		98			
№ точки	Помещение	Категория	Стоимость аренды	Название магазина	Статус_договора
98	303	Электроника	130,000 Р/мес	Эльдорадо	Активен

Рис. 2.57 Страница управления торговыми точками после добавления новой

Раздел «Оплаты аренды и услуг» (рис. 2.58) предоставляет сотруднику финансовый обзор. В верхней части отображается сводная статистика: общее количество оплат, сумма за месяц, средний и максимальный платёж. Ниже расположена панель фильтров, позволяющая настроить выборку по периоду и номеру договора, а также кнопки для добавления новой оплаты и обновления данных.

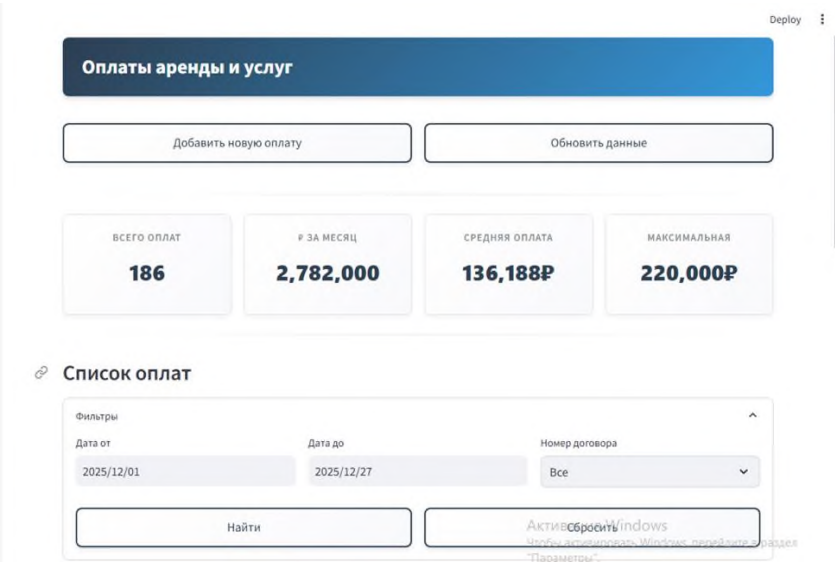


Рис. 2.58 Страница оплаты аренды и услуг

После применения фильтров система формирует детализированный список соответствующих оплат (рис. 2.59). Для дальнейшей работы с данными предусмотрена функция экспорта — полученную выборку можно скачать в формате Excel и PDF для анализа или отчётности.

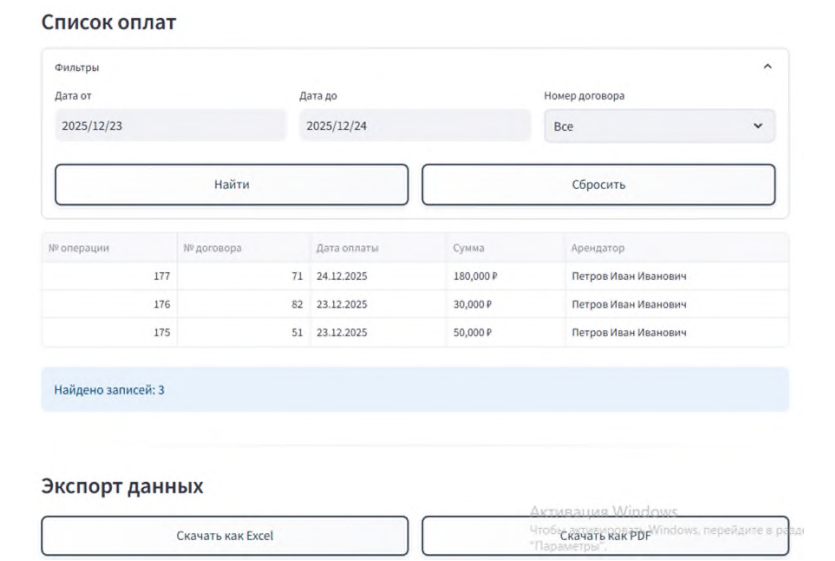


Рис. 2.59 Результаты фильтрации и экспорт данных

При использовании функции «Скачать как Excel» система формирует и загружает файл (рис. 2.60), содержащий структурированную таблицу с данными об оплатах. Файл включает все колонки из списка системы (№ операции, № договора, дата, сумма, арендатор) и готов для дальнейшего анализа или архивации.

	A	B	C	D	E
1	№ операции	№ договора	Дата оплаты	Сумма	Арендатор
2	177	71	24.12.2025	180000	Петров Иван Иванович
3	176	82	23.12.2025	30000	Петров Иван Иванович
4	175	51	23.12.2025	50000	Петров Иван Иванович

Рис. 2.60 Файл Excel с экспортированными данными об оплатах

Для формирования официального документа система также позволяет создать и скачать структурированный отчет в формате PDF (рис. 2.61). Он включает сводную информацию: период, количество записей, общую сумму, данные составителя, детальную таблицу оплат, дату формирования и поле для подписи. Такой PDF-отчёт может использоваться для внутреннего документооборота или предоставления контрагентам.

Отчет по оплатам

Период: 23.12.2025 - 24.12.2025
 Количество записей: 3
 Общая сумма: 260,000.00 Р
 Отчет составил: Богатырев Глеб Игоревич

№ операции	№ договора	Дата оплаты	Сумма	Арендатор
177	71	24.12.2025	180,000.00 Р	Петров Иван Иванович
176	82	23.12.2025	30,000.00 Р	Петров Иван Иванович
175	51	23.12.2025	50,000.00 Р	Петров Иван Иванович

Сформировано: 27.12.2025 07:50
 Подпись: _____
 Отчет составил: Богатырев Глеб Игоревич

Рис. 2.61 PDF-отчёт по оплатам за выбранный период

Для регистрации платежа вне онлайн-оплаты сотрудник может использовать функцию «Добавить новую оплату». В открывшейся форме (рис.

2.62) необходимо выбрать договор, указать дату и сумму оплаты, после чего сохранить данные.

The form is titled "Договор" (Contract) and contains a dropdown menu with the selected value "№83 - Аксенова Мария Сергеевна". Below this, there are two input fields: "Дата оплаты" (Payment date) with the value "2025/12/27" and "Сумма (₽)" (Sum in rubles) with the value "120000". The sum field has a red border and a small text "Press Enter to submit form" with minus and plus icons. At the bottom, there are two buttons: "Сохранить" (Save) and "Отмена" (Cancel).

Рис. 2.62 Форма для ручного добавления новой оплаты

После сохранения новая оплата автоматически появляется в общем списке. При применении фильтра по соответствующему периоду можно убедиться, что платёж корректно добавлен: в таблице отображается новая операция с указанием номера договора, даты, суммы и арендатора (рис. 2.66)

Список оплат

Фильтры

Дата от: 2025/12/27 Дата до: 2025/12/29 Номер договора: Все

Найти Сбросить

№ операции	№ договора	Дата оплаты	Сумма	Арендатор
187	83	27.12.2025	120,000 ₽	Аксенова Мария Сергеевна

Рис. 2.63 Список оплат с добавленным платежом

Для завершения сеанса работы сотрудник нажимает кнопку «Выйти из системы», расположенную в боковом меню (аналогично рис. 2.20 для арендатора). Это действие безопасно завершает текущий сеанс и перенаправляет пользователя на страницу входа в систему.

ЗАКЛЮЧЕНИЕ

В проекте была рассмотрена тема проектирования базы данных для системы управления арендой помещений торгового центра. Современные торговые центры требуют эффективного управления арендными отношениями, учёта помещений, контроля договоров, автоматизации платежей и анализа деятельности арендаторов. Ручное управление этими процессами приводит к увеличению временных затрат, ошибкам в учёте и снижению качества обслуживания как арендаторов, так и посетителей.

В ходе выполнения проекта были получены следующие результаты:

1. Анализ предметной области – изучены существующие программные решения для управления торговыми центрами, включая «1С:Управление торговым центром» [1] и «ArendaSoft» [2]. Проведённый анализ позволил выделить ключевые функции, которые были реализованы в проекте: ведение данных арендаторов, контроль договоров аренды, учёт платежей, управление помещениями и формирование отчётности.
2. Концептуальное проектирование – определены основные роли пользователей системы: покупатель (клиент), арендатор и менеджер по аренде. Построена диаграмма вариантов использования, которая отражает взаимодействие пользователей с системой и выделены ключевые сущности будущей базы данных: арендаторы, договоры, помещения, торговые точки, оплаты, бронирования и отзывы.
3. Логическое проектирование – разработаны ER-диаграммы, отражающие взаимосвязи между сущностями, определены их атрибуты, типы связей и ограничения предметной области. На основе диаграмм сформирована нормализованная схема базы данных, соответствующая требованиям третьей нормальной формы.
4. Физическое проектирование – осуществлено создание таблиц, установление связей между ними, реализация ограничений целостности и бизнес-правил с использованием триггеров и хранимых процедур. В качестве среды разработки выбрана Microsoft SQL Server Management Studio [7], с помощью

которой были созданы и заполнены тестовыми данными все необходимые таблицы.

5. Реализация программного продукта – разработан и развёрнут веб-интерфейс системы управления арендой торгового центра. Для реализации использовались: серверная логика на Python с фреймворком Streamlit, фронтенд на HTML [5] и CSS [6], а также Microsoft SQL Server [7] в качестве СУБД. Интерфейс предоставляет персонализированный доступ для всех трёх категорий пользователей:

- Покупателям доступны просмотр торговых точек, отзывов и карты ТЦ.
- Арендаторам – личный кабинет, управление точками, оплата услуг и расторжение договоров.
- Менеджерам по аренде – полный контроль за помещениями, договорами, бронированиями, ремонтами и отчётностью.

Была реализована система авторизации по номеру телефона, фильтрация данных, формирование чеков и экспорт отчётов в форматах Excel и PDF. Работа интерфейса детально проиллюстрирована в главе 2.

Таким образом, в рамках работы была успешно спроектирована, реализована и протестирована база данных и веб-приложение для автоматизации управления арендой помещений торгового центра. Система позволяет оптимизировать бизнес-процессы, повысить прозрачность взаимодействия с арендаторами и улучшить качество обслуживания посетителей.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Программный продукт «1С:Управление торговым центром» [Электронный ресурс] – Режим доступа: <https://solutions.1c.ru/catalog/>, свободный. Дата обращения: 21.12.2025 г.
2. Программный продукт «ArendaSoft» [Электронный ресурс] – Режим доступа: <https://arendasoft.ru/demo/>, свободный. Дата обращения: 21.12.2025 г.
3. Официальная документация Python [Электронный ресурс] – Режим доступа: <https://docs.python.org/3/>, свободный. Дата обращения: 22.12.2025 г.
4. Справочник по Transact-SQL (T-SQL) [Электронный ресурс] – Режим доступа: <https://learn.microsoft.com/ru-ru/sql/t-sql/language-reference>, свободный. Дата обращения: 22.12.2025 г.
5. MDN Web Docs: Справочник по HTML [Электронный ресурс] – Режим доступа: <https://developer.mozilla.org/ru/docs/Web/HTML>, свободный. Дата обращения: 22.12.2025 г.
6. Сайт «Справочник по CSS» [Электронный ресурс] – Режим доступа: <https://developer.mozilla.org/ru/docs/Web/CSS>, свободный. Дата обращения: 23.11.2025 г.
7. Документация по Microsoft SQL Server [Электронный ресурс] – Режим доступа: <https://learn.microsoft.com/ru-ru/sql/sql-server/>, свободный. Дата обращения: 23.12.2025 г.
8. Документация по Visual Studio Code [Электронный ресурс] – Режим доступа: <https://code.visualstudio.com/docs>, свободный. Дата обращения: 24.12.2025 г.
9. Официальная документация Streamlit [Электронный ресурс] – Режим доступа: <https://docs.streamlit.io>, свободный. Дата обращения: 25.12.2025 г.

Заполнение таблиц данными

	ID_арендатора	ФИО	Номер_телефона
	1	Петров Иван Иванович	+79158046362
	2	Сушкина Мария Петровна	+79262345678
	3	Сидорова Анна Михайловна	+79373456789
	4	Дударь Семен Олегович	+79484567890
	5	Баранов Семен Семеныч	+79595678901
	6	Третьяков Андрей Иванович	+79606789012
	7	Мышкин Константин Никитьевич	+79717890123
	8	Орлов Сергей Викторович	+79828901234
	9	Ступак Павел Анатольевич	+79939012345
	10	Захаров Игорь Дмитриевич	+79040123456
	11	Дружко Альбина Васильевна	+79151234567
	12	Облачко Максим Степанович	+79373456781
	13	Шкурко Марина Сергеевна	+79161234501
	14	Петров Петр Петрович	+79161234502
	15	Смирнова Лариса Витальевна	+79161234503
	16	Козлов Дмитрий Сергеевич	+79161234504
	17	Смирнова Елена Владимировна	+79161234505
	18	Федоров Алексей Николаевич	+79161234506
	19	Калинина Ульяна Михайловна	+79161234507
▶	20	Аксенова Мария Сергеевна	+79161234508
*	NULL	NULL	NULL

Рис. А.1 Данные в таблице «Арендатор»

	N_договора	ID_арендатора	N_помещения	ID_сотрудника	Дата_оформления	Дата_окончания	Тариф
	64	14	401	1	2025-03-30	2025-06-30	180000,0
	65	15	102	2	2025-05-05	2025-08-05	54000,0
	66	16	202	3	2025-06-10	2025-09-10	90000,0
	67	17	302	1	2025-07-15	2025-10-15	132000,0
	68	18	402	2	2025-08-20	2025-11-20	168000,0
	69	19	103	3	2025-09-25	2025-12-25	48000,0
	70	20	203	1	2025-10-30	2025-12-25	84000,0
	71	1	401	2	2025-11-05	2025-12-25	180000,0
	72	2	102	3	2025-01-10	2025-04-10	54000,0
	73	3	202	1	2025-02-15	2025-05-15	90000,0
	74	4	302	2	2025-03-20	2025-06-20	132000,0
	75	5	402	3	2025-04-25	2025-07-25	168000,0
	76	6	103	1	2025-05-30	2025-08-30	48000,0
	77	7	203	2	2025-07-05	2025-10-05	84000,0
	78	8	303	3	2025-08-10	2025-11-10	120000,0
	79	9	403	1	2025-09-15	2025-12-15	162000,0
	80	10	104	2	2025-10-20	2025-12-25	72000,0
	81	11	204	3	2025-11-25	2025-12-25	102000,0
	82	1	406	2	2025-11-19	2025-12-25	30000,0
	83	20	203	2	2025-12-26	2025-12-25	84000,0
	84	20	105	2	2025-12-26	2025-12-25	66000,0
	85	20	201	2	2025-12-26	2025-12-25	96000,0
	86	20	202	2	2025-12-26	2025-12-25	90000,0
	87	20	204	2	2025-12-26	2025-12-25	102000,0
	88	16	301	2	2025-12-26	2026-12-25	144000,0
	89	20	303	2	2025-12-26	2026-12-25	120000,0
	90	5	104	2	2025-12-26	2025-12-25	72000,0
	91	20	105	1	2025-12-26	2026-12-26	66000,0
▶*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Рис. А.2 Данные в таблице «Договор»

	ID_сотрудника	ФИО	Должность	Номер_телефона	Статус
	1	Абросимов Вадим Петрович	Менеджер по аренде	+79161111501	Активен
	2	Богатырев Глеб Игоревич	Менеджер по аренде	+79161111502	Активен
	3	Вельяминова София Максимовна	Менеджер по аренде	+79161111503	Активен
	4	Головин Платон Михайлович	Инженер	+79161111504	Активен
	5	Демьянова Злата Денисовна	Инженер	+79161111505	Активен
	6	Мартынюк Галина Ивановна	Администратор	+79188878363	Уволен
	7	Пичкова Валерия Владимировна	Менеджер по аренде	+89102302323	Активен
▶*	NULL	NULL	NULL	NULL	NULL

Рис. А.3 Данные в таблице «Сотрудник»

	N_торговой_точки	N_помещения	категория_торговой_точки	стоимость_аренды	Название_магазина	Статус_договора
	66	202	Сапожная мастерская	90000,00	Ремонт обуви	Расторгнут
	67	302	Туристическое агентство	132000,00	Турагентство	Истек
	68	402	Стоматология	168000,00	Стоматология	Истек
	69	103	Лавка подарков	48000,00	Подарки	Расторгнут
	70	203	Ателье	84000,00	Ателье	Расторгнут
	71	401	Банковский офис	180000,00	Тинькофф	Расторгнут
	72	102	Канцелярия	54000,00	Офисмаг	Расторгнут
	73	202	Ремонт часов	90000,00	Ремонт часов	Расторгнут
	74	302	Страховой агент	132000,00	Росгосстрах	Истек
	75	402	Медицинская лаборатория	168000,00	Гемотест	Истек
	76	103	Табачная лавка	48000,00	Сигареты-табак	Истек
	77	203	Парикмахерская	84000,00	Парикмахерская	Расторгнут
	78	303	Бьюти-салон	120000,00	Салон красоты	Расторгнут
	79	403	Юридическая консультация	162000,00	Юридическая помощь	Истек
	80	104	Кофе-точка	72000,00	Кофе с собой	Расторгнут
	81	204	Цветочный киоск	102000,00	Цветочный ларек	Расторгнут
	82	406	Кофейня	30000,00	Шоколадница	Расторгнут
	83	203	Одежда	84000,00	Глория джинс	Расторгнут
	84	105	Одежда	66000,00	Глория джинс	Расторгнут
	85	201	Одежда	96000,00	Глория джинс	Расторгнут
	86	202	Одежда	90000,00	Глория джинс	Расторгнут
	87	204	Одежда	102000,00	Глория джинс	Расторгнут
	88	301	Ресторан	144000,00	Вкусно и точка	Активен
	89	303	Одежда	120000,00	Джинсомания	Активен
	90	104	Одежда	72000,00	СекендХенд	Расторгнут
▶	91	105	Продукты	66000,00	Пятёрочка	Активен
*	NULL	NULL	NULL	NULL	NULL	NULL

Рис. А.4 Данные в таблице «Торговая точка»

	N_операции	N_договора	дата_оплаты	сумма
	167	47	2025-12-18	50000,00
	168	47	2025-12-18	50000,00
	169	47	2025-12-18	50000,00
	170	47	2025-12-18	50000,00
	171	27	2025-12-18	50000,00
	172	81	2025-12-18	50000,00
	173	71	2025-12-18	180000,00
	174	82	2025-12-18	180000,00
	175	51	2025-12-23	50000,00
	176	82	2025-12-23	30000,00
	177	71	2025-12-24	180000,00
	178	50	2025-12-25	50000,00
	179	51	2025-12-25	120000,00
	180	51	2025-12-25	50000,00
	181	49	2025-12-26	54000,00
	182	29	2025-12-26	84000,00
	183	52	2025-12-26	162000,00
*	NULL	NULL	NULL	NULL

1 of 183

Рис. А.5 Данные в таблице «Оплата аренды и ком. услуг»

	ID_брони	N_помещения	ID_арендатора	Дата_бронирования	Дата_истечения	Статус_брони
▶	1	305	15	2025-11-17	2025-11-18	Неактивно
	2	205	3	2025-11-20	2025-11-21	Неактивно
	3	402	16	2025-12-14	2025-12-15	Неактивно
	4	302	14	2025-12-17	2025-12-19	Неактивно
	5	302	1	2025-12-18	2025-12-20	Неактивно
	6	302	4	2025-12-23	2025-12-25	Неактивно
	7	305	5	2025-12-24	2025-12-25	Неактивно
	8	402	1	2025-12-22	2025-12-24	Неактивно
	9	101	20	2025-12-23	2025-12-25	Неактивно
	10	104	11	2025-12-26	2025-12-27	Активно
	11	102	20	2025-12-26	2025-12-28	Активно
*	NULL	NULL	NULL	NULL	NULL	NULL

1 of 11

Рис. А.6 Данные в таблице «Бронирование»

	N_заявки	N_помещения	описание_проблемы	дата_подачи	Статус
	89	202	Неисправность отопления	2025-06-04	Неактивно
	90	303	Протечка с потолка	2025-06-18	Неактивно
	91	105	Не работает информационное табло	2025-07-02	Неактивно
	92	401	Повреждение кафеля	2025-07-16	Неактивно
	93	204	Забит канализационный сток	2025-07-30	Неактивно
	94	302	Не работает система климат-контроля	2025-08-13	Неактивно
	95	103	Сломан вентиль	2025-08-27	Неактивно
	96	405	Протечка водоснабжения	2025-09-10	Неактивно
	97	201	Не закрывается дверь	2025-09-24	Неактивно
	98	304	Проблемы с вентиляцией	2025-10-08	Неактивно
	99	102	Трещина в полу	2025-10-22	Неактивно
	100	305	Не работает система оповещения	2025-11-20	Неактивно
	101	305	Течет кран	2025-11-25	Неактивно
	102	405	Крыша течет	2025-12-25	Неактивно
	103	202	Сломана дверь	2025-12-25	Неактивно
	104	405	Кран течет	2025-12-25	Неактивно
	105	101	Не работает свет	2025-12-25	Неактивно
	106	102	Нет света	2025-12-26	Неактивно
	107	102	Сломана дверь	2025-12-26	Неактивно
	108	404	Пожар	2025-12-26	Неактивно
»*	NULL	NULL	NULL	NULL	NULL

109 of 109

Рис. А.7 Данные в таблице «Заявка на ремонт»

	N_помещения	Этаж	Площадь	Стоимость_ар...	Статус
	101	1	50	60000,00	Забронировано
	102	1	45	54000,00	Свободно
	103	1	40	48000,00	Свободно
	104	1	60	72000,00	Забронировано
	105	1	55	66000,00	Свободно
	201	2	80	96000,00	Занятое
	202	2	75	90000,00	Свободно
	203	2	70	84000,00	Свободно
	204	2	85	102000,00	Свободно
	205	2	65	78000,00	Свободно
	301	3	120	144000,00	Занятое
	302	3	110	132000,00	Свободно
	303	3	100	120000,00	Занятое
	304	3	130	156000,00	Свободно
	305	3	105	126000,00	Свободно
	401	4	150	180000,00	Свободно
	402	4	140	168000,00	Свободно
	403	4	135	162000,00	Свободно
	404	4	160	192000,00	Свободно
	405	4	145	174000,00	Свободно
	406	4	25	30000,00	Свободно
»*	NULL	NULL	NULL	NULL	NULL

22 of 22

Рис. А.8 Данные в таблице «Помещение»

	N_отзыва	N_торговой_точки	описание	дата_отзыва	автор	оценка
	89	34	Интересные туры, профессиональные консультанты	2025-04-15	Ксения Стрелкова	5
	90	36	Косметика известных брендов	2025-04-20	Галина Осипова	4
	91	35	Быстрая печать, качественные материалы	2025-04-25	Мargarита Семенова	5
	92	37	Нотариальная контора с опытными специалистами	2025-05-01	Нина Фролова	5
	93	36	Широкий выбор косметических средств	2025-05-05	Любовь Комарова	5
	94	38	Тату салон с профессиональными мастерами	2025-05-10	Алла Горбунова	5
	95	37	Квалифицированные нотариусы	2025-05-15	Регина Тихонова	5
	96	39	Фитнес-бар с полезными напитками	2025-05-20	Эльвира Афанасьева	5
	97	38	Качественные тату, стерильные условия	2025-05-25	Инна Маслова	5
	98	40	Обмен валют с выгодным курсом	2025-06-01	Клавдия Суханова	4
	99	39	Полезные коктейли, свежие ингредиенты	2025-06-05	Лидия Ефимова	5
	100	41	Спортивный бар для активных людей	2025-06-10	Агата Мельникова	4
	101	56	Супер	2025-12-04	Алла Горбунова	5
	102	37	Ужасно!	2025-12-05	Алла Горбунова	1
	103	53	Нормально	2025-12-12	Алла Горбунова	4
	104	64	Не качественно	2025-12-12	Алла Горбунова	2
	105	60	Красивая мебель, очень понравилось!	2025-12-12	Алла Горбунова	5
	106	43	Очень понравилось, хороший персонал и приятные скидки!	2025-12-15	Мария Мартинова	5
	107	43	Не очень!!!!	2025-12-16	Анонимный пользователь	2
	108	82	Не все было вкусно!	2025-12-18	Аноним	4
	109	69	Круто! Обожаю этот магазин, хожу каждый день!!!	2025-12-26	Виктор Алешин	5
*	NULL	NULL	NULL	NULL	NULL	NULL

1 of 109

Рис. А.9 Данные в таблице «Отзыв»

SQL-операторы создания объектов базы данных

1. Создание триггера «Обновить_Статус_Бронь»:

```
Create trigger Обновить_Статус_Бронь on Бронь
after insert, update, delete
As
Begin
    ехес Пересчитать_Статус_Помещений
end
```

2. Создание триггера «Бронь_Проверка_Занятости»:

```
Create trigger Бронь_Проверка_Занятости on Бронирование
After Insert, Update, Delete
As
Begin
    Set nocount on
    If exists (select 1 from inserted
        Join Помещение on Помещение.N_помещения = inserted.N_помещения
        Where Помещение.Статус = N'Занятое')
    Begin
        Raiserror (N'ОШИБКА: Нельзя забронировать помещение, которое имеет
статус "Занятое".', 16, 1)
        Rollback Transaction
        return
    end
    If exists (select 1 from inserted
        Join Помещение on Помещение.N_помещения = inserted.N_помещения
        Where Помещение.Статус = N'Ремонт')
    Begin
        Raiserror (N'ОШИБКА: Помещение находится на ремонте. Бронирование
запрещено.', 16, 1)
        Rollback transaction
        return
    end
    ехес Пересчитать_Статус_Помещений;
end
```

3. Создание триггера «Договор_аренды_Проверка_сотрудника»:

```
Create trigger Договор_аренды_Проверка_сотрудника on Договор
After insert,update
As
Begin
    Set nocount on
    If exists (select * from inserted join Сотрудник on inserted.ID_сотрудника =
Сотрудник.ID_сотрудника
        where Сотрудник.Должность <> N'Менеджер по аренде' )
    Begin
        Print N'Ошибка: Оформлять договоры аренды может только сотрудник с
должностью "Менеджер по аренде".'
```

```

        Rollback transaction
    return
end
Print N'Договор успешно оформлен. Проверка пройдена.'
end

```

4. Создание триггера «Договор_Проверка_Статуса_Помещения_Ремонт»:

```

Create trigger Договор_Проверка_Статуса_Помещения_Ремонт on Договор
For Insert
As
Begin
    If exists (select * from inserted join Помещение on Помещение.N_помещения =
inserted.N_помещения
        where Помещение.Статус = N'Ремонт')
    Begin
        Raiserror('ОШИБКА: Нельзя подписать договор в помещении со статусом
"Ремонт".', 16, 1)
        Rollback Transaction
    return
    end
end

```

5. Создание триггера «Запрет_Уволенного_Сотрудника»

```

Create trigger Запрет_Уволенного_Сотрудника on Договор
For Insert
As
Begin
    If exists (select * from inserted join Сотрудник on inserted.ID_сотрудника =
Сотрудник.ID_сотрудника
        where Сотрудник.Статус = N'Уволен')
    Begin
        Print 'ОШИБКА: Уволенный сотрудник не может участвовать в подписании
договора.'
        Rollback transaction
    return
    end
end

```

6. Создание триггера «Обновить_Статус_Договор»

```

Create trigger Обновить_Статус_Договор on Договор
After Insert, Update, Delete
As
Begin
    Ехес Пересчитать_Статус_Помещений
End

```

7. Создание триггера «Проверка_Статуса_Помещения_При_Договоре»

```

Create trigger Проверка_Статуса_Помещения_При_Договоре on Договор
For insert, update
As
Begin

```



```

If exists (select Помещение.Статус from Помещение, inserted
where Помещение.N_помещения = inserted.N_помещения
and Помещение.Статус = 'Занято' )
Begin
Print 'ОШИБКА: договор нельзя подписать, помещение занято.'
rollback transaction
end
end

```

8. Создание триггера «Обновить_Статус_Ремонт»

```

Create trigger Обновить_Статус_Ремонт on Заявки_на_ремонт
after insert, update, delete
As
Begin
exec Пересчитать_Статус_Помещений
end

```

9. Создание триггера «Расчет_Аренды»:

```

Create trigger Расчет_Аренды on Помещение
For insert, update
As
Begin
Update Помещение set Стоимость_аренды = inserted.Площадь * 1200 from
Помещение
Join inserted on Помещение.N_помещения = inserted.N_помещения
end

```

10. Создание процедуры «Пересчитать_Статус_Помещений»

```

Create procedure Пересчитать_Статус_Помещений
As
Begin
Update Помещения set Статус_помещения = case
When exists (select 1 from Заявки_на_ремонт
where Заявки_на_ремонт.ID_помещения = Помещения.ID_помещения
and Заявки_на_ремонт.Статус = 'Активна') then 'Ремонт'
when exists (select 1 from Договор where Договор.ID_помещения =
Помещения.ID_помещения
and getdate() between Договор.Дата_начала
and Договор.Дата_окончания
and Торговая_точка.Статус_договора = 'Активен') then 'Занято'
when exists (select 1 from Бронь where Бронь.ID_помещения =
Помещения.ID_помещения
and getdate() between Бронь.Дата_начала
and Бронь.Дата_окончания
and Бронь.Статус_брони = 'Активно') then 'Забронировано'
else 'Свободно'
end
end

```


Код программы

Файл main.py

```

import io
from openpyxl import Workbook
from openpyxl.styles import Font, Alignment
import streamlit as st
import pandas as pd
import time
from datetime import datetime, timedelta
from database import DatabaseManager
from reportlab.lib.pagesizes import A4
from reportlab.platypus import SimpleDocTemplate, Table, TableStyle, Paragraph, Spacer
from reportlab.lib import colors
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.lib.units import cm
st.set_page_config(page_title="Торговый Центр | Управление", layout="wide",
initial_sidebar_state="expanded")
@st.cache_resource
def get_db():
    return DatabaseManager()
db = get_db()
if 'logged_in' not in st.session_state:
    st.session_state.logged_in = False
if 'user_data' not in st.session_state:
    st.session_state.user_data = None
if 'user_type' not in st.session_state:
    st.session_state.user_type = None
if 'page' not in st.session_state:
    st.session_state.page = "login"
if 'edit_mode' not in st.session_state:
    st.session_state.edit_mode = {}
if 'show_add_form' not in st.session_state:
    st.session_state.show_add_form = {}

def check_user_login(phone_number):
    """Проверяет номер телефона"""
    try:
        if not phone_number or not phone_number.strip():
            return False, None, "Введите номер телефона"
        phone = phone_number.strip()
        success, user_data, error_message = db.check_user_login(phone)
        if success:
            return True, user_data, None
        else:
            if error_message and any(keyword in error_message.lower() for keyword in ['уволен', 'не активен', 'неактивен']):
                return False, None, error_message
            else:
                return False, None, error_message if error_message else "Пользователь не найден"
    except Exception as e:

```

```

        return False, None, f"Ошибка при проверке: {str(e)}"

def login_page():
    """Страница входа"""
    col1, col2, col3 = st.columns([1, 2, 1])
    with col2:
        st.markdown('<div class="section-header">Вход в систему</div>',
unsafe_allow_html=True)
        with st.container():
            with st.form("login_form"):
                phone = st.text_input(
                    "Номер телефона",
                    placeholder="Пример: +7XXXXXXXXXX",
                    help="Введите номер телефона для входа")
                submitted = st.form_submit_button("Войти в систему")
            if submitted:
                if phone.strip():
                    success, user_data, error_message = check_user_login(phone.strip())
                    if success and user_data:
                        st.session_state.logged_in = True
                        st.session_state.user_data = user_data
                        st.session_state.user_type = user_data.get('user_type', 'tenant')
                        st.session_state.page = "dashboard"
                        st.success(f"Добро пожаловать, {user_data.get('ФИО', 'Пользователь')}!")
                        time.sleep(1)
                        st.rerun()
                    else:
                        if error_message:
                            st.error(f"{error_message}")
                        else:
                            st.error("Пользователь не найден")
                else:
                    st.error("Введите номер телефона")

        st.markdown('</div>', unsafe_allow_html=True)

def dashboard_page():
    """Панель управления"""
    st.markdown('<div class="section-header">Панель управления</div>',
unsafe_allow_html=True)
    if st.session_state.user_data:
        user_type = st.session_state.user_type
        user_data = st.session_state.user_data
        if user_type == 'employee':
            st.markdown("### Информация о сотруднике")
            col_info, col_duties = st.columns([1, 2])
            with col_info:
                st.markdown("#### Основные данные")
                st.markdown("---")
                st.markdown(f"***ФИО:** {user_data.get('ФИО', 'Не указано')}")
                st.markdown(f"***Должность:** {user_data.get('Должность', 'Не указана')}")
                st.markdown(f"***Телефон:** {user_data.get('Номер_телефона', 'Не указан')}")

```

```

st.markdown(f"***Статус:** {user_data.get('Статус', 'Не указан')}")
st.markdown(f"***ID сотрудника:** {user_data.get('ID_сотрудника', '—')}")
st.markdown("---")
st.markdown(f"***Вход выполнен:** {datetime.now().strftime('%d.%m.%Y
%H:%M')}")
position = user_data.get('Должность', "").lower()
if 'менеджер' in position:
    st.success("Полный доступ к системе управления арендой")
elif 'инженер' in position or 'техник' in position:
    st.info("Доступ к управлению ремонтами и техническому обслуживанию")
elif 'администратор' in position:
    st.info("Доступ к административным функциям")
elif 'охранник' in position:
    st.info("Доступ к системе безопасности")
elif 'уборщик' in position or 'клининг' in position:
    st.info("Доступ к системе учета уборки")
else:
    st.info("Доступ в соответствии с должностными обязанностями")
with col_duties:
    st.markdown("#### Должностные обязанности")
    st.markdown("---")
    position = user_data.get('Должность', "").lower()
    if 'менеджер' in position:
        st.markdown("***Менеджер по аренде:**")
        st.markdown("- Управление помещениями и договорами аренды")
        st.markdown("- Работа с арендаторами и заключение новых договоров")
        st.markdown("- Контроль оплаты аренды и выставление счетов")
        st.markdown("- Анализ занятости помещений и планирование загрузки")
        st.markdown("- Проведение переговоров и согласование условий аренды")
        st.markdown("---")
        st.markdown("***Управление помещениями:**")
        st.markdown("- Добавление новых помещений")
        st.markdown("- Редактирование информации о помещениях")
        st.markdown("- Управление статусами помещений (свободно/занято/ремонт)")
        st.markdown("- Удаление помещений")
        st.markdown("---")
        st.markdown("***Оформление договоров:**")
        st.markdown("- Создание новых договоров аренды")
        st.markdown("- Привязка арендаторов к помещениям")
        st.markdown("- Установка сроков и условий аренды")
        st.markdown("- Просмотр списка действующих договоров")
        st.markdown("---")
        st.markdown("***Управление арендаторами:**")
        st.markdown("- Добавление новых арендаторов")
        st.markdown("- Редактирование контактной информации")
        st.markdown("- Просмотр списка арендаторов")
        st.markdown("---")
        st.markdown("***Бронирование помещений:**")
        st.markdown("- Создание временных бронирований")
        st.markdown("- Управление статусами брони")
        st.markdown("- Просмотр активных бронирований")
        st.markdown("---")

```

```

        st.markdown("***Заявки на ремонт:***")
        st.markdown("- Регистрация заявок на ремонт")
        st.markdown("- Назначение статуса заявок")
        st.markdown("- Отслеживание выполнения работ")
    elif 'инженер' in position or 'техник' in position:
        pass
    elif 'администратор' in position:
        pass
    elif 'охранник' in position:
        pass
    elif 'уборщик' in position or 'клининг' in position:
        pass
    else:
        st.markdown("***Общие обязанности сотрудника:***")
        st.markdown("- Соблюдение трудовой дисциплины")
        st.markdown("- Выполнение поручений руководства")
        st.markdown("- Соблюдение правил техники безопасности")
        st.markdown("- Поддержание рабочего места в порядке")
        st.markdown("- Прохождение инструктажей и обучений")
        st.markdown("---")
        st.info("Для получения полного списка обязанностей обратитесь к
руководителю")
    elif user_type == 'tenant':
        st.markdown("### Личный кабинет арендатора")
        try:
            contracts = db.get_all_contracts()
            shops = db.get_all_shops()
            col1, col2, col3 = st.columns(3)
            with col1:
                if not contracts.empty and 'ID_арендатора' in contracts.columns:
                    tenant_contracts = contracts[contracts['ID_арендатора'] ==
user_data.get('ID_арендатора')]
                    active_contracts = len(tenant_contracts)
                    st.markdown(f"
<div class='stat-box'>
    <div class='stat-label'>Активных договоров</div>
    <div class='stat-value'>{active_contracts}</div>
</div>
", unsafe_allow_html=True)
            with col2:
                tenant_shops_count = 0
                if not shops.empty and not contracts.empty:
                    tenant_rooms = contracts[contracts['ID_арендатора'] ==
user_data.get('ID_арендатора')]
                    if not tenant_rooms.empty:
                        tenant_room_numbers = tenant_rooms['N_помещения'].tolist()
                        tenant_shops = shops[shops['N_помещения'].isin(tenant_room_numbers)]
                        tenant_shops_count = len(tenant_shops)
                    st.markdown(f"
<div class='stat-box'>
    <div class='stat-label'>Торговых точек</div>
    <div class='stat-value'>{tenant_shops_count}</div>

```

```

</div>
", unsafe_allow_html=True)
with col3:
    payments = db.get_all_payments()
    unpaid_amount = 0
    if not payments.empty and not contracts.empty:
        tenant_contracts = contracts[contracts['ID_арендатора'] ==
user_data.get('ID_арендатора')]
        if not tenant_contracts.empty:
            tenant_contract_ids = tenant_contracts['N_договора'].tolist()
    st.markdown(f"""
<div class="stat-box">
    <div class="stat-label">Долг (₽)</div>
    <div class="stat-value">{int(unpaid_amount);}</div>
</div>
", unsafe_allow_html=True)
except Exception as e:
    st.error(f"Ошибка загрузки статистики: {str(e)}")
st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
st.markdown("### Доступные функции")
col_func1, col_func2, col_func3 = st.columns(3)
with col_func1:
    if st.button("Мои торговые точки",
        use_container_width=True,
        key="tenant_shops"):
        st.session_state.page = "tenant_shops"
        st.rerun()
with col_func2:
    if st.button("Мои договоры",
        use_container_width=True,
        key="tenant_contracts"):
        st.session_state.page = "tenant_contracts"
        st.rerun()
with col_func3:
    if st.button("Отзывы на мои точки",
        use_container_width=True,
        key="tenant_reviews"):
        st.session_state.page = "tenant_reviews"
        st.rerun()
col_func4, col_func5, col_func6 = st.columns(3)
with col_func4:
    if st.button("Оплата услуг",
        use_container_width=True,
        key="tenant_payments"):
        st.session_state.page = "tenant_payments"
        st.rerun()
with col_func5:
    if st.button("Расторжение договора",
        use_container_width=True,
        key="tenant_terminate"):
        st.session_state.page = "tenant_terminate"
        st.rerun()

```

```

st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
st.markdown("### Ваши данные")
col_info1, col_info2 = st.columns(2)
with col_info1:
    st.markdown("#### Контактная информация")
    st.markdown(f"***ФИО:** {user_data.get('ФИО', 'Не указано')}")
    st.markdown(f"***Телефон:** {user_data.get('Номер_телефона', 'Не указан')}")
    st.markdown(f"***ID арендатора:** {user_data.get('ID_арендатора', '—')}")
with col_info2:
    st.markdown("#### Статус")
    st.markdown("***Статус:** Активный арендатор")
    st.markdown(f"***Вход выполнен:** {datetime.now().strftime('%d.%m.%Y %H:%M')}")
    st.info("Добро пожаловать в личный кабинет арендатора")
try:
    rooms = db.get_all_rooms()
    contracts = db.get_all_contracts()
    tenants = db.get_all_tenants()
    payments = db.get_all_payments()
    st.markdown("### Статистика торгового центра")
    col1, col2, col3, col4 = st.columns(4)
    with col1:
        total_rooms = len(rooms) if not rooms.empty else 0
        st.markdown(f"
        <div class='stat-box'>
          <div class='stat-label'>Помещений</div>
          <div class='stat-value'>{total_rooms}</div>
        </div>
        ", unsafe_allow_html=True)
    with col2:
        total_contracts = len(contracts) if not contracts.empty else 0
        st.markdown(f"
        <div class='stat-box'>
          <div class='stat-label'>Договоров</div>
          <div class='stat-value'>{total_contracts}</div>
        </div>
        ", unsafe_allow_html=True)
    with col3:
        total_tenants = len(tenants) if not tenants.empty else 0
        st.markdown(f"
        <div class='stat-box'>
          <div class='stat-label'>Арендаторов</div>
          <div class='stat-value'>{total_tenants}</div>
        </div>
        ", unsafe_allow_html=True)
    with col4:
        total_payments = payments['Сумма'].sum() if not payments.empty and 'Сумма' in
payments.columns else 0
        st.markdown(f"
        <div class='stat-box'>
          <div class='stat-label'>Оплачено (₽)</div>
          <div class='stat-value'>{int(total_payments):,}</div>
        ")

```

```

</div>
    "", unsafe_allow_html=True)
st.markdown('</div>', unsafe_allow_html=True)
col5, col6, col7, col8 = st.columns(4)
with col5:
    if not rooms.empty:
        free_rooms = len(rooms[rooms['Статус'] == 'Свободно'])
        st.markdown(f"""
        <div class="stat-box">
            <div class="stat-label">Свободных помещений</div>
            <div class="stat-value">{free_rooms}</div>
        </div>
        """, unsafe_allow_html=True)
with col6:
    if not rooms.empty:
        occupied_rooms = len(rooms[rooms['Статус'] == 'Занятое'])
        st.markdown(f"""
        <div class="stat-box">
            <div class="stat-label">Занятых помещений</div>
            <div class="stat-value">{occupied_rooms}</div>
        </div>
        """, unsafe_allow_html=True)
with col7:
    if not contracts.empty:
        today = datetime.now().date()
        expiring_soon = 0
        for _, contract in contracts.iterrows():
            end_date = contract.get('Дата_окончания')
            if end_date:
                try:
                    if isinstance(end_date, str):
                        end_date = datetime.strptime(end_date, '%Y-%m-%d').date()
                        days_left = (end_date - today).days
                        if 0 <= days_left <= 30:
                            expiring_soon += 1
                except:
                    pass
        st.markdown(f"""
        <div class="stat-box">
            <div class="stat-label">Истекают через месяц</div>
            <div class="stat-value">{expiring_soon}</div>
        </div>
        """, unsafe_allow_html=True)
with col8:
    if not payments.empty and 'Сумма' in payments.columns:
        current_month = datetime.now().replace(day=1)
        next_month = (current_month + timedelta(days=32)).replace(day=1)
        if 'дата_оплаты' in payments.columns:
            payments['дата_оплаты_parsed'] = pd.to_datetime(payments['дата_оплаты'],
errors='coerce')
            mask = (payments['дата_оплаты_parsed'] >= pd.Timestamp(current_month)) & \
                (payments['дата_оплаты_parsed'] < pd.Timestamp(next_month))

```



```

        monthly_data = payments.loc[mask, 'Сумма']
        monthly_payments = monthly_data.sum() if not monthly_data.empty else 0
    st.markdown(f"
<div class='stat-box'>
    <div class='stat-label'>Оплата за месяц (₽)</div>
    <div class='stat-value'>{int(monthly_payments):,</div>
</div>
", unsafe_allow_html=True)
except Exception as e:
    st.error(f"Ошибка загрузки статистики: {str(e)}")

def rooms_page():
    """Управление помещениями"""
    st.markdown('<div class="section-header">Управление помещениями</div>',
unsafe_allow_html=True)

    try:
        rooms = db.get_all_rooms()
        contracts = db.get_all_contracts()
        tenants = db.get_all_tenants()
        bookings = db.get_all_bookings()
        col_btn1, col_btn2 = st.columns(2)
        with col_btn1:
            if st.button("Добавить новое помещение",
                key="add_room_btn",
                use_container_width=True):
                st.session_state.show_add_form['room'] = True
        with col_btn2:
            if st.button("Обновить данные",
                key="refresh_rooms_btn",
                use_container_width=True):
                st.rerun()
        if st.session_state.show_add_form.get('room', False):
            st.markdown('<div class="form-container">', unsafe_allow_html=True)
            st.markdown("### Добавление нового помещения")
            with st.form("add_room_form"):
                col1, col2 = st.columns(2)
                with col1:
                    room_number = st.number_input("Номер помещения", min_value=1,
max_value=999, value=101)
                    floor = st.number_input("Этаж", min_value=1, max_value=20, value=1)
                    area = st.number_input("Площадь (м²)", min_value=10.0, max_value=500.0,
value=50.0)
                with col2:
                    status = st.selectbox("Статус", ["Свободно", "Занятое", "Ремонт",
"Забронировано"])
                    rent_cost = st.number_input("Стоимость аренды (₽/мес)", min_value=1000,
max_value=1000000, value=50000)
                col_save, col_cancel = st.columns(2)
                with col_save:
                    submitted = st.form_submit_button("Сохранить помещение",
use_container_width=True)

```

```

with col_cancel:
    if st.form_submit_button("Отмена", use_container_width=True):
        st.session_state.show_add_form['room'] = False
        st.rerun()
if submitted:
    success = db.add_room(room_number, floor, area, rent_cost, status)
    if success:
        st.success(f"Помещение №{room_number} успешно добавлено!")
        st.session_state.show_add_form['room'] = False
        time.sleep(1)
        st.rerun()
    else:
        st.error("Ошибка при добавлении помещения")
st.markdown('</div>', unsafe_allow_html=True)
st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
if not rooms.empty:
    total_rooms = len(rooms)
    occupied = len(rooms[rooms['Статус'] == 'Занятое'])
    free = len(rooms[rooms['Статус'] == 'Свободно'])
    booked = len(rooms[rooms['Статус'] == 'Забронировано'])
    repair = len(rooms[rooms['Статус'] == 'Ремонт'])
    col_stat1, col_stat2, col_stat3, col_stat4, col_stat5 = st.columns(5)
    with col_stat1:
        st.markdown(f"""
        <div class="stat-box">
            <div class="stat-label">Всего помещений</div>
            <div class="stat-value">{total_rooms}</div>
        </div>
        """, unsafe_allow_html=True)
    with col_stat2:
        st.markdown(f"""
        <div class="stat-box">
            <div class="stat-label">Свободно</div>
            <div class="stat-value">{free}</div>
        </div>
        """, unsafe_allow_html=True)
    with col_stat3:
        st.markdown(f"""
        <div class="stat-box">
            <div class="stat-label">Занято</div>
            <div class="stat-value">{occupied}</div>
        </div>
        """, unsafe_allow_html=True)
    with col_stat4:
        st.markdown(f"""
        <div class="stat-box">
            <div class="stat-label">Забронировано</div>
            <div class="stat-value">{booked}</div>
        </div>
        """, unsafe_allow_html=True)
    with col_stat5:
        st.markdown(f"""

```

```

<div class="stat-box">
    <div class="stat-label">Ремонт</div>
    <div class="stat-value">{repair}</div>
</div>
", unsafe_allow_html=True)
st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
st.markdown("### Список помещений")
tenant_map = {}
if not tenants.empty:
    for _, tenant in tenants.iterrows():
        tenant_id = tenant.get('ID_арендатора')
        tenant_name = tenant.get('ФИО', 'Неизвестно')
        if not pd.isna(tenant_id) and not pd.isna(tenant_name):
            tenant_map[int(tenant_id)] = str(tenant_name)
room_tenant_contract_map = {}
if not contracts.empty:
    today = datetime.now().date()
    for _, contract in contracts.iterrows():
        room_num = contract.get('N_помещения')
        tenant_id = contract.get('ID_арендатора')
        end_date = contract.get('Дата_окончания')
        is_active = True
        if pd.notna(end_date):
            try:
                if isinstance(end_date, str):
                    end_date = datetime.strptime(end_date, '%Y-%m-%d').date()
                if end_date < today:
                    is_active = False
            except:
                pass
        if pd.notna(room_num) and pd.notna(tenant_id) and is_active:
            tenant_id_int = int(tenant_id)
            if tenant_id_int in tenant_map:
                room_tenant_contract_map[int(room_num)] = tenant_map[tenant_id_int]
            else:
                room_tenant_contract_map[int(room_num)] = f"Арендатор ID: {tenant_id}"
room_tenant_booking_map = {}
if not bookings.empty:
    today = datetime.now().date()
    for _, booking in bookings.iterrows():
        room_num = booking.get('N_помещения')
        tenant_id = booking.get('ID_арендатора')
        expiry_date = booking.get('Дата_истечения')
        status = booking.get('Статус_брони', '')
        is_active = str(status).strip().lower() == 'активно'
        if pd.notna(expiry_date):
            try:
                if isinstance(expiry_date, str):
                    expiry_date = datetime.strptime(expiry_date, '%Y-%m-%d').date()
                if expiry_date < today:
                    is_active = False
            except:

```

```

        pass
    if pd.isna(room_num) and pd.isna(tenant_id) and is_active:
        tenant_id_int = int(tenant_id)
        if tenant_id_int in tenant_map:
            room_tenant_booking_map[int(room_num)] = tenant_map[tenant_id_int]
        else:
            room_tenant_booking_map[int(room_num)] = f"Арендатор ID: {tenant_id}"
    rooms_with_tenant = rooms.copy()
    def get_tenant_name(room_num):
        room_status = rooms_with_tenant.loc[rooms_with_tenant['N_помещения'] ==
room_num, 'Статус'].iloc[0] if not rooms_with_tenant[rooms_with_tenant['N_помещения'] ==
room_num].empty else ""
        try:
            room_num_int = int(room_num)
        except:
            room_num_int = room_num
        if room_status in ['Свободно', 'Ремонт']:
            return ""
        if room_status == 'Занятое':
            return room_tenant_contract_map.get(room_num_int, "")
        if room_status == 'Забронировано':
            tenant_name = room_tenant_booking_map.get(room_num_int, "")
            if tenant_name != "":
                return tenant_name
            else:
                return "Забронировано"
        return ""
    rooms_with_tenant['Арендатор'] =
rooms_with_tenant['N_помещения'].apply(get_tenant_name)
    col_filter1, col_filter2, col_filter3 = st.columns(3)
    with col_filter1:
        status_filter = st.selectbox("Фильтр по статусу",
["Все", "Свободно", "Занятое", "Ремонт", "Забронировано"],
key="room_status_filter")
    with col_filter2:
        floor_filter_options = ["Все этажи"] +
sorted(rooms_with_tenant['Этаж'].unique().tolist())
        floor_filter = st.selectbox("Фильтр по этажу", floor_filter_options,
key="room_floor_filter")
    with col_filter3:
        sort_options = ["По номеру (возр.)", "По номеру (убыв.)", "По этажу", "По
площади", "По стоимости"]
        sort_by = st.selectbox("Сортировка", sort_options, key="room_sort_filter")
        filtered_rooms = rooms_with_tenant
        if status_filter != "Все":
            filtered_rooms = filtered_rooms[filtered_rooms['Статус'] == status_filter]
        if floor_filter != "Все этажи":
            filtered_rooms = filtered_rooms[filtered_rooms['Этаж'] == floor_filter]
        if sort_by == "По номеру (возр.)":
            filtered_rooms = filtered_rooms.sort_values('N_помещения', ascending=True)
        elif sort_by == "По номеру (убыв.)":
            filtered_rooms = filtered_rooms.sort_values('N_помещения', ascending=False)

```

```

elif sort_by == "По этажу":
    filtered_rooms = filtered_rooms.sort_values(['Этаж', 'N_помещения'])
elif sort_by == "По площади":
    filtered_rooms = filtered_rooms.sort_values('Площадь', ascending=False)
elif sort_by == "По стоимости":
    filtered_rooms = filtered_rooms.sort_values('Стоимость_аренды', ascending=False)
if not filtered_rooms.empty:
    display_rooms = filtered_rooms.copy()
    if 'Стоимость_аренды' in display_rooms.columns:
        display_rooms['Стоимость_аренды'] =
display_rooms['Стоимость_аренды'].apply(
    lambda x: f'{int(x):,} ₽/мес" if pd.notnull(x) else "—"
)
    if 'Площадь' in display_rooms.columns:
        display_rooms['Площадь'] = display_rooms['Площадь'].apply(
            lambda x: f'{x:.1f} м²" if pd.notnull(x) else "—"
        )
    def format_status(status):
        if status == 'Свободно':
            return '<span style="color: #28a745; font-weight: bold;">● Свободно</span>'
        elif status == 'Занятое':
            return '<span style="color: #dc3545; font-weight: bold;">● Занято</span>'
        elif status == 'Забронировано':
            return '<span style="color: #ffc107; font-weight: bold;">●
Забронировано</span>'
        elif status == 'Ремонт':
            return '<span style="color: #6c757d; font-weight: bold;">● Ремонт</span>'
        else:
            return f'<span style="color: #6c757d; font-weight: bold;">● {status}</span>'
    display_rooms['Статус'] = display_rooms['Статус'].apply(format_status)
    rename_dict = {
        'N_помещения': '№ помещения',
        'Этаж': 'Этаж',
        'Площадь': 'Площадь',
        'Стоимость_аренды': 'Стоимость аренды',
        'Статус': 'Статус',
        'Арендатор': 'Арендатор (ФИО)'}
    columns_to_display = ['N_помещения', 'Этаж', 'Площадь', 'Стоимость_аренды',
'Sтатус', 'Арендатор']
    display_rooms = display_rooms[columns_to_display]
    display_rooms.rename(columns=rename_dict, inplace=True)
    st.markdown(f"***Показано помещений:** {len(display_rooms)}")
    st.write(display_rooms.to_html(escape=False, index=False), unsafe_allow_html=True)
    st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
    st.markdown("### Детальная информация о помещении")
    selected_room_number = st.selectbox(
        "Выберите помещение для просмотра деталей",
        sorted(rooms['N_помещения'].unique().tolist()),
        key="room_details_select"
    )
    if selected_room_number:
        room_details = rooms[rooms['N_помещения'] == selected_room_number]

```

```

if not room_details.empty:
    room_data = room_details.iloc[0]
    room_status = room_data['Статус']
    with st.container():
        col1, col2 = st.columns(2)
        with col1:
            st.markdown("#### Основная информация")
            st.markdown(f"***Номер помещения:** {room_data['N_помещения']}")
            st.markdown(f"***Этаж:** {room_data['Этаж']}")
            st.markdown(f"***Площадь:** {room_data['Площадь']} м²")
            if room_status == 'Свободно':
                status_display = f'<span style="color: #28a745; font-weight: bold;">●
{room_status}</span>'
            elif room_status == 'Занятое':
                status_display = f'<span style="color: #dc3545; font-weight: bold;">●
{room_status}</span>'
            elif room_status == 'Забронировано':
                status_display = f'<span style="color: #ffc107; font-weight: bold;">●
{room_status}</span>'
            elif room_status == 'Ремонт':
                status_display = f'<span style="color: #6c757d; font-weight: bold;">●
{room_status}</span>'
            else:
                status_display = f'<span style="color: #6c757d; font-weight: bold;">●
{room_status}</span>'
            st.markdown(f"***Статус:** {status_display}", unsafe_allow_html=True)
        with col2:
            st.markdown("#### Финансовая информация")
            rent = room_data['Стоимость_аренды']
            area = room_data['Площадь']
            st.markdown(f"***Стоимость аренды:** {int(rent):,} Р/мес")
            st.markdown(f"***Стоимость за год:** {int(rent * 12):,} Р")
            if area > 0:
                cost_per_m2 = rent / area
                st.markdown(f"***Стоимость за м²:** {int(cost_per_m2):,} Р/м²")
            else:
                st.markdown(f"***Стоимость за м²:** —")
            st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
            st.markdown("#### Информация об арендаторе")
            try:
                room_num_int = int(selected_room_number)
            except:
                room_num_int = selected_room_number
            if room_status == 'Занятое':
                if not contracts.empty:
                    today = datetime.now().date()
                    active_contracts = []
                    for _, contract in contracts.iterrows():
                        if contract.get('N_помещения') == room_num_int:
                            end_date = contract.get('Дата_окончания')
                            is_active = True
                            if pd.notna(end_date):

```

```

try:
    if isinstance(end_date, str):
        end_date = datetime.strptime(end_date, '%Y-%m-%d').date()
    if end_date < today:
        is_active = False
except:
    pass
if is_active:
    active_contracts.append(contract)
if active_contracts:
    contract_data = active_contracts[0]
    tenant_id = contract_data.get('ID_арендатора')
    tenant_name = "Неизвестно"
try:
    tenant_id_int = int(tenant_id)
    if tenant_id_int in tenant_map:
        tenant_name = tenant_map[tenant_id_int]
except:
    pass
col_tenant1, col_tenant2 = st.columns(2)
with col_tenant1:
    st.markdown(f"***ФИО арендатора:** {tenant_name}")
    st.markdown(f"***Номер договора:** {contract_data.get('N_договора', '—')}")
with col_tenant2:
    start_date = contract_data.get('Дата_оформления', '—')
    end_date = contract_data.get('Дата_окончания', '—')
    if hasattr(start_date, 'strftime'):
        start_date = start_date.strftime('%d.%m.%Y')
    elif pd.isna(start_date):
        try:
            start_date = pd.to_datetime(start_date).strftime('%d.%m.%Y')
        except:
            pass
    if hasattr(end_date, 'strftime'):
        end_date = end_date.strftime('%d.%m.%Y')
    elif pd.isna(end_date):
        try:
            end_date = pd.to_datetime(end_date).strftime('%d.%m.%Y')
        except:
            pass
    st.markdown(f"***Дата начала:** {start_date}")
    st.markdown(f"***Дата окончания:** {end_date}")
try:
    if isinstance(end_date, str):
        end_date_obj = datetime.strptime(end_date, '%d.%m.%Y')
    elif hasattr(end_date, 'date'):
        end_date_obj = end_date
    else:
        end_date_obj = None
    if end_date_obj:

```

```

        days_left = (end_date_obj.date() if hasattr(end_date_obj, 'date')
else end_date_obj) - datetime.now().date()
        if days_left.days > 0:
            st.markdown(f"***Осталось дней:** {days_left.days}")
        elif days_left.days < 0:
            st.warning(f"***Просрочено:** {abs(days_left.days)} дней
назад")

        except:
            pass
    else:
        st.markdown("***Активный договор не найден***")
    else:
        st.markdown("***Договор не найден***")
    elif room_status == 'Забронировано':
        if not bookings.empty:
            today = datetime.now().date()
            active_bookings = []
            for _, booking in bookings.iterrows():
                if booking.get('N_помещения') == room_num_int:
                    expiry_date = booking.get('Дата_истечения')
                    status = booking.get('Статус_брони', '')
                    is_active = str(status).strip().lower() == 'активно'
                    if pd.notna(expiry_date):
                        try:
                            if isinstance(expiry_date, str):
                                expiry_date = datetime.strptime(expiry_date, '%Y-%m-
%d').date()

                                if expiry_date < today:
                                    is_active = False
                        except:
                            pass
                    if is_active:
                        active_bookings.append(booking)
            if active_bookings:
                booking_data = active_bookings[0]
                tenant_id = booking_data.get('ID_арендатора')
                tenant_name = "Неизвестно"
                try:
                    tenant_id_int = int(tenant_id)
                    if tenant_id_int in tenant_map:
                        tenant_name = tenant_map[tenant_id_int]
                except:
                    pass
                st.markdown(f"***ФИО арендатора:** {tenant_name}")
                booking_date = booking_data.get('Дата_бронирования', '')
                expiry_date = booking_data.get('Дата_истечения', '')
                if hasattr(booking_date, 'strftime'):
                    booking_date = booking_date.strftime('%d.%m.%Y')
                elif pd.notna(booking_date):
                    try:
                        booking_date =
pd.to_datetime(booking_date).strftime('%d.%m.%Y')

```



```

        except:
            pass
        if hasattr(expiry_date, 'strftime'):
            expiry_date = expiry_date.strftime('%d.%m.%Y')
        elif pd.isna(expiry_date):
            try:
                expiry_date = pd.to_datetime(expiry_date).strftime('%d.%m.%Y')
            except:
                pass
        st.markdown(f"Даты брони: {booking_date} - {expiry_date}")
        try:
            if booking_date != '—' and expiry_date != '—':
                start = datetime.strptime(booking_date, '%d.%m.%Y')
                end = datetime.strptime(expiry_date, '%d.%m.%Y')
                duration = (end - start).days
                st.markdown(f"Продолжительность: {duration} дней")
            except:
                pass
        else:
            st.markdown("Активное бронирование не найдено")
        elif room_status in ['Свободно', 'Ремонт']:
            st.markdown("Помещение свободно" if room_status == 'Свободно'
else "Помещение на ремонте")
            st.markdown("Можно сдать в аренду новому арендатору.")

        st.markdown('</div>', unsafe_allow_html=True)
    st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
    st.markdown("### Сводная информация")
    total_rent = rooms['Стоимость_аренды'].sum()
    avg_rent = rooms['Стоимость_аренды'].mean()
    avg_area = rooms['Площадь'].mean()
    col_summary1, col_summary2, col_summary3 = st.columns(3)
    with col_summary1:
        st.markdown(f"
        <div class='stat-box'>
            <div class='stat-label'>Общая стоимость аренды</div>
            <div class='stat-value'>{int(total_rent):,} Р/мес</div>
        </div>
        ", unsafe_allow_html=True)
    with col_summary2:
        st.markdown(f"
        <div class='stat-box'>
            <div class='stat-label'>Средняя стоимость</div>
            <div class='stat-value'>{int(avg_rent):,} Р/мес</div>
        </div>
        ", unsafe_allow_html=True)
    with col_summary3:
        st.markdown(f"
        <div class='stat-box'>
            <div class='stat-label'>Средняя площадь</div>

```

```

        <div class="stat-value">{avg_area:.1f} м²</div>
    </div>
    "", unsafe_allow_html=True)
else:
    st.info("Нет данных о помещениях")
    if st.button("Добавить первое помещение", use_container_width=True,
key="add_first_room"):
        st.session_state.show_add_form['room'] = True
        st.rerun()
except Exception as e:
    st.error(f"Ошибка загрузки данных: {str(e)}")
    import traceback
    traceback.print_exc()

def bookings_page():
    """Управление бронированиями (с учетом ограничения в 2 дня)"""
    st.markdown('<div class="section-header">Управление бронированиями</div>',
unsafe_allow_html=True)
    try:
        bookings = db.get_all_bookings()
        col_btn1, col_btn2 = st.columns(2)
        with col_btn1:
            if st.button("Создать новое бронирование",
                key="create_booking_btn",
                use_container_width=True,
                help="Создать новое бронирование помещения"):
                st.session_state.show_add_form['booking'] = True
        with col_btn2:
            if st.button("Обновить список",
                key="refresh_booking_list_btn",
                use_container_width=True,
                help="Обновить список бронирований"):
                st.rerun()
        if st.session_state.show_add_form.get('booking', False):
            show_add_booking_form()
        st.markdown('<div class="divider"></div>', unsafe_allow_html=True)

        if not bookings.empty:
            show_bookings_statistics(bookings)
            st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
            st.markdown("### Список бронирований")
            col_filter1, col_filter2 = st.columns(2)
            with col_filter1:
                status_filter = st.selectbox("Фильтр по статусу",
                    ["Все", "Активно", "Неактивно"],
                    key="booking_status_filter")
            with col_filter2:
                tenants = db.get_all_tenants()
                tenant_list = ["Все арендаторы"]
                if not tenants.empty and 'ФИО' in tenants.columns:
                    tenant_list.extend(tenants['ФИО'].unique().tolist())
                tenant_filter = st.selectbox("Фильтр по арендатору",

```

```

        tenant_list,
        key="booking_tenant_filter")
    filtered_bookings = bookings.copy()
    if status_filter != "Все":
        filtered_bookings = filtered_bookings[filtered_bookings['Статус_брони'].str.strip() ==
status_filter]
    if tenant_filter != "Все арендаторы":
        filtered_bookings = filtered_bookings[filtered_bookings['Арендатор_ФИО'] ==
tenant_filter]
    if not filtered_bookings.empty:
        display_bookings = prepare_bookings_for_display(filtered_bookings)
        st.dataframe(
            display_bookings,
            use_container_width=True,
            hide_index=True,
            column_config={
                "№ брони": st.column_config.NumberColumn(format="%d"),
                "Помещение": st.column_config.TextColumn(),
                "Арендатор": st.column_config.TextColumn(),
                "Дата брони": st.column_config.DateColumn(format="DD.ММ.YYYY"),
                "Дата истечения": st.column_config.DateColumn(format="DD.ММ.YYYY"),
                "Статус": st.column_config.TextColumn())
        st.info(f"Показано бронирований: {len(filtered_bookings)} из {len(bookings)}")
    else:
        st.info("Нет бронирований, соответствующих выбранным фильтрам")
    else:
        st.info("Нет данных о бронированиях")
    if st.button("Создать первое бронирование",
        key="create_first_booking_btn",
        use_container_width=True):
        st.session_state.show_add_form['booking'] = True
        st.rerun()
except Exception as e:
    st.error(f"Ошибка загрузки данных: {str(e)}")
    import traceback
    st.code(traceback.format_exc())
def show_add_booking_form():
    """Показывает форму создания нового бронирования"""
    st.markdown("### Создание нового бронирования")
    st.info("**Ограничение:** Бронирование возможно только на 2 дня")
    with st.form("add_booking_form", clear_on_submit=True):
        tenants = db.get_all_tenants()
        rooms = db.get_all_rooms()
        free_rooms = rooms[rooms['Статус'] == 'Свободно'] if not rooms.empty else
pd.DataFrame()
        col1, col2 = st.columns(2)
        with col1:
            tenant_list = []
            tenant_data = {}
            if not tenants.empty and 'ФИО' in tenants.columns and 'ID_арендатора' in
tenants.columns:
                for idx, tenant in tenants.iterrows():

```

```

        tenant_info = f"{tenant['ФИО']} (тел: {tenant['Номер_телефона']})"
        tenant_list.append(tenant_info)
        tenant_data[tenant_info] = tenant['ID_арендатора']
    if tenant_list:
        tenant_info = st.selectbox("Арендатор *", tenant_list, key="tenant_select")
        tenant_id = tenant_data.get(tenant_info)
    else:
        st.warning("Нет доступных арендаторов")
        tenant_id = None
    with col2:
        room_list = []
        room_data = {}
        if not free_rooms.empty():
            for idx, room in free_rooms.iterrows():
                room_info = f"№{room['N_помещения']} (Этаж {room['Этаж']}, {room['Площадь']}м², {room['Стоимость_аренды']:,}Р/мес)"
                room_list.append(room_info)
                room_data[room_info] = room['N_помещения']
            if room_list:
                room_info = st.selectbox("Помещение *", room_list, key="room_select")
                room_id = room_data.get(room_info)
            else:
                st.warning("Нет свободных помещений")
                room_id = None
    st.markdown("### Период бронирования")
    today = datetime.now().date()
    tomorrow = today + timedelta(days=1)
    max_date = today + timedelta(days=2)
    col_date1, col_date2 = st.columns(2)
    with col_date1:
        booking_date = st.date_input(
            "Дата начала *",
            value=today,
            min_value=today,
            max_value=max_date,
            key="booking_start_date",
            format="DD.MM.YYYY")
    with col_date2:
        end_date = st.date_input(
            "Дата окончания *",
            value=tomorrow,
            min_value=tomorrow,
            max_value=max_date,
            key="booking_end_date",
            format="DD.MM.YYYY")
    st.markdown(f""")
<div style="font-size: 0.85em; color: #666; margin-top: -10px; margin-bottom: 15px;">
    Сегодня: {today.strftime('%d.%m.%Y')} | Максимальная дата:
{max_date.strftime('%d.%m.%Y')}
</div>
""", unsafe_allow_html=True)

```

```

days_count = (end_date - booking_date).days
if days_count > 2:
    st.error("Ошибка: Срок бронирования превышает 2 дня!")
    st.info(f"Максимальный срок - 2 дня. Вы выбрали: {days_count} дней")
    can_submit = False
elif days_count <= 0:
    st.error("Ошибка: Дата окончания должна быть позже даты начала!")
    can_submit = False
else:
    st.success(f"Продолжительность бронирования: {days_count} дней")
    can_submit = True
status = st.selectbox("Статус бронирования *", ["Активно", "Неактивно"],
key="status_select")
st.markdown("---")
st.markdown("* - обязательные поля")
col_save, col_cancel = st.columns(2)
with col_save:
    submitted = st.form_submit_button(
        "Создать бронирование",
        type="primary",
        use_container_width=True,
        disabled=not can_submit)
with col_cancel:
    cancel_clicked = st.form_submit_button(
        "Отмена",
        use_container_width=True)
if cancel_clicked:
    st.session_state.show_add_form['booking'] = False
    st.rerun()

if submitted:
    if not tenant_id or not room_id:
        st.error(" Заполните все обязательные поля!")
    elif not can_submit:
        st.error(" Проверьте правильность введенных данных!")
    else:
        with st.spinner("Создание бронирования..."):
            success = db.add_booking(room_id, tenant_id, booking_date, end_date, status)
            if success:
                st.success(f"Бронирование успешно создано на {days_count} дней!")
                st.session_state.show_add_form['booking'] = False
                time.sleep(2)
                st.rerun()
            else:
                st.error(" Ошибка при создании бронирования")
def show_bookings_statistics(bookings):
    """Показывает статистику по бронированиям"""
    st.markdown("### Статистика бронирований")
    col_stat1, col_stat2, col_stat3, col_stat4 = st.columns(4)
    with col_stat1:
        total_bookings = len(bookings)
        st.markdown(f"

```

```

<div class="stat-box">
  <div class="stat-label">Всего бронирований</div>
  <div class="stat-value">{total_bookings}</div>
</div>
", unsafe_allow_html=True)

with col_stat2:
    active_bookings = len(bookings[bookings['Статус_брони'].str.strip() == 'Активно'])
    st.markdown(f"""
    <div class="stat-box">
      <div class="stat-label">Активных</div>
      <div class="stat-value">{active_bookings}</div>
    </div>
    ", unsafe_allow_html=True)

with col_stat3:
    inactive_bookings = len(bookings[bookings['Статус_брони'].str.strip() == 'Неактивно'])
    st.markdown(f"""
    <div class="stat-box">
      <div class="stat-label">Неактивных</div>
      <div class="stat-value">{inactive_bookings}</div>
    </div>
    ", unsafe_allow_html=True)

with col_stat4:
    today = datetime.now().date()
    expiring_today = 0
    for _, booking in bookings.iterrows():
        expiry_date = booking.get('Дата_истечения')
        if expiry_date:
            try:
                if isinstance(expiry_date, str):
                    expiry_date = datetime.strptime(expiry_date, '%Y-%m-%d').date()
                if expiry_date == today:
                    expiring_today += 1
            except:
                pass

    st.markdown(f"""
    <div class="stat-box">
      <div class="stat-label">Истекают сегодня</div>
      <div class="stat-value">{expiring_today}</div>
    </div>
    ", unsafe_allow_html=True)

def prepare_bookings_for_display(bookings):
    """Подготавливает бронирования для отображения в таблице"""
    display_bookings = bookings.copy()
    date_columns = ['Дата_бронирования', 'Дата_истечения']
    for col in date_columns:
        if col in display_bookings.columns:
            if not pd.api.types.is_datetime64_any_dtype(display_bookings[col]):
                display_bookings[col] = pd.to_datetime(display_bookings[col], errors='coerce')
    rename_dict = {
        'ID_брони': '№ брони',

```

```

    'N_помещения': 'Помещение',
    'Арендатор_ФИО': 'Арендатор',
    'Дата_бронирования': 'Дата брони',
    'Дата_истечения': 'Дата истечения',
    'Статус_брони': 'Статус' }
    rename_dict = {k: v for k, v in rename_dict.items() if k in display_bookings.columns}
    display_bookings.rename(columns=rename_dict, inplace=True)
    columns_to_keep = ['№ брони', 'Помещение', 'Арендатор', 'Дата брони', 'Дата истечения',
    'Статус']
    for col in display_bookings.columns:
        if col not in columns_to_keep:
            display_bookings = display_bookings.drop(columns=[col])

    return display_bookings

def repairs_page():
    """"Управление заявками на ремонт""""
    st.markdown('<div class="section-header">Заявки на ремонт</div>',
    unsafe_allow_html=True)
    try:
        repairs = db.get_all_repairs()
        col_btn1, col_btn2 = st.columns(2)
        with col_btn1:
            if st.button("Создать новую заявку"):
                st.session_state.show_add_form['repair'] = True
        with col_btn2:
            if st.button("Обновить список"):
                st.rerun()
        if st.session_state.show_add_form.get('repair', False):
            st.markdown("### Создание новой заявки на ремонт")
            with st.form("add_repair_form"):
                rooms = db.get_all_rooms()
                col1, col2 = st.columns(2)
                with col1:
                    room_list = []
                    room_ids = {}
                    if not rooms.empty:
                        for idx, room in rooms.iterrows():
                            room_info = f"№{room['N_помещения']} (Этаж {room['Этаж']},
                            {room['Площадь']}м²)"
                            room_list.append(room_info)
                            room_ids[room_info] = room['N_помещения']

                            room_info = st.selectbox("Помещение", room_list) if room_list else
st.selectbox("Помещение", ["Нет помещений"])
                            if room_info != "Нет помещений":
                                room_id = room_ids.get(room_info)
                with col2:
                    status = st.selectbox("Статус", ["Активна", "Неактивно"])
                    problem_description = st.text_area("Описание проблемы", height=100)
                    col_save, col_cancel = st.columns(2)
                    with col_save:

```

```

        submitted = st.form_submit_button("Создать заявку")
    with col_cancel:
        if st.form_submit_button("Отмена"):
            st.session_state.show_add_form['repair'] = False
            st.rerun()
    if submitted:
        if room_id and problem_description:
            success = db.add_repair(room_id, problem_description, status)
            if success:
                st.success("Заявка на ремонт успешно создана!")
                st.session_state.show_add_form['repair'] = False
                time.sleep(1)
                st.rerun()
            else:
                st.error("Ошибка при создании заявки")
        else:
            st.error("Заполните обязательные поля")
    st.markdown('</div>', unsafe_allow_html=True)
    st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
    if not repairs.empty:
        active_repairs = 0
        if 'Статус' in repairs.columns:
            active_repairs = len(repairs[
                repairs['Статус'].astype(str).str.strip().str.lower() == 'активна' ])
        col_stat1, col_stat2, col_stat3 = st.columns(3)
        with col_stat1:
            total_repairs = len(repairs)
            st.markdown(f'''
<div class="stat-box">
    <div class="stat-label">Всего заявок</div>
    <div class="stat-value">{total_repairs}</div>
</div>
''', unsafe_allow_html=True)
        with col_stat2:
            st.markdown(f'''
<div class="stat-box">
    <div class="stat-label">Активных</div>
    <div class="stat-value">{active_repairs}</div>
</div>
''', unsafe_allow_html=True)
        with col_stat3:
            today = datetime.now().date()
            new_today = 0
            if 'дата_подачи' in repairs.columns:
                for _, repair in repairs.iterrows():
                    request_date = repair.get('дата_подачи')
                    if request_date:
                        try:
                            if isinstance(request_date, str):
                                request_date = datetime.strptime(request_date, '%Y-%m-%d').date()
                            if request_date == today:
                                new_today += 1

```



```

        except:
            pass
    st.markdown(f"""
    <div class="stat-box">
        <div class="stat-label">Новых сегодня</div>
        <div class="stat-value">{new_today}</div>
    </div>
    """, unsafe_allow_html=True)
    st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
    st.markdown("### Список заявок на ремонт")
    required_columns = ['N_заявки', 'N_помещения', 'описание_проблемы',
'dата_подачи', 'Статус']
    missing_columns = [col for col in required_columns if col not in repairs.columns]
    if missing_columns:
        st.warning(f"Отсутствуют колонки: {missing_columns}")
    status_filter = st.selectbox("Фильтр по статусу", ["Все", "Активна", "Неактивно"])
    filtered_repairs = repairs.copy()
    if status_filter != "Все":
        filtered_repairs = repairs[repairs['Статус'] == status_filter]
    if not filtered_repairs.empty:
        display_repairs = filtered_repairs.copy()
        if 'дата_подачи' in display_repairs.columns:
            display_repairs['дата_подачи'] = pd.to_datetime(display_repairs['дата_подачи'],
errors='coerce')
            display_repairs['дата_подачи'] =
display_repairs['дата_подачи'].dt.strftime('%d.%m.%Y')
            rename_dict = {}
            if 'N_заявки' in display_repairs.columns:
                rename_dict['N_заявки'] = '№ заявки'
            if 'N_помещения' in display_repairs.columns:
                rename_dict['N_помещения'] = '№ помещения'
            if 'описание_проблемы' in display_repairs.columns:
                rename_dict['описание_проблемы'] = 'Описание проблемы'
            if 'дата_подачи' in display_repairs.columns:
                rename_dict['дата_подачи'] = 'Дата подачи'
            if 'Статус' in display_repairs.columns:
                rename_dict['Статус'] = 'Статус'
            if rename_dict:
                display_repairs.rename(columns=rename_dict, inplace=True)
            st.dataframe(display_repairs, use_container_width=True, hide_index=True)
        else:
            st.info("Нет заявок, соответствующих фильтру")
    else:
        st.info("Нет данных о заявках на ремонт")
except Exception as e:
    st.error(f"Ошибка загрузки данных: {str(e)}")
import traceback
traceback.print_exc()

def shops_page():
    """Управление торговыми точками"""

```

```

st.markdown('<div class="section-header">Управление торговыми точками</div>',
unsafe_allow_html=True)
try:
    shops = db.get_all_shops()
    col_btn1, col_btn2 = st.columns(2)
    with col_btn1:
        if st.button("Добавить новую торговую точку"):
            st.session_state.show_add_form['shop'] = True
    with col_btn2:
        if st.button("Обновить список"):
            st.rerun()
    if st.session_state.show_add_form.get('shop', False):
        st.markdown("#### Добавление новой торговой точки")
        with st.form("add_shop_form"):
            rooms = db.get_all_rooms()
            col1, col2 = st.columns(2)
            with col1:
                room_list = []
                room_ids = {}
                if not rooms.empty:
                    for idx, room in rooms.iterrows():
                        room_info = f"№{room['N_помещения']} (Этаж {room['Этаж']},
{room['Площадь']}м²)"
                        room_list.append(room_info)
                        room_ids[room_info] = room['N_помещения']
                room_info = st.selectbox("Помещение", room_list) if room_list else
st.selectbox("Помещение", ["Нет помещений"])
                if room_info != "Нет помещений":
                    room_id = room_ids.get(room_info)
            with col2:
                category = st.selectbox("Категория", ["Одежда", "Продукты", "Электроника",
"Косметика", "Ресторан", "Другое"])
                rent_cost = st.number_input("Стоимость аренды (₽/мес)", min_value=1000,
max_value=1000000, value=50000)
                shop_name = st.text_input("Название магазина")
            col_save, col_cancel = st.columns(2)
            with col_save:
                submitted = st.form_submit_button("Добавить торговую точку")
            with col_cancel:
                if st.form_submit_button("Отмена"):
                    st.session_state.show_add_form['shop'] = False
                    st.rerun()
            if submitted:
                if room_id and shop_name and rent_cost > 0:
                    success = db.add_shop(room_id, category, rent_cost, shop_name)
                    if success:
                        st.success(f"Торговая точка '{shop_name}' успешно добавлена!")
                        st.session_state.show_add_form['shop'] = False
                        time.sleep(1)
                        st.rerun()
                    else:
                        st.error("Ошибка при добавлении торговой точки")

```

```

        else:
            st.error("Заполните обязательные поля")
        st.markdown('</div>', unsafe_allow_html=True)
    st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
    if not shops.empty:
        col_stat1, col_stat2 = st.columns(2)
        with col_stat1:
            total_shops = len(shops)
            st.markdown(f'''
                <div class="stat-box">
                    <div class="stat-label">Всего точек</div>
                    <div class="stat-value">{total_shops}</div>
                </div>
            ''', unsafe_allow_html=True)

        with col_stat2:
            if 'категория_торговой_точки' in shops.columns:
                categories = shops['категория_торговой_точки'].nunique()
            else:
                categories = 0
            st.markdown(f'''
                <div class="stat-box">
                    <div class="stat-label">Категорий</div>
                    <div class="stat-value">{categories}</div>
                </div>
            ''', unsafe_allow_html=True)
    st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
    st.markdown("#### Список торговых точек")
    col_filter1, col_filter2 = st.columns(2)
    with col_filter1:
        categories = ["Все"] + list(shops['категория_торговой_точки'].unique())
        category_filter = st.selectbox("Фильтр по категории", categories)
    with col_filter2:
        shop_numbers = ["Все"] + sorted(shops['N_торговой_точки'].unique().tolist())
        number_filter = st.selectbox("Фильтр по номеру точки", shop_numbers)
    filtered_shops = shops.copy()
    if category_filter != "Все":
        filtered_shops = filtered_shops[filtered_shops['категория_торговой_точки'] ==
category_filter]
    if number_filter != "Все":
        filtered_shops = filtered_shops[filtered_shops['N_торговой_точки'] == number_filter]

    if not filtered_shops.empty:
        display_shops = filtered_shops.copy()
        if 'стоимость_аренды' in display_shops.columns:
            display_shops['стоимость_аренды'] = display_shops['стоимость_аренды'].apply(
                lambda x: f'{int(x):,} ₽/мес' if pd.notnull(x) else "—")
        rename_dict = {
            'N_торговой_точки': '№ точки',
            'N_помещения': 'Помещение',
            'категория_торговой_точки': 'Категория',
            'стоимость_аренды': 'Стоимость аренды',

```

```

        'Название_магазина': 'Название магазина'}
display_shops.rename(columns=rename_dict, inplace=True)
st.dataframe(display_shops, use_container_width=True, hide_index=True)
else:
    st.info("Нет данных о торговых точках")
except Exception as e:
    st.error(f"Ошибка загрузки данных: {str(e)}")

def tenants_page():
    """Управление арендаторами"""
    st.markdown('<div class="section-header">Управление арендаторов</div>',
unsafe_allow_html=True)
    try:
        tenants = db.get_all_tenants()
        col_btn1, col_btn2 = st.columns(2)
        with col_btn1:
            if st.button("Добавить нового арендатора"):
                st.session_state.show_add_form['tenant'] = True

        with col_btn2:
            if st.button("Обновить список"):
                st.rerun()
        if st.session_state.show_add_form.get('tenant', False):
            st.markdown("### Добавление нового арендатора")
            with st.form("add_tenant_form"):
                col1, col2 = st.columns(2)
                with col1:
                    full_name = st.text_input("ФИО")
                with col2:
                    phone = st.text_input("Номер телефона")
                col_save, col_cancel = st.columns(2)
                with col_save:
                    submitted = st.form_submit_button("Добавить арендатора")
                with col_cancel:
                    if st.form_submit_button("Отмена"):
                        st.session_state.show_add_form['tenant'] = False
                        st.rerun()
            if submitted:
                if full_name and phone:
                    success = db.add_tenant(full_name, phone)
                    if success:
                        st.success(f"Арендатор {full_name} успешно добавлен!")
                        st.session_state.show_add_form['tenant'] = False
                        time.sleep(1)
                        st.rerun()
                    else:
                        st.error("Ошибка при добавлении арендатора")
                else:
                    st.error("Заполните обязательные поля")

            st.markdown('</div>', unsafe_allow_html=True)
    st.markdown('<div class="divider"></div>', unsafe_allow_html=True)

```

```

if not tenants.empty:
    col_stat1, col_stat2 = st.columns(2)
    with col_stat1:
        total_tenants = len(tenants)
        st.markdown(f'''
            <div class="stat-box">
                <div class="stat-label">Всего арендаторов</div>
                <div class="stat-value">{total_tenants}</div>
            </div>
            ''', unsafe_allow_html=True)
    with col_stat2:
        contracts = db.get_all_contracts()
        active_tenants = 0
        if not contracts.empty and not tenants.empty:
            tenant_ids = tenants['ID_арендатора'].tolist()
            contract_tenants = contracts['ID_арендатора'].unique() if 'ID_арендатора' in
contracts.columns else []
            active_tenants = len(set(tenant_ids) & set(contract_tenants))

        st.markdown(f'''
            <div class="stat-box">
                <div class="stat-label">С договорами</div>
                <div class="stat-value">{active_tenants}</div>
            </div>
            ''', unsafe_allow_html=True)
    st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
    st.markdown("### Список арендаторов")

    display_tenants = tenants.copy()
    rename_dict = {
        'ID_арендатора': 'ID',
        'ФИО': 'ФИО',
        'Номер_телефона': 'Телефон'}
    display_tenants.rename(columns=rename_dict, inplace=True)
    st.dataframe(display_tenants, use_container_width=True, hide_index=True)
else:
    st.info("Нет данных об арендаторах")
except Exception as e:
    st.error(f"Ошибка загрузки данных: {str(e)}")

def payments_page():
    """Просмотр оплат"""
    st.markdown('<div class="section-header">Оплаты аренды и услуг</div>',
unsafe_allow_html=True)
    try:
        payments = db.get_all_payments()
        col_btn1, col_btn2 = st.columns(2)
        with col_btn1:
            if st.button("Добавить новую оплату", use_container_width=True):
                st.session_state.show_add_form['payment'] = True
        with col_btn2:
            if st.button("Обновить данные", use_container_width=True):

```

```

st.rerun()
if st.session_state.show_add_form.get('payment', False):
    st.markdown("###Внести новую оплату")
    with st.form("add_payment_form"):
        contracts = db.get_all_contracts()
        st.markdown("**Договор**")
        contract_list = []
        contract_info = {}
        if not contracts.empty:
            for idx, contract in contracts.iterrows():
                contract_id = contract.get('N_договора')
                tenant_name = contract.get('Арендатор_ФИО', 'Неизвестно')
                contract_info_str = f'№{contract_id} - {tenant_name}'
                contract_list.append(contract_info_str)
                contract_info[contract_info_str] = contract_id
        selected_contract = st.selectbox(
            "Выберите договор",
            contract_list if contract_list else ["Нет договоров"],
            label_visibility="collapsed")
        col1, col2 = st.columns(2)
        with col1:
            st.markdown("**Дата оплаты**")
            payment_date = st.date_input(
                "Выберите дату",
                datetime.now(),
                label_visibility="collapsed"
            )
        with col2:
            st.markdown("**Сумма (₽)**")
            amount = st.number_input(
                "Введите сумму",
                min_value=1,
                max_value=10000000,
                value=50000,
                step=1000,
                label_visibility="collapsed")
        col_save, col_cancel = st.columns(2)
        with col_save:
            submitted = st.form_submit_button(
                "Сохранить",
                use_container_width=True,
                type="primary"
            )
        with col_cancel:
            canceled = st.form_submit_button(
                "Отмена",
                use_container_width=True
            )
        if submitted:
            if selected_contract != "Нет договоров" and amount > 0:
                contract_id = contract_info.get(selected_contract)
                success = db.add_payment(contract_id, payment_date, amount)

```

```

        if success:
            st.success("Оплата добавлена!")
            st.session_state.show_add_form['payment'] = False
            time.sleep(1)
            st.rerun()
        else:
            st.error("Ошибка при добавлении")
    else:
        if selected_contract == "Нет договоров":
            st.error("Выберите договор")
        if amount <= 0:
            st.error("Сумма должна быть больше 0")
    if canceled:
        st.session_state.show_add_form['payment'] = False
        st.rerun()

st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
if not payments.empty:
    col_stat1, col_stat2, col_stat3, col_stat4 = st.columns(4)
    with col_stat1:
        total_payments = len(payments)
        st.markdown(f'
        <div class="stat-box">
            <div class="stat-label">Всего оплат</div>
            <div class="stat-value">{total_payments}</div>
        </div>
        ', unsafe_allow_html=True)
    with col_stat2:
        current_month = datetime.now().replace(day=1)
        next_month = (current_month + timedelta(days=32)).replace(day=1)
        monthly_payments = 0
        if 'дата_оплаты' in payments.columns and 'Сумма' in payments.columns:
            payments['дата_оплаты_parsed'] = pd.to_datetime(payments['дата_оплаты'],
errors='coerce')
            mask = (payments['дата_оплаты_parsed'] >= pd.Timestamp(current_month)) & \
                (payments['дата_оплаты_parsed'] < pd.Timestamp(next_month))
            monthly_data = payments.loc[mask, 'Сумма']
            monthly_payments = monthly_data.sum() if not monthly_data.empty else 0
        st.markdown(f'
        <div class="stat-box">
            <div class="stat-label">Р за месяц</div>
            <div class="stat-value">{int(monthly_payments):,}</div>
        </div>
        ', unsafe_allow_html=True)
    with col_stat3:
        avg_payment = payments['Сумма'].mean() if 'Сумма' in payments.columns else 0
        st.markdown(f'
        <div class="stat-box">
            <div class="stat-label">Средняя оплата</div>
            <div class="stat-value">{int(avg_payment):,}Р</div>
        </div>
        ', unsafe_allow_html=True)

```

```

with col_stat4:
    max_payment = payments['Сумма'].max() if 'Сумма' in payments.columns else 0
    st.markdown(f"""
    <div class="stat-box">
        <div class="stat-label">Максимальная</div>
        <div class="stat-value">{int(max_payment):,}₽</div>
    </div>
    """, unsafe_allow_html=True)
st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
st.markdown("### Список оплат")
with st.expander("Фильтры", expanded=True):
    col_filter1, col_filter2, col_filter3 = st.columns(3)
    with col_filter1:
        start_date = st.date_input("Дата от", datetime.now().replace(day=1))
    with col_filter2:
        end_date = st.date_input("Дата до", datetime.now())
    with col_filter3:
        if 'N_договора' in payments.columns:
            contract_numbers = ["Все"] +
sorted(payments['N_договора'].unique().astype(str).tolist())
            contract_filter = st.selectbox("Номер договора", contract_numbers)
        else:
            contract_filter = "Все"
    col_btn_search, col_btn_reset = st.columns(2)
    with col_btn_search:
        search_clicked = st.button("Найти", use_container_width=True)
    with col_btn_reset:
        reset_clicked = st.button("Сбросить", use_container_width=True)
    if 'filtered_payments' not in st.session_state:
        st.session_state.filtered_payments = payments.copy()
    if reset_clicked:
        st.session_state.filtered_payments = payments.copy()
        st.rerun()
    if search_clicked:
        filtered_payments = payments.copy()
        if 'дата_оплаты' in filtered_payments.columns:
            filtered_payments['дата_оплаты_parsed'] =
pd.to_datetime(filtered_payments['дата_оплаты'], errors='coerce')
            if 'дата_оплаты_parsed' in filtered_payments.columns:
                mask = (filtered_payments['дата_оплаты_parsed'] >= pd.Timestamp(start_date)) &
                    (filtered_payments['дата_оплаты_parsed'] <= pd.Timestamp(end_date))
                filtered_payments = filtered_payments.loc[mask]
            if contract_filter != "Все" and 'N_договора' in filtered_payments.columns:
                filtered_payments = filtered_payments[filtered_payments['N_договора'].astype(str)
== contract_filter]
            st.session_state.filtered_payments = filtered_payments
        filtered_payments = st.session_state.filtered_payments
    if not filtered_payments.empty:
        display_payments = filtered_payments.copy()
        if 'дата_оплаты_parsed' in display_payments.columns:
            display_payments = display_payments.drop('дата_оплаты_parsed', axis=1)
        if 'дата_оплаты' in display_payments.columns:

```



```

display_payments['дата_оплаты'] = display_payments['дата_оплаты'].apply(
    lambda x: x.strftime('%d.%m.%Y') if pd.notnull(x) and hasattr(x, 'strftime') else
str(x))
if 'Сумма' in display_payments.columns:
    display_payments['Сумма'] = display_payments['Сумма'].apply(
        lambda x: f'{int(float(x)):.} Р' if pd.notnull(x) else "—")
if 'Арендатор_ФИО' in display_payments.columns:
    display_payments['Арендатор_ФИО'] =
display_payments['Арендатор_ФИО'].apply(
    lambda x: str(x).strip() if pd.notnull(x) else "")
columns_to_show = []
if 'N_операции' in display_payments.columns:
    columns_to_show.append('N_операции')
if 'N_договора' in display_payments.columns:
    columns_to_show.append('N_договора')
if 'дата_оплаты' in display_payments.columns:
    columns_to_show.append('дата_оплаты')
if 'Сумма' in display_payments.columns:
    columns_to_show.append('Сумма')
if 'Арендатор_ФИО' in display_payments.columns:
    columns_to_show.append('Арендатор_ФИО')
display_payments = display_payments[columns_to_show]
rename_dict = {
    'N_операции': '№ операции',
    'N_договора': '№ договора',
    'дата_оплаты': 'Дата оплаты',
    'Сумма': 'Сумма',
    'Арендатор_ФИО': 'Арендатор' }
display_payments.rename(columns=rename_dict, inplace=True)
st.dataframe(
    display_payments,
    use_container_width=True,
    hide_index=True,
    column_order=["№ операции", "№ договора", "Дата оплаты", "Сумма",
"Арендатор"])
st.info(f"Найдено записей: {len(filtered_payments)}")
st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
st.markdown("### Экспорт данных")
export_data = filtered_payments.copy()
if 'дата_оплаты' in export_data.columns:
    export_data['дата_оплаты'] = pd.to_datetime(export_data['дата_оплаты'],
errors='coerce')
    export_data['дата_оплаты'] = export_data['дата_оплаты'].dt.strftime('%d.%m.%Y')
    export_data = export_data[['N_операции', 'N_договора', 'дата_оплаты', 'Сумма',
'Арендатор_ФИО']]
    export_data.columns = ['№ операции', '№ договора', 'Дата оплаты', 'Сумма',
'Арендатор']
    export_data['Сумма'] = pd.to_numeric(export_data['Сумма'], errors='coerce')
    col_export1, col_export2 = st.columns(2)
    with col_export1:
        wb = Workbook()
        ws = wb.active

```

```

ws.title = "Оплаты"
headers = ['№ операции', '№ договора', 'Дата оплаты', 'Сумма', 'Арендатор']
ws.append(headers)
for cell in ws[1]:
    cell.font = Font(bold=True)
    cell.alignment = Alignment(horizontal='center')
for _, row in export_data.iterrows():
    ws.append([
        row['№ операции'],
        row['№ договора'],
        row['Дата оплаты'],
        float(row['Сумма']) if pd.notnull(row['Сумма']) else 0,
        str(row['Арендатор'])
    ])
for column in ws.columns:
    max_length = 0
    column_letter = column[0].column_letter
    for cell in column:
        try:
            if len(str(cell.value)) > max_length:
                max_length = len(str(cell.value))
        except:
            pass
    adjusted_width = min(max_length + 2, 50)
    ws.column_dimensions[column_letter].width = adjusted_width
buffer = io.BytesIO()
wb.save(buffer)
buffer.seek(0)
st.download_button(
    label="Скачать как Excel",
    data=buffer,
    file_name=f"оплаты_{datetime.now().strftime('%Y%m%d_%H%M%S')}.xlsx",
    mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
    use_container_width=True)
with col_export2:
    try:
        from reportlab.pdfbase import pdfmetrics
        from reportlab.pdfbase.ttfonts import TTFont
        try:
            pdfmetrics.registerFont(TTFont('DejaVu', 'DejaVuSans.ttf'))
            font_name = 'DejaVu'
        except:
            try:
                pdfmetrics.registerFont(TTFont('ArialUnicode', 'arial.ttf'))
                font_name = 'ArialUnicode'
            except:
                font_name = 'Helvetica'
    pdf_buffer = io.BytesIO()
    doc = SimpleDocTemplate(
        pdf_buffer,
        pagesize=A4,
        title=f"Отчет по оплатам {datetime.now().strftime('%d.%m.%Y')}",

```

```

leftMargin=1*cm,
rightMargin=1*cm,
topMargin=1*cm,
bottomMargin=1*cm)
styles = getSampleStyleSheet()
for style_name in ['Normal', 'Title', 'Heading1', 'Heading2']:
    if style_name in styles:
        styles[style_name].fontName = font_name
        styles[style_name].encoding = 'UTF-8'
elements = []

title = Paragraph(f"Отчет по оплатам", styles['Title'])
elements.append(title)
date_text = f"Период: {start_date.strftime('%d.%m.%Y')} -
{end_date.strftime('%d.%m.%Y')}"
date_para = Paragraph(date_text, styles['Normal'])
elements.append(date_para)
if contract_filter != "Все":
    contract_text = f"Фильтр по договору: №{contract_filter}"
    contract_para = Paragraph(contract_text, styles['Normal'])
    elements.append(contract_para)
count_text = f"Количество записей: {len(export_data)}"
count_para = Paragraph(count_text, styles['Normal'])
elements.append(count_para)
total_sum = export_data['Сумма'].sum() if 'Сумма' in export_data.columns else 0
total_text = f"Общая сумма: {total_sum:,.2f} ₽"
total_para = Paragraph(total_text, styles['Normal'])
elements.append(total_para)
employee_name = ""
if st.session_state.logged_in and st.session_state.user_data:
    employee_name = st.session_state.user_data.get('ФИО', 'Неизвестный
сотрудник')
else:
    employee_name = "Неавторизованный пользователь"
employee_text = f"Отчет составил: {employee_name}"
employee_para = Paragraph(employee_text, styles['Normal'])
elements.append(employee_para)
elements.append(Spacer(1, 20))
pdf_data = []
headers = ['№ операции', '№ договора', 'Дата оплаты', 'Сумма', 'Арендатор']
pdf_data.append(headers)
for _, row in export_data.iterrows():
    pdf_data.append([
        str(row['№ операции']),
        str(row['№ договора']),
        str(row['Дата оплаты']),
        f'{float(row['Сумма']):.2f} ₽' if pd.notnull(row['Сумма']) else "0.00 ₽",
        str(row['Арендатор']) if pd.notnull(row['Арендатор']) else ""
    ])
table = Table(pdf_data, repeatRows=1)
col_widths = [
    2.5*cm,

```

```

        2.5*cm,
        3.0*cm,
        3.0*cm,
        6.0*cm
    ]
    table._argW = col_widths
    style = TableStyle([
        ('BACKGROUND', (0, 0), (-1, 0), colors.HexColor('#2c3e50')),
        ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
        ('ALIGN', (0, 0), (-1, 0), 'CENTER'),
        ('FONTNAME', (0, 0), (-1, 0), font_name),
        ('FONTSIZE', (0, 0), (-1, 0), 11),
        ('BOTTOMPADDING', (0, 0), (-1, 0), 12),
        ('TOPPADDING', (0, 0), (-1, 0), 12),
        ('BACKGROUND', (0, 1), (-1, -1), colors.beige),
        ('TEXTCOLOR', (0, 1), (-1, -1), colors.black),
        ('FONTSIZE', (0, 1), (-1, -1), 10),
        ('ALIGN', (0, 1), (3, -1), 'CENTER'),
        ('ALIGN', (4, 1), (4, -1), 'LEFT'),
        ('VALIGN', (0, 0), (-1, -1), 'MIDDLE'),
        ('FONTNAME', (0, 1), (-1, -1), font_name),
        ('GRID', (0, 0), (-1, -1), 0.5, colors.grey),
        ('BOX', (0, 0), (-1, -1), 1, colors.black),
        ('ROWBACKGROUNDS', (0, 1), (-1, -1), [colors.white,
colors.HexColor('#f5f5f5')]),
    ])
    table.setStyle(style)
    elements.append(table)
    elements.append(Spacer(1, 30))
    date_generated = f"Сформировано: {datetime.now().strftime('%d.%m.%Y
%H:%M')}"
    date_gen_para = Paragraph(date_generated, styles['Normal'])
    elements.append(date_gen_para)
    signature = "Подпись: _____"
    signature_para = Paragraph(signature, styles['Normal'])
    elements.append(signature_para)
    employee_footer = f"Отчет составил: {employee_name}"
    employee_footer_para = Paragraph(employee_footer, styles['Normal'])
    elements.append(employee_footer_para)
    doc.build(elements)
    pdf_buffer.seek(0)
    st.download_button(
        label="Скачать как PDF",
        data=pdf_buffer,
file_name=f"оплаты_{datetime.now().strftime('%Y%m%d_%H%M%S')}.pdf",
        mime="application/pdf",
        use_container_width=True)
    except Exception as pdf_error:
        st.error(f"Ошибка при создании PDF: {pdf_error}")
        import traceback
        traceback.print_exc()
    else:

```

```

        st.info("Нет данных по выбранному фильтру")
        if st.button("Загрузить все данные", use_container_width=True):
            st.rerun()
    else:
        st.info("Нет данных об оплатах")
        if st.button("Загрузить данные", use_container_width=True):
            st.rerun()
except Exception as e:
    st.error(f"Ошибка загрузки данных: {str(e)}")
    import traceback
    traceback.print_exc()
def employees_page():
    """Управление сотрудниками"""
    st.markdown('<div class="section-header">Управление сотрудниками</div>',
unsafe_allow_html=True)
    try:
        employees = db.get_all_employees()
        col_btn1, col_btn2 = st.columns(2)
        with col_btn1:
            if st.button("Добавить нового сотрудника",
                key="add_employee_btn",
                use_container_width=True):
                st.session_state.show_add_form['employee'] = True
        with col_btn2:
            if st.button("Обновить список",
                key="refresh_employees_btn",
                use_container_width=True):
                st.rerun()
        if st.session_state.show_add_form.get('employee', False):
            st.markdown("### Добавление нового сотрудника")
            with st.form("add_employee_form"):
                col1, col2 = st.columns(2)
                with col1:
                    full_name = st.text_input("ФИО *")
                    position = st.text_input("Должность *")
                with col2:
                    phone = st.text_input("Номер телефона *")
                    status = st.selectbox("Статус", ["Активен", "Неактивен"])
                col_save, col_cancel = st.columns(2)
                with col_save:
                    submitted = st.form_submit_button("Добавить сотрудника",
use_container_width=True)
                with col_cancel:
                    if st.form_submit_button("Отмена", use_container_width=True):
                        st.session_state.show_add_form['employee'] = False
                        st.rerun()
            if submitted:
                if full_name and position and phone:
                    success = db.add_employee(full_name, position, phone, status)
                    if success:
                        st.success(f"Сотрудник {full_name} успешно добавлен!")
                        st.session_state.show_add_form['employee'] = False

```

```

        time.sleep(1)
        st.rerun()
    else:
        st.error("Ошибка при добавлении сотрудника")
    else:
        st.error("Заполните все обязательные поля (отмеченные *)")
    st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
    if not employees.empty:
        active_employees = len(employees[employees['Статус'] == 'Активен'])
        inactive_employees = len(employees[employees['Статус'] == 'Неактивен'])
        col_stat1, col_stat2, col_stat3 = st.columns(3)
        with col_stat1:
            total_employees = len(employees)
            st.markdown(f'
            <div class="stat-box">
                <div class="stat-label">Всего сотрудников</div>
                <div class="stat-value">{total_employees}</div>
            </div>
            ', unsafe_allow_html=True)
        with col_stat2:
            st.markdown(f'
            <div class="stat-box">
                <div class="stat-label">Активных</div>
                <div class="stat-value">{active_employees}</div>
            </div>
            ', unsafe_allow_html=True)
        with col_stat3:
            st.markdown(f'
            <div class="stat-box">
                <div class="stat-label">Неактивных</div>
                <div class="stat-value">{inactive_employees}</div>
            </div>
            ', unsafe_allow_html=True)
    st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
    st.markdown("### Список сотрудников")
    col_filter1, col_filter2, col_filter3 = st.columns(3)
    with col_filter1:
        status_filter = st.selectbox("Фильтр по статусу",
                                     ["Все", "Активен", "Неактивен"],
                                     key="employee_status_filter")
    with col_filter2:
        position_filter = st.selectbox("Фильтр по должности",
                                       ["Все"] + list(employees['Должность'].unique()))
    with col_filter3:
        sort_options = ["По ФИО (А-Я)", "По ФИО (Я-А)", "По должности", "По статусу"]
        sort_by = st.selectbox("Сортировка", sort_options)
    filtered_employees = employees.copy()
    if status_filter != "Все":
        filtered_employees = filtered_employees[filtered_employees['Статус'] ==
status_filter]
    if position_filter != "Все":

```

```

        filtered_employees = filtered_employees[filtered_employees['Должность'] ==
position_filter]
    if sort_by == "По ФИО (А-Я)":
        filtered_employees = filtered_employees.sort_values('ФИО', ascending=True)
    elif sort_by == "По ФИО (Я-А)":
        filtered_employees = filtered_employees.sort_values('ФИО', ascending=False)
    elif sort_by == "По должности":
        filtered_employees = filtered_employees.sort_values('Должность', ascending=True)
    elif sort_by == "По статусу":
        filtered_employees = filtered_employees.sort_values('Статус', ascending=False)

    if not filtered_employees.empty:
        display_employees = filtered_employees.copy()
        def format_status(status):
            if status == 'Активен':
                return '<span style="color: #28a745; font-weight: bold;">● Активен</span>'
            else:
                return '<span style="color: #dc3545; font-weight: bold;">● Неактивен</span>'
        display_employees['Статус'] = display_employees['Статус'].apply(format_status)
        st.markdown(f"***Показано сотрудников:*** {len(display_employees)}")
        st.write(display_employees.to_html(escape=False, index=False),
unsafe_allow_html=True)
        st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
        st.markdown("### Изменение статуса сотрудника")
        employee_options = []
        employee_data = {}
        for _, emp in filtered_employees.iterrows():
            emp_id = emp.get('ID_сотрудника')
            emp_name = emp.get('ФИО')
            current_status = emp.get('Статус')
            option_text = f"{emp_name} (ID: {emp_id}) - {current_status}"
            employee_options.append(option_text)
            employee_data[option_text] = {
                'id': emp_id,
                'name': emp_name,
                'current_status': current_status}
            if employee_options:
                selected_employee = st.selectbox(
                    "Выберите сотрудника для изменения статуса",
                    ["Выберите сотрудника"] + employee_options,
                    key="employee_select_status")
            if selected_employee != "Выберите сотрудника":
                emp_info = employee_data.get(selected_employee)
                if emp_info:
                    col_info1, col_info2 = st.columns(2)
                    with col_info1:
                        st.info(f"***ID:*** {emp_info['id']}")
                        st.info(f"***ФИО:*** {emp_info['name']}")
                    with col_info2:
                        st.info(f"***Текущий статус:*** {emp_info['current_status']}")
                    new_status = st.selectbox(
                        "Новый статус",

```

```

        ["Активен", "Неактивен"],
        index=0 if emp_info['current_status'] == "Активен" else 1,
        key=f"new_employee_status_{emp_info['id']}")
    if st.button(
        f"Изменить статус на '{new_status}'",
        key=f"update_employee_status_{emp_info['id']}",
        type="primary",
        use_container_width=True):
        with st.spinner("Обновление статуса..."):
            success = db.update_employee_status(emp_info['id'], new_status)
            if success:
                st.success(f"Статус сотрудника {emp_info['name']} успешно
изменен!")
                time.sleep(2)
                st.rerun()
            else:
                st.error(f"Ошибка при изменении статуса сотрудника")
        else:
            st.info("Нет сотрудников, соответствующих выбранному фильтру")
    else:
        st.info("Нет данных о сотрудниках")
        if st.button("Добавить первого сотрудника", use_container_width=True,
key="add_first_employee"):
            st.session_state.show_add_form['employee'] = True
            st.rerun()
except Exception as e:
    st.error(f"Ошибка загрузки данных: {str(e)}")
import traceback
traceback.print_exc()
floor_styles = {
    1: {
        'bg_gradient': 'linear-gradient(135deg, #e3f2fd 0%, #bbdefb 100%)',
        'border_color': '#2196f3',
        'text_color': '#0d47a1'
    },
    2: {
        'bg_gradient': 'linear-gradient(135deg, #e8f5e9 0%, #c8e6c9 100%)',
        'border_color': '#4caf50',
        'text_color': '#1b5e20'
    },
    3: {
        'bg_gradient': 'linear-gradient(135deg, #fff3e0 0%, #ffe0b2 100%)',
        'border_color': '#ff9800',
        'text_color': '#e65100'
    },
    4: {
        'bg_gradient': 'linear-gradient(135deg, #f3e5f5 0%, #e1bee7 100%)',
        'border_color': '#9c27b0',
        'text_color': '#4a148c'
    },
    5: {
        'bg_gradient': 'linear-gradient(135deg, #fff8e1 0%, #ffecb3 100%)',

```



```

        'border_color': '#ffc107',
        'text_color': '#ff6f00'
    }
}
default_style = {
    'bg_gradient': 'linear-gradient(135deg, #f5f5f5 0%, #e0e0e0 100%)',
    'border_color': '#9e9e9e',
    'text_color': '#424242'}
def map_page():
    """Карта торгового центра"""
    st.markdown('<div class="section-header">Карта торгового центра</div>',
unsafe_allow_html=True)
    try:
        all_floors = db.get_all_floors_list()
        if not all_floors:
            st.warning("В базе данных нет информации об этажах")
            st.info("Пожалуйста, добавьте помещения в базу данных")
            return
        col_stat1, col_stat2, col_stat3 = st.columns(3)
        with col_stat1:
            total_floors = len(all_floors)
            st.markdown(f'
<div class="stat-box">
    <div class="stat-label">Всего этажей</div>
    <div class="stat-value">{total_floors}</div>
</div>
', unsafe_allow_html=True)
        with col_stat2:
            rooms = db.get_all_rooms()
            total_rooms = len(rooms) if not rooms.empty else 0
            st.markdown(f'
<div class="stat-box">
    <div class="stat-label">Всего помещений</div>
    <div class="stat-value">{total_rooms}</div>
</div>
', unsafe_allow_html=True)
        with col_stat3:
            active_shops = db.get_all_shops()
            active_shops_count = len(active_shops[active_shops['Статус_договора'] == 'Активен'])
            if not active_shops.empty else 0
            st.markdown(f'
<div class="stat-box">
    <div class="stat-label">Активных магазинов</div>
    <div class="stat-value">{active_shops_count}</div>
</div>
', unsafe_allow_html=True)
        floors_with_shops = db.get_floors_with_shops()
        if not floors_with_shops:
            st.info("На данный момент нет активных торговых точек")
            return
        tab_names = [f"{floor} этаж" for floor in floors_with_shops]
        tabs = st.tabs(tab_names)

```

```
for idx, floor in enumerate(floors_with_shops):
    with tabs[idx]:
        style = floor_styles.get(floor, default_style)
        st.markdown(f"""
<div style="
background: linear-gradient(135deg, #2c3e50 0%, #3498db 100%);
color: white;
padding: 15px 25px;
border-radius: 8px;
margin-bottom: 25px;
font-size: 1.4em;
font-weight: 700;
box-shadow: 0 4px 6px rgba(0,0,0,0.1);
text-align: center;
">
{floor} этаж
</div>
""", unsafe_allow_html=True)
    floor_shops = db.get_active_shops_by_floor(floor)
    if not floor_shops.empty():
        categories = {}
        for _, shop in floor_shops.iterrows():
            category = shop.get('категория_торговой_точки', 'Другое')
            if category not in categories:
                categories[category] = []
            categories[category].append(shop)
        for category, shops_list in categories.items():
            with st.expander(f" {category} ({len(shops_list)})", expanded=True):
                cols = st.columns(3)
                for shop_idx, shop in enumerate(shops_list):
                    col_idx = shop_idx % 3
                    with cols[col_idx]:
                        shop_name = shop.get('Название_магазина', 'Без названия')
                        room_num = shop.get('N_помещения', '—')
                        category_type = shop.get('категория_торговой_точки', '—')
                        rent_cost = shop.get('стоимость_аренды', 0)
                        tenant_name = shop.get('Арендатор_ФИО', 'Неизвестно')
                        rent_formatted = f"{int(rent_cost):,} ₽/мес" if rent_cost else "—"
                        st.markdown(f"""
<div style="
background: {style['bg_gradient']};
border: 2px solid {style['border_color']};
border-radius: 8px;
padding: 15px;
margin-bottom: 10px;
box-shadow: 0 2px 4px rgba(0,0,0,0.05);
transition: all 0.3s ease;
height: 100%;
">
<div style="
font-weight: bold;
color: {style['text_color']};
```

```

        margin-bottom: 8px;
        font-size: 16px;
    ">
        {shop_name}
    </div>
    <div style="font-size: 13px; color: #6c757d; margin-bottom: 5px;">
        <strong>Категория:</strong> {category_type}
    </div>
    <div style="font-size: 13px; color: #6c757d; margin-bottom: 5px;">
        <strong>Помещение:</strong> №{room_num}
    </div>
    <div style="font-size: 13px; color: #6c757d; margin-bottom: 5px;">
        <strong>Арендатор:</strong> {tenant_name}
    </div>
    <div style="font-size: 13px; color: #6c757d;">
        <strong>Стоимость аренды:</strong> {rent_formatted}
    </div>
    </div>
    """', unsafe_allow_html=True)
st.markdown(f'"""
<div style="
    background: #f8f9fa;
    border: 1px solid #e0e6ed;
    border-radius: 8px;
    padding: 15px;
    margin-top: 20px;
    font-size: 14px;
    color: #6c757d;
">
    <strong>Этаж {floor}</strong> Всего {len(floor_shops)} активных торговых
точек

</div>
"""', unsafe_allow_html=True)

else:
    st.info(f'На {floor} этаже пока нет активных торговых точек')
except Exception as e:
    st.error(f'Ошибка загрузки карты торгового центра: {str(e)}')
def reviews_page():
    """Просмотр отзывов"""
    st.markdown('<div class="section-header">Отзывы</div>', unsafe_allow_html=True)
    try:
        col_btn1, col_btn2 = st.columns(2)
        with col_btn1:
            if st.button("Оставить отзыв", key="add_review_btn"):
                st.session_state.page = "add_review"
                st.rerun()
        with col_btn2:
            if st.button("Обновить отзывы", key="refresh_reviews_btn"):
                st.rerun()
        reviews = db.get_all_reviews()
        if not reviews.empty:

```

```

col_stat1, col_stat2, col_stat3, col_stat4 = st.columns(4)
with col_stat1:
    total_reviews = len(reviews)
    st.markdown(f"""
    <div class="stat-box">
        <div class="stat-label">Всего отзывов</div>
        <div class="stat-value">{total_reviews}</div>
    </div>
    """, unsafe_allow_html=True)
with col_stat2:
    avg_rating = reviews['оценка'].mean() if 'оценка' in reviews.columns else 0
    st.markdown(f"""
    <div class="stat-box">
        <div class="stat-label">Средний рейтинг</div>
        <div class="stat-value">{avg_rating:.1f}/5</div>
    </div>
    """, unsafe_allow_html=True)
with col_stat3:
    five_star = len(reviews[reviews['оценка'] == 5]) if 'оценка' in reviews.columns else 0
    st.markdown(f"""
    <div class="stat-box">
        <div class="stat-label">5 звезд</div>
        <div class="stat-value">{five_star}</div>
    </div>
    """, unsafe_allow_html=True)
with col_stat4:
    today = datetime.now().date()
    today_reviews = 0
    if 'дата_отзыва' in reviews.columns:
        for _, review in reviews.iterrows():
            review_date = review.get('дата_отзыва')
            if review_date:
                try:
                    if isinstance(review_date, str):
                        review_date = datetime.strptime(review_date, '%Y-%m-%d').date()
                    if review_date == today:
                        today_reviews += 1
                except:
                    pass
    st.markdown(f"""
    <div class="stat-box">
        <div class="stat-label">Сегодня</div>
        <div class="stat-value">{today_reviews}</div>
    </div>
    """, unsafe_allow_html=True)
st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
st.markdown("### Все отзывы")
col_filter1, col_filter2, col_filter3 = st.columns(3)
with col_filter1:
    rating_filter = st.selectbox("Фильтр по оценке", ["Все", "5 звезд", "4 звезды", "3
звезды", "2 звезды", "1 звезда"])
with col_filter2:

```

```

shop_filter = st.selectbox("Фильтр по магазину", ["Все"] +
list(reviews['Название_магазина'].unique()))
with col_filter3:
    sort_option = st.selectbox("Сортировка", ["Сначала новые", "Сначала старые",
"Высокие оценки", "Низкие оценки"])
    filtered_reviews = reviews.copy()
    if rating_filter != "Все":
        rating_value = int(rating_filter[0])
        filtered_reviews = filtered_reviews[filtered_reviews['оценка'] == rating_value]
    if shop_filter != "Все":
        filtered_reviews = filtered_reviews[filtered_reviews['Название_магазина'] ==
shop_filter]
    if sort_option == "Сначала новые":
        filtered_reviews = filtered_reviews.sort_values('дата_отзыва', ascending=False)
    elif sort_option == "Сначала старые":
        filtered_reviews = filtered_reviews.sort_values('дата_отзыва', ascending=True)
    elif sort_option == "Высокие оценки":
        filtered_reviews = filtered_reviews.sort_values('оценка', ascending=False)
    elif sort_option == "Низкие оценки":
        filtered_reviews = filtered_reviews.sort_values('оценка', ascending=True)
    if not filtered_reviews.empty:
        for idx, review in filtered_reviews.iterrows():
            author = str(review.get('автор', 'Аноним'))
            rating = int(review.get('оценка', 0))
            description = str(review.get('описание', ''))
            shop_name = str(review.get('Название_магазина', ''))
            date_str = review.get('дата_отзыва', '')
            if date_str:
                try:
                    if isinstance(date_str, str):
                        date_obj = datetime.strptime(date_str, '%Y-%m-%d')
                        date_str = date_obj.strftime('%d.%m.%Y')
                    elif hasattr(date_str, 'strftime'):
                        date_str = date_str.strftime('%d.%m.%Y')
                except:
                    date_str = str(date_str)
            rating_color = "#28a745" if rating >= 4 else "#ffc107" if rating == 3 else "#dc3545"
            col1, col2 = st.columns([3, 1])
            with col1:
                st.markdown(f"***{author if author else 'Аноним'}***")
                if shop_name and shop_name != 'nan' and shop_name != 'None':
                    st.markdown(f"***{shop_name}***")
            with col2:
                st.markdown(f"***{date_str}***")
                st.markdown(f"<span style='color: {rating_color}; font-weight:
bold;'>{rating}/5</span>", unsafe_allow_html=True)
                stars = "★" * rating + "☆" * (5 - rating)
                st.markdown(f"<div style='color: #FFD700; font-size: 20px; margin: 5px
0;'>{stars}</div>", unsafe_allow_html=True)
                if description and description != 'nan' and description != 'None':
                    st.markdown(description)
            st.markdown("---")

```

```

        st.info(f"Найдено отзывов: {len(filtered_reviews)}")
    else:
        st.info("Нет отзывов, соответствующих фильтру")
    else:
        st.markdown("""
<div style="
    text-align: center;
    padding: 60px 30px;
    background: linear-gradient(135deg, #f8f9fa 0%, #ffffff 100%);
    border-radius: 15px;
    border: 2px dashed #e0e6ed;
    margin: 40px 0;
">
    <div style="font-size: 48px; color: #e0e6ed; margin-bottom: 20px;">
        ★★☆☆☆
    </div>
    <h3 style="color: #2c3e50; margin-bottom: 10px;">
        Пока нет отзывов
    </h3>
    <p style="color: #7f8c8d; margin-bottom: 20px;">
        Будьте первым, кто оставит отзыв о магазине
    </p>
</div>
""", unsafe_allow_html=True)
except Exception as e:
    st.error(f"Ошибка загрузки отзывов: {str(e)}")
def contracts_page():
    """Управление договорами аренды"""
    st.markdown('<div class="section-header">Управление договорами аренды</div>',
unsafe_allow_html=True)
    try:
        user_info = st.session_state.user_data if st.session_state.logged_in else None
        if not user_info:
            st.error("Пользователь не авторизован")
            return
        if 'created_contracts' not in st.session_state:
            st.session_state.created_contracts = {}
        if 'contract_creation_in_progress' not in st.session_state:
            st.session_state.contract_creation_in_progress = False
        if 'last_created_contract' not in st.session_state:
            st.session_state.last_created_contract = None
        if 'show_contract_success' not in st.session_state:
            st.session_state.show_contract_success = False
        if 'termination_in_progress' not in st.session_state:
            st.session_state.termination_in_progress = False
        if 'terminated_contract' not in st.session_state:
            st.session_state.terminated_contract = None
        tab1, tab2, tab3 = st.tabs(["Все договоры", "Новый договор", "Расторжение"])
        with tab1:
            st.subheader("Список всех договоров")
            contracts = db.get_all_contracts()
            if not contracts.empty:

```

```

shops = db.get_all_shops()
contract_status_map = {}
if not shops.empty:
    for _, shop in shops.iterrows():
        shop_id = shop.get('N_торговой_точки')
        status = shop.get('Статус_договора')
        if shop_id is not None and status is not None:
            contract_status_map[shop_id] = status
def get_contract_status(contract_id):
    return contract_status_map.get(contract_id, 'Не указан')
contracts['Статус_договора'] = contracts['N_договора'].apply(get_contract_status)
st.markdown("### Фильтры договоров")
col_filter1, col_filter2, col_filter3 = st.columns(3)
with col_filter1:
    all_statuses = ["Все статусы"] +
sorted(contracts['Статус_договора'].dropna().unique().tolist())
    status_filter = st.selectbox("Фильтр по статусу", all_statuses,
key="status_filter_all")

    with col_filter2:
        all_tenants = ["Все арендаторы"] +
sorted(contracts['Арендатор_ФИО'].dropna().unique().tolist())
        tenant_filter = st.selectbox("Фильтр по арендатору", all_tenants,
key="tenant_filter_all")
    with col_filter3:
        all_rooms = ["Все помещения"] +
sorted(contracts['N_помещения'].dropna().unique().tolist())
        room_filter = st.selectbox("Фильтр по помещению", all_rooms,
key="room_filter_all")
    col_filter4, col_filter5 = st.columns(2)
    with col_filter4:
        date_filter_options = ["Все", "Активные (не истекли)", "Истекшие", "Истекают в
этом месяце"]
        date_filter = st.selectbox("Фильтр по дате", date_filter_options,
key="date_filter_all")
    with col_filter5:
        sort_options = [
            "По номеру договора",
            "По арендатору",
            "По дате начала",
            "По дате окончания",
            "По стоимости",
            "По номеру помещения"
        ]
        sort_by = st.selectbox("Сортировка", sort_options, key="sort_filter_all")
    filtered_contracts = contracts.copy()
    if status_filter != "Все статусы":
        filtered_contracts = filtered_contracts[filtered_contracts['Статус_договора'] ==
status_filter]
    if tenant_filter != "Все арендаторы":
        filtered_contracts = filtered_contracts[filtered_contracts['Арендатор_ФИО'] ==
tenant_filter]

```

```

        if room_filter != "Все помещения":
            filtered_contracts = filtered_contracts[filtered_contracts['N_помещения'] ==
room_filter]
            today = datetime.now().date()
            if date_filter == "Активные (не истекли)":
                filtered_contracts['Дата_окончания_dt'] =
pd.to_datetime(filtered_contracts['Дата_окончания'], errors='coerce')
                filtered_contracts = filtered_contracts[filtered_contracts['Дата_окончания_dt'] >
pd.Timestamp(today)]
            elif date_filter == "Истекшие":
                filtered_contracts['Дата_окончания_dt'] =
pd.to_datetime(filtered_contracts['Дата_окончания'], errors='coerce')
                filtered_contracts = filtered_contracts[filtered_contracts['Дата_окончания_dt'] <=
pd.Timestamp(today)]
            elif date_filter == "Истекают в этом месяце":
                filtered_contracts['Дата_окончания_dt'] =
pd.to_datetime(filtered_contracts['Дата_окончания'], errors='coerce')
                end_of_month = (today.replace(day=1) + timedelta(days=32)).replace(day=1) -
timedelta(days=1)
                filtered_contracts = filtered_contracts[
                    (filtered_contracts['Дата_окончания_dt'] >= pd.Timestamp(today)) &
                    (filtered_contracts['Дата_окончания_dt'] <= pd.Timestamp(end_of_month))
                ]
            if sort_by == "По номеру договора":
                filtered_contracts = filtered_contracts.sort_values('N_договора', ascending=True)
            elif sort_by == "По арендатору":
                filtered_contracts = filtered_contracts.sort_values('Арендатор_ФИО',
ascending=True)
            elif sort_by == "По дате начала":
                filtered_contracts['Дата_оформления_dt'] =
pd.to_datetime(filtered_contracts['Дата_оформления'], errors='coerce')
                filtered_contracts = filtered_contracts.sort_values('Дата_оформления_dt',
ascending=False)
            elif sort_by == "По дате окончания":
                filtered_contracts['Дата_окончания_dt'] =
pd.to_datetime(filtered_contracts['Дата_окончания'], errors='coerce')
                filtered_contracts = filtered_contracts.sort_values('Дата_окончания_dt',
ascending=True)
            elif sort_by == "По стоимости":
                filtered_contracts = filtered_contracts.sort_values('Тариф', ascending=False)
            elif sort_by == "По номеру помещения":
                filtered_contracts = filtered_contracts.sort_values('N_помещения', ascending=True)
            display_df = filtered_contracts.copy()
            if 'Дата_оформления' in display_df.columns:
                display_df['Дата_оформления'] = pd.to_datetime(display_df['Дата_оформления'],
errors='coerce')
                display_df['Дата_оформления'] =
display_df['Дата_оформления'].dt.strftime('%d.%m.%Y')
                display_df['Дата_окончания'] = pd.to_datetime(display_df['Дата_окончания'],
errors='coerce')
                display_df['Дата_окончания'] =
display_df['Дата_окончания'].dt.strftime('%d.%m.%Y')

```



```

if 'Тариф' in display_df.columns:
    display_df['Тариф'] = pd.to_numeric(display_df['Тариф'], errors='coerce')
    display_df['Стоимость аренды'] = display_df['Тариф'].apply(
        lambda x: f'{x:,.0f} Р/мес' if pd.notnull(x) else "0 Р/мес")
def format_status(status):
    status_str = str(status)
    if status_str == 'Активен':
        return f'<span style="color: #28a745; font-weight: bold;">{status_str}</span>'
    elif status_str == 'Расторгнут':
        return f'<span style="color: #dc3545; font-weight: bold;">{status_str}</span>'
    elif status_str == 'Истек':
        return f'<span style="color: #6c757d; font-weight: bold;">{status_str}</span>'
    elif status_str == 'Новый':
        return f'<span style="color: #17a2b8; font-weight: bold;">{status_str}</span>'
    elif status_str == 'В процессе оформления':
        return f'<span style="color: #ffc107; font-weight: bold;">{status_str}</span>'
    else:
        return f'<span style="color: #6c757d; font-weight: bold;">{status_str}</span>'
display_df['№ договора'] = display_df['N_договора'].apply(lambda x: f'№{int(x)}' if
pd.notnull(x) else "—")
display_df['№ помещения'] = display_df['N_помещения'].apply(lambda x:
f'№{int(x)}' if pd.notnull(x) else "—")
display_df = display_df.rename(columns={
    'Арендатор_ФИО': 'Арендатор',
    'Дата_оформления': 'Дата начала',
    'Дата_окончания': 'Дата окончания',
    'Статус_договора': 'Статус договора'})
columns_to_show = [
    '№ договора',
    '№ помещения',
    'Арендатор',
    'Дата начала',
    'Дата окончания',
    'Стоимость аренды',
    'Статус договора']
display_df = display_df[columns_to_show]
st.write(f'***Найдено договоров:** {len(display_df)} из {len(contracts)}')
st.write(display_df.to_html(
    escape=False,
    formatters={'Статус договора': lambda x: format_status(x)},
    index=False
), unsafe_allow_html=True)
st.markdown("### Статистика")
col_stat1, col_stat2, col_stat3, col_stat4, col_stat5 = st.columns(5)
with col_stat1:
    st.metric("Всего договоров", len(contracts))
with col_stat2:
    active_by_date = contracts[pd.to_datetime(contracts['Дата_окончания']) >
datetime.now()]
    st.metric("Активных по дате", len(active_by_date))
with col_stat3:

```

```

        expired_contracts = contracts[pd.to_datetime(contracts['Дата_окончания']) <=
datetime.now()]
        st.metric("Истекших", len(expired_contracts))
    with col_stat4:
        active_status_count = len(contracts[contracts['Статус_договора'] == 'Активен'])
        st.metric("Статус 'Активен'", active_status_count)
    with col_stat5:
        unique_rooms = contracts['N_помещения'].nunique()
        st.metric("Уникальных помещений", unique_rooms)
    st.markdown("---")
    if st.button("Экспорт в Excel", use_container_width=True, key="export_excel_btn"):
        output = io.BytesIO()
        with pd.ExcelWriter(output, engine='openpyxl') as writer:
            export_df = filtered_contracts.copy()
            if 'Дата_оформления' in export_df.columns:
                export_df['Дата_оформления'] =
pd.to_datetime(export_df['Дата_оформления'], errors='coerce')
                export_df['Дата_оформления'] =
export_df['Дата_оформления'].dt.strftime('%d.%m.%Y')
            if 'Дата_окончания' in export_df.columns:
                export_df['Дата_окончания'] = pd.to_datetime(export_df['Дата_окончания'],
errors='coerce')
                export_df['Дата_окончания'] =
export_df['Дата_окончания'].dt.strftime('%d.%m.%Y')
            export_df = export_df[[
                'N_договора',
                'N_помещения',
                'Арендатор_ФИО',
                'Дата_оформления',
                'Дата_окончания',
                'Тариф',
                'Статус_договора'
            ]]
            export_df.columns = ['№ договора', '№ помещения', 'Арендатор', 'Дата начала',
'Дата окончания', 'Стоимость (Р)', 'Статус']
            export_df.to_excel(writer, sheet_name='Договоры', index=False)
            output.seek(0)
            st.download_button(
                label="Скачать Excel файл",
                data=output,
                file_name=f"договоры_{datetime.now().strftime('%Y%m%d_%H%M%S')}.xlsx",
                mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
                use_container_width=True,
                key="download_excel_btn")
        else:
            st.info("Договоры не найдены")
    with tab2:
        st.subheader("Добавление нового договора")
        if st.session_state.show_contract_success:
            # Показываем сообщение об успехе
            last_contract = st.session_state.last_created_contract
            if last_contract:

```

```

        st.success("Договор успешно оформлен! Создана торговая точка.")
        st.info(f"***Номер договора:** {last_contract.get('id', 'N/A')}")
        st.info(f"***Магазин:** {last_contract.get('shop_name', 'N/A')}")
        st.info(f"***Помещение:** №{last_contract.get('room_id', 'N/A')}")
        st.info(f"***Период:** {last_contract.get('start_date', 'N/A')} -
{last_contract.get('end_date', 'N/A')}")
        col_next1, col_next2 = st.columns(2)
        with col_next1:
            if st.button("Создать еще один договор", use_container_width=True,
key="create_another_contract"):
                st.session_state.show_contract_success = False
                st.rerun()
        with col_next2:
            if st.button("Вернуться к списку договоров", use_container_width=True,
key="back_to_contracts_list"):
                st.session_state.show_contract_success = False
                st.rerun()
        return
    tenants = db.get_all_tenants()
    rooms = db.get_all_rooms()
    if tenants.empty or rooms.empty:
        st.error("Недостаточно данных для создания договора")
    else:
        if st.session_state.contract_creation_in_progress:
            st.warning("Идет процесс создания договора. Пожалуйста, подождите...")
            return
        with st.form("add_contract_form", clear_on_submit=True):
            col1, col2 = st.columns(2)
            with col1:
                tenant_list = []
                tenant_mapping = {}
                for _, tenant in tenants.iterrows():
                    tenant_info = f"{tenant['ФИО']} (Тел: {tenant['Номер_телефона']})"
                    tenant_list.append(tenant_info)
                    tenant_mapping[tenant_info] = tenant['ID_арендатора']

                selected_tenant = st.selectbox("Арендатор", tenant_list,
key="select_tenant_new")
                tenant_id = tenant_mapping[selected_tenant] if selected_tenant else None
                free_rooms = rooms[rooms['Статус'] == 'Свободно']
                room_list = []
                room_mapping = {}
                for _, room in free_rooms.iterrows():
                    room_info = f"№{room['N_помещения']} (Этаж {room['Этаж']},
{room['Площадь']}м², {room['Стоимость_аренды']}Р/мес)"
                    room_list.append(room_info)
                    room_mapping[room_info] = room['N_помещения']
                selected_room = st.selectbox("Помещение", room_list, key="select_room_new")
                room_id = room_mapping[selected_room] if selected_room else None
            with col2:
                employee_id = st.session_state.user_data.get('ID_сотрудника', 1)
                st.info(f"Оформляет: {st.session_state.user_data.get('ФИО', 'Менеджер')}")

```

```

        today = datetime.now().date()
        start_date = st.date_input("Дата начала", value=today, key="start_date_new")
        end_date = st.date_input("Дата окончания", value=today + timedelta(days=365),
key="end_date_new")
        st.markdown("#### Информация о торговой точке")
        col3, col4 = st.columns(2)
        with col3:
            category = st.selectbox(
                "Категория магазина",
                ["Одежда", "Продукты", "Электроника", "Косметика", "Ресторан", "Кафе",
"Другое"],
                key="category_new")
        with col4:
            shop_name = st.text_input(
                "Название магазина",
                placeholder="Введите название магазина",
                value="",
                key="shop_name_new")
        col_save, col_cancel = st.columns(2)
        with col_save:
            submitted = st.form_submit_button(
                "Создать договор",
                type="primary",
                use_container_width=True)
        with col_cancel:
            canceled = st.form_submit_button(
                "Отмена",
                use_container_width=True)
        if submitted:
            if not tenant_id or not room_id:
                st.error("Выберите арендатора и помещение")
            elif start_date >= end_date:
                st.error("Дата окончания должна быть позже даты начала")
            elif not shop_name.strip():
                st.error("Введите название магазина")
            else:
                contract_key = f"{tenant_id}_{room_id}_{start_date}_{end_date}"
                if contract_key in st.session_state.created_contracts:
                    st.error("Этот договор уже был создан в текущей сессии.")
                    st.info("Если нужно создать новый договор, обновите страницу или
выберите другие параметры.")
                return
            st.session_state.contract_creation_in_progress = True
            room_data = free_rooms[free_rooms['N_помещения'] == room_id].iloc[0]
            rent_cost = room_data['Стоимость_аренды']
            with st.spinner("Создание договора..."):
                contract_success = db.add_contract(
                    tenant_id=tenant_id,
                    room_id=room_id,
                    employee_id=employee_id,
                    start_date=start_date,
                    end_date=end_date)

```

```

if contract_success:
    max_query = "SELECT MAX(N_договора) as max_id FROM Договор"
    max_result = db.execute_query(max_query, fetch=True)
    if not max_result.empty:
        new_contract_id = int(max_result.iloc[0]['max_id'])
        st.session_state.last_created_contract = {
            'id': new_contract_id,
            'tenant_id': tenant_id,
            'room_id': room_id,
            'start_date': start_date.strftime('%d.%m.%Y'),
            'end_date': end_date.strftime('%d.%m.%Y'),
            'shop_name': shop_name.strip()}
        st.session_state.created_contracts[contract_key] = {
            'contract_id': new_contract_id,
            'timestamp': datetime.now()}
        shop_query = """
        INSERT INTO Торговая_точка (
            N_торговой_точки,
            N_помещения,
            категория_торговой_точки,
            стоимость_аренды,
            Название_магазина,
            Статус_договора
        ) VALUES (?, ?, ?, ?, ?, N'Активен')
        """
        shop_params = [
            new_contract_id,
            int(room_id),
            category,
            float(rent_cost),
            shop_name.strip()
        ]
        shop_result = db.execute_query(shop_query, shop_params,
fetch=False)

        if shop_result:
            st.session_state.show_contract_success = True
            st.session_state.contract_creation_in_progress = False
            time.sleep(1)
            st.rerun()
        else:
            st.error(" Ошибка при создании торговой точки")
            st.session_state.contract_creation_in_progress = False
        else:
            st.error("Не удалось получить ID созданного договора")
            st.session_state.contract_creation_in_progress = False
        else:
            st.error("Ошибка при оформлении договора")
            st.session_state.contract_creation_in_progress = False
    if canceled:
        st.session_state.contract_creation_in_progress = False
        st.rerun()
with tab3:

```

```

st.subheader("Расторжение договоров")
contracts_for_termination = db.get_active_contracts_for_termination()
    if contracts_for_termination.empty:
        st.info("Нет активных договоров для расторжения")
    else:
        st.markdown("#### Выберите договор для расторжения")
        col_filter1, col_filter2 = st.columns(2)
        with col_filter1:
            tenant_options = ["Все арендаторы"] +
sorted(contracts_for_termination['Арендатор_ФИО'].dropna().unique().tolist())
            selected_tenant = st.selectbox("Фильтр по арендатору", tenant_options,
key="termination_tenant")
        with col_filter2:
            shop_options = ["Все магазины"] +
sorted(contracts_for_termination['Название_магазина'].dropna().unique().tolist())
            selected_shop = st.selectbox("Фильтр по магазину", shop_options,
key="termination_shop")
        filtered_for_termination = contracts_for_termination.copy()
        if selected_tenant != "Все арендаторы":
            filtered_for_termination =
filtered_for_termination[filtered_for_termination['Арендатор_ФИО'] == selected_tenant]
        if selected_shop != "Все магазины":
            filtered_for_termination =
filtered_for_termination[filtered_for_termination['Название_магазина'] == selected_shop]
        if not filtered_for_termination.empty:
            contract_options = []
            contract_mapping = {}
            for _, contract in filtered_for_termination.iterrows():
                contract_num = contract.get('N_договора', '—')
                tenant_name = contract.get('Арендатор_ФИО', 'Неизвестно')
                shop_name = contract.get('Название_магазина', 'Без названия')
                room_num = contract.get('N_помещения', '—')
                end_date = contract.get('Дата_окончания', '—')
                if hasattr(end_date, 'strftime'):
                    end_date = end_date.strftime('%d.%m.%Y')
                option_text = f"Договор №{contract_num} - {tenant_name} (помещ.
№{room_num}, {shop_name}) до {end_date}"
                contract_options.append(option_text)
                contract_mapping[option_text] = contract_num
            selected_contract_text = st.selectbox("Выберите договор", contract_options,
key="contract_select_terminate")
            if selected_contract_text and not st.session_state.termination_in_progress:
                contract_num = contract_mapping.get(selected_contract_text)
                if not contract_num:
                    st.error("Не удалось определить номер договора")
                    return
                contract_info = filtered_for_termination[filtered_for_termination['N_договора']
== contract_num]
                if contract_info.empty:
                    st.error("Информация о договоре не найдена")
                    return
                contract_info = contract_info.iloc[0]

```

```

st.markdown("---")
st.markdown("### Информация о выбранном договоре")
col_info1, col_info2 = st.columns(2)
with col_info1:
    st.write(f"***Номер договора:** {contract_info['N_договора']}")
    st.write(f"***Арендатор:** {contract_info['Арендатор_ФИО']}")
    st.write(f"***Магазин:** {contract_info.get('Название_магазина', '—')}")
with col_info2:
    st.write(f"***Помещение:** №{contract_info['N_помещения']}")
    st.write(f"***Этаж:** {contract_info['Этаж']}")
    st.write(f"***Площадь:** {contract_info['Площадь']} м²")
    st.write(f"***Стоимость:** {contract_info.get('Тариф', 0):.0f} Р/мес")
st.markdown("---")
st.warning("Внимание: расторжение договора освободит помещение!")
col_btn1, col_btn2 = st.columns(2)
with col_btn1:
    if st.button("Расторгнуть договор",
                  key=f"terminate_btn_{contract_num}",
                  type="primary",
                  use_container_width=True,
                  disabled=st.session_state.termination_in_progress):
        st.session_state.termination_in_progress = True
        st.session_state.terminated_contract = contract_num
        with st.spinner("Расторжение договора..."):
            try:
                success = db.terminate_contract(contract_num)
                if success:
                    st.success(f"Договор №{contract_num} успешно расторгнут!")
                    time.sleep(2)
                    st.session_state.termination_in_progress = False
                    st.session_state.terminated_contract = None
                    st.rerun()
                else:
                    st.error("Ошибка при расторжении договора")
                    st.session_state.termination_in_progress = False
                    st.session_state.terminated_contract = None
            except Exception as e:
                st.error(f"Ошибка: {str(e)}")
                st.session_state.termination_in_progress = False
                st.session_state.terminated_contract = None
with col_btn2:
    if st.button("Отмена",
                  key=f"cancel_btn_{contract_num}",
                  use_container_width=True):
        st.session_state.termination_in_progress = False
        st.session_state.terminated_contract = None
        st.rerun()
    if st.session_state.termination_in_progress:
        st.info(f"Идет расторжение договора №{st.session_state.terminated_contract}...")
    else:

```

```

        st.info("Нет договоров, соответствующих фильтру")
except Exception as e:
    st.session_state.contract_creation_in_progress = False
    st.session_state.termination_in_progress = False
    st.error(f"Ошибка загрузки данных: {str(e)}")
    import traceback
    traceback.print_exc()
def add_review_page():
    """Добавление отзыва"""
    st.markdown('<div class="section-header">Оставить отзыв</div>',
unsafe_allow_html=True)
    try:
        if 'rating' not in st.session_state:
            st.session_state.rating = 5
        if st.button("← Назад к отзывам", use_container_width=True):
            st.session_state.page = "reviews"
            st.rerun()
        st.markdown("<br>", unsafe_allow_html=True)
        shops = db.get_shops_for_reviews()
        if shops.empty():
            st.error("Нет доступных магазинов для отзыва.")
            return
        with st.form("add_review_form", clear_on_submit=True):
            st.markdown("***Основная информация***")
            shop_options = []
            shop_mapping = {}
            for _, shop in shops.iterrows():
                shop_name = shop.get('Название_магазина', 'Магазин')
                shop_id = shop.get('N_торговой_точки')
                shop_options.append(shop_name)
                shop_mapping[shop_name] = shop_id
            shop_options = sorted(list(set(shop_options)))
            selected_shop = st.selectbox(
                "Магазин",
                shop_options,
                help="Выберите магазин")
            author = st.text_input(
                "Имя (необязательно)",
                placeholder="Можно оставить пустым",
                help="Анонимно, если не указано")
            st.markdown("***Оценка***")
            rating = st.slider(
                "",
                min_value=1,
                max_value=5,
                value=st.session_state.rating,
                step=1,
                key="rating_slider",
                label_visibility="collapsed")
            st.session_state.rating = rating
            st.markdown("***Текст отзыва***")
            description = st.text_area(

```



```

        "",
        height=60,
        placeholder="Напишите отзыв здесь...",
        label_visibility="collapsed")
    col1, col2 = st.columns(2)
    with col1:
        submitted = st.form_submit_button(
            "Отправить",
            type="primary",
            use_container_width=True)
    with col2:
        cancel_clicked = st.form_submit_button(
            "Отмена",
            use_container_width=True)
    if cancel_clicked:
        st.session_state.page = "reviews"
        st.rerun()
    if submitted:
        if not description or len(description.strip()) < 5:
            st.error("Минимум 5 символов")
        else:
            if not author or not author.strip():
                author = "Аноним"
            shop_id = shop_mapping[selected_shop]
            with st.spinner("Сохранение..."):
                success = db.add_review(shop_id, description.strip(), author.strip(), rating)
            if success:
                st.success("Спасибо!")
                time.sleep(1)
                st.session_state.page = "reviews"
                st.rerun()
            else:
                st.error("Ошибка")
    except Exception as e:
        st.error(f"Ошибка: {str(e)}")
def tenant_shops_page():
    """Просмотр торговых точек арендатора - ИСПРАВЛЕННАЯ ВЕРСИЯ"""
    if st.session_state.user_type != 'tenant':
        st.error("Доступно только для арендаторов")
        return
    st.title("Мои торговые точки")
    if st.button("Назад в личный кабинет"):
        st.session_state.page = "dashboard"
        st.rerun()
    try:
        user_data = st.session_state.user_data
        tenant_id = user_data.get('ID_арендатора')
        tenant_name = user_data.get('ФИО', 'Арендатор')
        st.markdown(f"### Здравствуйте, {tenant_name}!")
        st.markdown(f"**ID арендатора:** {tenant_id}")
        st.markdown("---")
        with st.spinner("Загрузка торговых точек..."):

```

```

shops_data = db.get_tenant_shops(tenant_id)
if shops_data.empty:
    st.info("У вас пока нет торговых точек")
    st.markdown("---")
    st.markdown("### Что вы можете сделать:")
    col1, col2, col3 = st.columns(3)
    with col1:
        if st.button("Посмотреть договоры",
                      key="view_contracts_from_shops",
                      use_container_width=True):
            st.session_state.page = "tenant_contracts"
            st.rerun()
    with col2:
        if st.button("Забронировать помещение",
                      key="book_room_from_shops",
                      use_container_width=True):
            st.session_state.page = "create_booking"
            st.rerun()
    with col3:
        if st.button("Связаться с менеджером",
                      key="contact_manager_from_shops",
                      use_container_width=True):
            st.info("Менеджер по телефону: +7 (495) 123-45-67")
    return
st.markdown("### Фильтры и сортировка")
col_filter1, col_filter2, col_filter3 = st.columns(3)
with col_filter1:
    all_statuses = ["Все статусы"] + sorted(shops_data['Статус'].dropna().unique().tolist())
    status_filter = st.selectbox("Статус договора", all_statuses)
with col_filter2:
    all_categories = ["Все категории"] +
sorted(shops_data['категория_торговой_точки'].dropna().unique().tolist())
    category_filter = st.selectbox("Категория магазина", all_categories)
with col_filter3:
    sort_options = ["По дате (новые)", "По дате (старые)", "По названию", "По
стоимости", "По статусу"]
    sort_by = st.selectbox("Сортировка", sort_options)
    filtered_shops = shops_data.copy()
    if status_filter != "Все статусы":
        filtered_shops = filtered_shops[filtered_shops['Статус'] == status_filter]
    if category_filter != "Все категории":
        filtered_shops = filtered_shops[filtered_shops['категория_торговой_точки'] ==
category_filter]
    if sort_by == "По дате (новые)":
        filtered_shops = filtered_shops.sort_values('Дата_оформления', ascending=False)
    elif sort_by == "По дате (старые)":
        filtered_shops = filtered_shops.sort_values('Дата_оформления', ascending=True)
    elif sort_by == "По названию":
        filtered_shops = filtered_shops.sort_values('Название_магазина', ascending=True)
    elif sort_by == "По стоимости":
        filtered_shops = filtered_shops.sort_values('стоимость_аренды', ascending=False)
    elif sort_by == "По статусу":

```

```

status_order = {"Активен": 1, "Истек": 2, "Расторгнут": 3}
filtered_shops['Статус_порядок'] = filtered_shops['Статус'].map(lambda x:
status_order.get(x, 4))
filtered_shops = filtered_shops.sort_values('Статус_порядок')
st.write(f'***Найдено:** {len(filtered_shops)} торговых точек')
st.markdown("---")
for index, shop in filtered_shops.iterrows():
    with st.container():
        shop_id = shop['N_торговой_точки']
        shop_name = shop['Название_магазина']
        category = shop['категория_торговой_точки']
        status = shop['Статус']
        room_num = shop['N_помещения']
        floor = shop['Этаж']
        area = shop['Площадь']
        rent_cost = shop['стоимость_аренды']
        contract_id = shop['N_договора']
        start_date = shop.get('Дата_оформления_формат', '-')
        end_date = shop.get('Дата_окончания_формат', '-')
        if status == 'Активен':
            status_color = "#28a745"
            status_text = "Активен"
        elif status == 'Истек':
            status_color = "#6c757d"
            status_text = "Истек"
        elif status == 'Расторгнут':
            status_color = "#dc3545"
            status_text = "Расторгнут"
        else:
            status_color = "#ffc107"
            status_text = f"{status}"
        st.markdown(f"### {shop_name}")
        col_info1, col_info2 = st.columns(2)
        with col_info1:
            st.markdown(f'***Категория:** {category}')
            st.markdown(f'***Помещение:** №{room_num}, {floor} этаж')
            st.markdown(f'***Площадь:** {area} м²')
            st.markdown(f'***Договор:** №{contract_id}')
        with col_info2:
            st.markdown(f'***Статус:** <span style=color:{status_color}; font-
weight:bold;>{status_text}</span>', unsafe_allow_html=True)
            st.markdown(f'***Стоимость аренды:** {int(rent_cost):,} Р/мес')
            st.markdown(f'***Период:** {start_date} – {end_date}')
            st.markdown(f'***ID торговой точки:** {shop_id}')
        st.markdown("<br>", unsafe_allow_html=True)
        col_btn1, col_btn2, col_btn3 = st.columns(3)
        with col_btn1:
            if st.button("Отзывы",
                key=f"reviews_{shop_id}_{index}",
                use_container_width=True):
                st.session_state.selected_shop_id = shop_id
                st.session_state.selected_shop_name = shop_name

```

```

        st.session_state.page = "tenant_shop_reviews"
        st.rerun()

    with col_btn2:
        if st.button("Платежи",
                    key=f"payments_{contract_id}_{index}",
                    use_container_width=True):
            st.session_state.selected_contract = contract_id
            st.session_state.selected_shop_name = shop_name
            st.session_state.page = "tenant_shop_payments"
            st.rerun()
    with col_btn3:
        if status == 'Активен':
            if st.button("Расторгнуть",
                        key=f"terminate_{contract_id}_{index}",
                        use_container_width=True,
                        type="secondary"):
                st.session_state.terminate_contract_id = contract_id
                st.session_state.selected_shop_name = shop_name
                st.session_state.page = "tenant_terminate"
                st.rerun()
        else:
            st.button("Неактивно",
                    key=f"inactive_{contract_id}_{index}",
                    use_container_width=True,
                    disabled=True)
    if index < len(filtered_shops) - 1:
        st.markdown("---")
    st.markdown("---")
    st.markdown("### Статистика торговых точек")
    active_count = len(shops_data[shops_data['Статус'] == 'Активен'])
    expired_count = len(shops_data[shops_data['Статус'] == 'Истек'])
    terminated_count = len(shops_data[shops_data['Статус'] == 'Расторгнут'])
    total_rent = shops_data['стоимость_аренды'].sum() if 'стоимость_аренды' in
shops_data.columns else 0
    col_stat1, col_stat2, col_stat3, col_stat4 = st.columns(4)
    with col_stat1:
        st.metric("Всего точек", len(shops_data))
    with col_stat2:
        st.metric("Активных", active_count)
    with col_stat3:
        st.metric("Аренда в месяц", f"{int(total_rent):,} P")
    with col_stat4:
        st.metric("Категорий", shops_data['категория_торговой_точки'].nunique())
    st.markdown("---")
    st.markdown("### Дополнительные действия")
    col_action1, col_action2 = st.columns(2)
    with col_action1:
        if st.button("Обновить список",
                    key="refresh_shops_list",
                    use_container_width=True):
            st.rerun()

```

```

with col_action2:
    if st.button("Посмотреть все договоры",
                  key="view_all_contracts",
                  use_container_width=True):
        st.session_state.page = "tenant_contracts"
        st.rerun()
except Exception as e:
    st.error(f"Ошибка загрузки торговых точек: {str(e)}")
    import traceback
    traceback.print_exc()
    if st.button("Попробовать снова", key="retry_shops"):
        st.rerun()
def tenant_shop_payments_page():
    """Платежи по конкретной торговой точке"""
    if 'selected_contract' not in st.session_state:
        st.error("Договор не выбран")
    if st.button("Вернуться к моим точкам"):
        st.session_state.page = "tenant_shops"
        st.rerun()
    return
    contract_id = st.session_state.selected_contract
    shop_name = st.session_state.get('selected_shop_name', f"Договор №{contract_id}")
    st.markdown(f"<div class='section-header'>Платежи: {shop_name}</div>",
unsafe_allow_html=True)
    if st.button("← Назад к торговым точкам",
                  key="back_to_shops_from_payments",
                  use_container_width=True):
        st.session_state.page = "tenant_shops"
        st.rerun()
try:
    all_payments = db.get_all_payments()
    if all_payments.empty():
        st.info("Пока нет платежей по этому договору")
        return
    contract_payments = all_payments[all_payments['N_договора'] == contract_id]
    if contract_payments.empty():
        st.info("Пока нет платежей по этому договору")
        return
    total_payments = len(contract_payments)
    total_amount = contract_payments['Сумма'].sum()
    avg_payment = contract_payments['Сумма'].mean()
    col1, col2, col3 = st.columns(3)
    with col1:
        st.markdown(f"
<div class='stat-box'>
    <div class='stat-label'>Всего платежей</div>
    <div class='stat-value'>{total_payments}</div>
</div>
", unsafe_allow_html=True)
        with col2:
            st.markdown(f"
<div class='stat-box'>

```

```

        <div class="stat-label">Общая сумма</div>
        <div class="stat-value">{int(total_amount):,} Р</div>
    </div>
    "", unsafe_allow_html=True)
with col3:
    st.markdown(f"""
    <div class="stat-box">
        <div class="stat-label">Средний платеж</div>
        <div class="stat-value">{int(avg_payment):,} Р</div>
    </div>
    "", unsafe_allow_html=True)
st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
col_date1, col_date2 = st.columns(2)
with col_date1:
    start_date = st.date_input("Дата от",
                                value=datetime.now().date() - timedelta(days=90))
    with col_date2:
        end_date = st.date_input("Дата до",
                                   value=datetime.now().date())
    filtered_payments = contract_payments.copy()
    if 'дата_оплаты' in filtered_payments.columns:
        filtered_payments['дата_оплаты_parsed'] =
pd.to_datetime(filtered_payments['дата_оплаты'], errors='coerce')
    mask = (filtered_payments['дата_оплаты_parsed'] >= pd.Timestamp(start_date)) & \
        (filtered_payments['дата_оплаты_parsed'] <= pd.Timestamp(end_date))
    filtered_payments = filtered_payments.loc[mask]
    if not filtered_payments.empty:
        st.markdown(f"### История платежей ({len(filtered_payments)})")
        display_payments = filtered_payments.copy()
        if 'дата_оплаты' in display_payments.columns:
            display_payments['дата_оплаты'] = display_payments['дата_оплаты'].apply(
                lambda x: x.strftime('%d.%m.%Y') if pd.notnull(x) and hasattr(x, 'strftime') else
str(x))
        if 'Сумма' in display_payments.columns:
            display_payments['Сумма'] = display_payments['Сумма'].apply(
                lambda x: f"{int(float(x)):,} Р" if pd.notnull(x) else "")
        display_payments = display_payments[['№ операции', 'дата_оплаты', 'Сумма']]
        display_payments.columns = ['№ операции', 'Дата оплаты', 'Сумма']
        st.dataframe(display_payments, use_container_width=True, hide_index=True)
    else:
        st.info("Нет платежей за выбранный период")
st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
if st.button("Совершить новый платеж",
            use_container_width=True,
            key="new_payment_shop"):
    st.session_state.page = "tenant_payments"
    st.rerun()
except Exception as e:
    st.error(f"Ошибка загрузки платежей: {str(e)}")
    import traceback
    traceback.print_exc()
def tenant_contracts_page():

```

```

if st.session_state.user_type != 'tenant':
    st.error("Доступ только для арендаторов")
    return

st.title("Мои договоры аренды")
if st.button("Назад в личный кабинет"):
    st.session_state.page = "dashboard"
    st.rerun()
try:
    tenant_id = st.session_state.user_data.get('ID_арендатора')
    contracts = db.get_all_tenant_contracts(tenant_id)
    if contracts.empty:
        st.info("У вас пока нет договоров аренды")
    if st.button("Найти помещение для аренды"):
        st.session_state.page = "tenant_booking"
        st.rerun()
    return
st.markdown("---")
all_statuses = ["Все договоры"] + sorted(contracts['Статус'].dropna().unique().tolist())
selected_status = st.selectbox("Фильтр по статусу договора", all_statuses)
if selected_status != "Все договоры":
    contracts = contracts[contracts['Статус'] == selected_status]
st.write(f"***Найдено договоров:** {len(contracts)}")
for _, contract in contracts.iterrows():
    with st.container():
        contract_num = contract.get('N_договора', '—')
        status = contract.get('Статус', 'Неизвестно')
        shop_name = contract.get('Название_магазина', '—')
        room = contract.get('N_помещения', '—')
        floor = contract.get('Этаж', '—')
        area = contract.get('Площадь', 0)
        start_date = contract.get('Дата_оформления_формат', '—')
        end_date = contract.get('Дата_окончания_формат', '—')
        price = f"{int(contract.get('Тариф', 0))}, П/мес"
        if status == 'Активен':
            status_color = "green"
            status_display = f":{status_color}***{status}***"
        elif status == 'Расторгнут':
            status_color = "red"
            status_display = f":{status_color}***{status}***"
        elif status == 'Истек':
            status_color = "gray"
            status_display = f":{status_color}***{status}***"
        else:
            status_display = f"***{status}***"
        st.markdown(f"### Договор №{contract_num} • {status_display}")
        st.markdown(f"***Магазин:** {shop_name}")
        st.markdown(f"***Помещение:** №{room}, {floor} этаж • **Площадь:** {area} м²")
        st.markdown(f"***Период:** {start_date} – {end_date}")
        st.markdown(f"***Стоимость:** {price}")
    col1, col2, col3 = st.columns(3)

```

```

with col1:
    if st.button("Оплатить", key=f"pay_{contract_num}", use_container_width=True):
        st.session_state.selected_contract = contract_num
        st.session_state.page = "tenant_payments"
        st.rerun()
with col2:
    if st.button("Платежи", key=f"view_{contract_num}", use_container_width=True):
        st.session_state.selected_contract = contract_num
        st.session_state.page = "tenant_shop_payments"
        st.rerun()

with col3:
    if status == 'Активен':
        if st.button("Расторгнуть", key=f"terminate_{contract_num}",
                    use_container_width=True, type="secondary"):
            st.session_state.terminate_contract_id = contract_num
            st.session_state.page = "tenant_terminate"
            st.rerun()
    else:
        st.button(
            "Расторгнут" if status == 'Расторгнут' else "Истёк",
            key=f"inactive_{contract_num}",
            use_container_width=True,
            disabled=True)
    st.markdown("---")
except Exception as e:
    st.error(f"Ошибка при загрузке договоров: {str(e)}")
    if st.button("Попробовать снова"):
        st.rerun()

def tenant_contracts_page():
    if st.session_state.user_type != 'tenant':
        st.error("Доступ только для арендаторов")
        return
    st.markdown('<div class="section-header">Мои договоры аренды</div>',
unsafe_allow_html=True)
    if st.button("Назад в личный кабинет",
                key="back_to_dashboard_from_contracts",
                use_container_width=True):
        st.session_state.page = "dashboard"
        st.rerun()
    try:
        tenant_id = st.session_state.user_data.get('ID_арендатора')
        contracts = db.get_all_tenant_contracts(tenant_id)
        if contracts.empty:
            st.info("У вас пока нет договоров аренды")
            if st.button("Найти помещение для аренды",
                        key="find_room_for_rent",
                        use_container_width=True):
                st.session_state.page = "tenant_booking"
                st.rerun()
        return

```



```

st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
all_statuses = ["Все договоры"] + sorted(contracts['Статус'].dropna().unique().tolist())
selected_status = st.selectbox("Фильтр по статусу договора", all_statuses)
    if selected_status != "Все договоры":
        contracts = contracts[contracts['Статус'] == selected_status]
st.write(f"Найдено договоров:** {len(contracts)}")
st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
for index, contract in contracts.iterrows():
    with st.container():
        contract_num = contract.get('N_договора', '—')
        status = contract.get('Статус', 'Неизвестно')
        shop_name = contract.get('Название_магазина', '—')
        room = contract.get('N_помещения', '—')
        floor = contract.get('Этаж', '—')
        area = contract.get('Площадь', 0)
        start_date = contract.get('Дата_оформления_формат', '—')
        end_date = contract.get('Дата_окончания_формат', '—')
        price = f"{int(contract.get('Тариф', 0))}:,} Р/мес"
        if status == 'Активен':
            status_color = "green"
            status_display = f":{status_color}**{status}**"
        elif status == 'Расторгнут':
            status_color = "red"
            status_display = f":{status_color}**{status}**"
        elif status == 'Истек':
            status_color = "gray"
            status_display = f":{status_color}**{status}**"
        else:
            status_display = f"***{status}***"
        st.markdown(f"### Договор №{contract_num} • {status_display}")
        st.markdown(f"***Магазин:** {shop_name}")
        st.markdown(f"***Помещение:** №{room}, {floor} этаж • **Площадь:** {area}
м²")
        st.markdown(f"***Период:** {start_date} – {end_date}")
        st.markdown(f"***Стоимость:** {price}")
        col1, col2 = st.columns(2)
        with col1:
            if st.button("Оплатить",
                        key=f"pay_{contract_num}_{index}",
                        use_container_width=True):
                st.session_state.selected_contract = contract_num
                st.session_state.page = "tenant_payments"
                st.rerun()
        with col2:
            if st.button("Платежи",
                        key=f"view_{contract_num}_{index}",
                        use_container_width=True):
                st.session_state.selected_contract = contract_num
                st.session_state.page = "tenant_shop_payments"
                st.rerun()
    if index < len(contracts) - 1:
        st.markdown("---")

```

```

except Exception as e:
    st.error(f"Ошибка при загрузке договоров: {str(e)}")
    if st.button("Попробовать снова",
                key="retry_contracts",
                use_container_width=True):
        st.rerun()
def tenant_payments_page():
    """Оплата коммунальных услуг и аренды арендатором"""
    st.markdown('<div class="section-header">Оплата коммунальных услуг и аренды</div>',
unsafe_allow_html=True)
    if st.session_state.user_type != 'tenant':
        st.error("Доступно только для арендаторов")
        return
    if 'payment_processing' not in st.session_state:
        st.session_state.payment_processing = False
    if 'payment_complete' not in st.session_state:
        st.session_state.payment_complete = False
    if 'payment_details' not in st.session_state:
        st.session_state.payment_details = None
    try:
        user_data = st.session_state.user_data
        tenant_id = user_data.get('ID_арендатора')
        if st.session_state.payment_complete and st.session_state.payment_details:
            show_payment_confirmation()
            return
        show_payment_form(tenant_id)
    except Exception as e:
        st.error(f"Ошибка загрузки данных: {str(e)}")
def show_payment_form(tenant_id):
    """Показать форму оплаты"""
    contracts = db.get_all_contracts()
    payments = db.get_all_payments()
    if contracts.empty:
        st.info("У вас нет действующих договоров")
        return
    tenant_contracts = contracts[contracts['ID_арендатора'] == tenant_id]
    if tenant_contracts.empty:
        st.info("У вас нет действующих договоров")
        return
    if not payments.empty:
        tenant_contract_ids = tenant_contracts['N_договора'].tolist()
        tenant_payments = payments[payments['N_договора'].isin(tenant_contract_ids)]
        if not tenant_payments.empty:
            st.markdown("### История платежей")
            display_payments = tenant_payments.copy()
            if 'дата_оплаты' in display_payments.columns:
                display_payments['дата_оплаты'] = display_payments['дата_оплаты'].apply(
                    lambda x: x.strftime('%d.%m.%Y') if pd.notnull(x) and hasattr(x, 'strftime') else
str(x))
            if 'Сумма' in display_payments.columns:
                display_payments['Сумма'] = display_payments['Сумма'].apply(
                    lambda x: f'{int(float(x)):.} Р' if pd.notnull(x) else "—" )

```

```

display_payments = display_payments[['N_операции', 'N_договора', 'дата_оплаты',
'Sумма']]
display_payments.columns = ['№ операции', '№ договора', 'Дата оплаты', 'Сумма']
st.dataframe(display_payments, use_container_width=True, hide_index=True)
st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
st.markdown("### Совершить новый платеж")
contract_options = []
contract_info = {}
for _, contract in tenant_contracts.iterrows():
    contract_id = contract.get('N_договора')
    room_num = contract.get('N_помещения')

    try:
        if pd.isna(contract_id):
            continue
        contract_id_int = int(float(contract_id))
    except (ValueError, TypeError):
        continue
    tariff = contract.get('Тариф', 0)
    contract_str = f"Договор №{contract_id_int} (Помещение №{room_num}, {int(tariff):,}
Р/мес)"
    contract_options.append(contract_str)
    contract_info[contract_str] = {
        'id': contract_id_int,
        'room': room_num,
        'tariff': tariff}
if not contract_options:
    st.error("Нет доступных договоров для оплаты")
    return
with st.form(key=f"payment_form_tenant_{tenant_id}", clear_on_submit=False):
    selected_contract = st.selectbox(
        "Выберите договор для оплаты",
        contract_options)
    amount = st.number_input(
        "Сумма оплаты (Р)",
        min_value=100,
        max_value=1000000,
        value=50000,
        step=1000,
        key="amount_input")
    payment_date = datetime.now().date()
    st.info(f"***Дата оплаты:** {payment_date.strftime('%d.%m.%Y')}")
    col1, col2 = st.columns(2)
    with col1:
        submit_button = st.form_submit_button("Оплатить", type="primary",
use_container_width=True)
    with col2:
        cancel_button = st.form_submit_button("Отмена", use_container_width=True)
    if cancel_button:
        st.rerun()
    if submit_button:
        if selected_contract and amount > 0:

```

```

contract_data = contract_info[selected_contract]
contract_id = contract_data['id']
with st.spinner("Обработка платежа..."):
    time.sleep(1)
    success = db.add_payment(contract_id, payment_date, amount)
    if success:
        st.session_state.payment_complete = True
        st.session_state.payment_details = {
            'contract_id': contract_id,
            'amount': amount,
            'date': payment_date,
            'room': contract_data['room'],
            'tenant_name': st.session_state.user_data.get('ФИО', 'Арендатор'),
            'tenant_phone': st.session_state.user_data.get('Номер_телефона', 'Не указан')
        }
        payments = db.get_all_payments()
        if not payments.empty:
            recent_payments = payments[
                (payments['N_договора'] == contract_id) &
                (payments['Сумма'] == amount)
            ]
            if not recent_payments.empty:
                st.session_state.payment_details['operation_id'] = int(recent_payments.iloc[-
1]['N_операции'])
            else:
                st.session_state.payment_details['operation_id'] = int(payments.iloc[-
1]['N_операции'])

        st.rerun()
    else:
        st.error("Ошибка при обработке платежа. Проверьте подключение к базе
данных.")
    else:
        if not selected_contract:
            st.error("Выберите договор для оплаты")
        if amount <= 0:
            st.error("Сумма оплаты должна быть больше 0")
def generate_pdf_receipt(payment_details):
    """Генерация PDF чека"""
    try:
        from reportlab.lib.pagesizes import A4
        from reportlab.pdfgen import canvas
        from reportlab.lib.units import cm
        from reportlab.pdfbase import pdfmetrics
        from reportlab.pdfbase.ttfonts import TTFont
        import io
        buffer = io.BytesIO()
        c = canvas.Canvas(buffer, pagesize=A4)
        width, height = A4
        try:
            pdfmetrics.registerFont(TTFont('DejaVu', 'DejaVuSans.ttf'))
            font_name = 'DejaVu'

```

```

except:
    try:
        pdfmetrics.registerFont(TTFont('ArialUnicode', 'arial.ttf'))
        font_name = 'ArialUnicode'
    except:
        font_name = 'Helvetica'
c.setFont(font_name, 12)
c.setFont(font_name, 16)
c.drawCentredString(width/2, height - 2*cm, "ТОРГОВЫЙ ЦЕНТР")
c.setFont(font_name, 12)
c.drawCentredString(width/2, height - 2.5*cm, "Адрес: Москва, ул. Торговая, 1")
c.drawCentredString(width/2, height - 3*cm, "Телефон: +7 (495) 123-45-67")
c.line(2*cm, height - 3.5*cm, width - 2*cm, height - 3.5*cm)
operation_id = payment_details.get('operation_id', '0000')
date = payment_details['date'].strftime('%d.%m.%Y')
time_str = datetime.now().strftime('%H:%M')
c.setFont(font_name, 14)
c.drawCentredString(width/2, height - 4.5*cm, f"ЧЕК №{operation_id} • {date} • {time_str}")
c.setFont(font_name, 12)
c.drawCentredString(width/2, height - 5*cm, "СТАТУС: ОПЛАЧЕНО")
c.line(2*cm, height - 5.5*cm, width - 2*cm, height - 5.5*cm)
amount = int(payment_details['amount'])
c.setFont(font_name, 24)
c.drawCentredString(width/2, height - 7*cm, f"{amount:,.} Р".replace(',', ' '))
c.setFont(font_name, 12)
y_position = height - 8*cm
room = payment_details.get('room', "")
contract_id = payment_details.get('contract_id', "")
tenant_name = payment_details.get('tenant_name', "")
tenant_phone = payment_details.get('tenant_phone', "")
c.drawString(3*cm, y_position, f"Услуга: Аренда помещения ({room})")
y_position -= 0.7*cm
c.drawString(3*cm, y_position, f"Договор: {contract_id}")
y_position -= 0.7*cm
c.drawString(3*cm, y_position, f"Плательщик: {tenant_name}")
y_position -= 0.7*cm
c.drawString(3*cm, y_position, f"Телефон: {tenant_phone}")
y_position -= 1*cm
c.line(2*cm, y_position, width - 2*cm, y_position)
y_position -= 0.7*cm
c.drawString(3*cm, y_position, f"Сформировано: {date} {time_str}")
y_position -= 0.7*cm
c.drawString(3*cm, y_position, "Электронная квитанция • Действительна без печати")
c.drawString(3*cm, y_position - 0.5*cm, "Подпись: _____")
c.save()
buffer.seek(0)
return buffer.getvalue()
except Exception as e:
    print(f"Ошибка при создании PDF: {str(e)}")
    return generate_text_receipt(payment_details)
def generate_text_receipt(payment_details):

```

```

"""Создание текстового чека в указанном формате"""
operation_id = payment_details.get('operation_id', '0000')
date = payment_details['date'].strftime('%d.%m.%Y')
time = datetime.now().strftime('%H:%M')
amount = f'{int(payment_details['amount']):,} Р'.replace(',', ' ')
room = payment_details.get('room', '')
contract_id = payment_details.get('contract_id', '')
tenant_name = payment_details.get('tenant_name', '')
tenant_phone = payment_details.get('tenant_phone', '')

receipt_text = f"""

```

ТОРГОВЫЙ ЦЕНТР

Адрес: Москва, ул. Торговая, 1
Телефон: +7 (495) 123-45-67

ЧЕК №{operation_id} • {date} • {time} • ОПЛАЧЕНО

{amount}

Услуга: Аренда помещения ({room})
Договор: {contract_id}

Платательщик: {tenant_name}
Телефон: {tenant_phone}

Сформировано: {date} {time}

Электронная квитанция • Действительна без печати

"""

```

    return receipt_text.encode('utf-8')
def show_payment_confirmation():
    """Показать подтверждение оплаты"""
    payment_details = st.session_state.payment_details
    st.markdown(f"""
<div style="
    text-align: center;
    padding: 40px;
    margin: 30px 0;
    background: linear-gradient(135deg, #d4edda 0%, #c3e6cb 100%);
    border: 2px solid #28a745;
    border-radius: 15px;

```

```

">
    <div style="
        width: 80px;
        height: 80px;
        background: #28a745;
        border-radius: 50%;
        display: flex;
        align-items: center;
        justify-content: center;
        margin: 0 auto 20px auto;
    ">

        <span style="color: white; font-size: 40px; font-weight: bold;">✓</span>
    </div>
    <h2 style="color: #155724; margin-bottom: 10px;">
        Оплата успешно проведена
    </h2>
    <p style="color: #155724; font-size: 24px; font-weight: bold; margin: 20px 0;">
        {int(payment_details['amount']):,} ₺
    </p>
    <p style="color: #0c5460; font-size: 16px;">
        Договор №{payment_details['contract_id']}, помещение №{payment_details['room']}
    </p>
    <p style="color: #0c5460; font-size: 14px; margin-top: 10px;">
        Дата оплаты: {payment_details['date'].strftime('%d.%m.%Y')}
    </p>
    <p style="color: #0c5460; font-size: 14px; margin-top: 10px;">
        Номер операции: {payment_details.get('operation_id', '0000')}
    </p>
</div>
""", unsafe_allow_html=True)

st.markdown('<div style="margin-top: 40px; margin-bottom: 20px;"></div>',
unsafe_allow_html=True)
pdf_data = generate_pdf_receipt(payment_details)
if pdf_data:
    try:
        if pdf_data.startswith(b'%PDF'):
            mime_type = "application/pdf"
            file_extension = "pdf"
            label = "СКАЧАТЬ ЧЕК (PDF)"
        else:
            mime_type = "text/plain"
            file_extension = "txt"
            label = "СКАЧАТЬ ЧЕК (TXT)"
    except:
        mime_type = "application/pdf"
        file_extension = "pdf"
        label = "СКАЧАТЬ ЧЕК"
    st.download_button(
        label=label,
        data=pdf_data,

```

```

        file_name=f"чек_оплаты_{payment_details.get('operation_id',
'0000')}__{datetime.now().strftime('%Y%m%d_%H%M%S')}.{file_extension}",
        mime=mime_type,
        use_container_width=True,
        key="pdf_download",
        type="primary",
        help=f"Нажмите, чтобы скачать квитанцию об оплате в формате
{file_extension.upper()}"
    )
    else:
        st.error("Не удалось создать чек. Пожалуйста, обратитесь в службу поддержки.")
    st.markdown('<div style="margin-top: 40px;"></div>', unsafe_allow_html=True)
    col1, col2 = st.columns(2)
    with col1:
        if st.button("Совершить новый платеж",
            use_container_width=True,
            key="new_payment_btn"):
            st.session_state.payment_complete = False
            st.session_state.payment_processing = False
            st.session_state.payment_details = None
            st.rerun()
    with col2:
        if st.button("Вернуться в личный кабинет",
            use_container_width=True,
            key="back_to_dashboard"):
            st.session_state.page = "dashboard"
            st.session_state.payment_complete = False
            st.rerun()
def tenant_new_contract_page():
    """Страница заключения нового договора для арендатора"""
    st.markdown('<div class="section-header">Заключение нового договора аренды</div>',
unsafe_allow_html=True)
    if st.session_state.user_type != 'tenant':
        st.error("Доступно только для арендаторов")
        return
    try:
        user_data = st.session_state.user_data
        tenant_id = user_data.get('ID_арендатора')
        col_back, col_refresh = st.columns([3, 1])
        with col_back:
            if st.button("← Вернуться в личный кабинет",
                key="back_to_dashboard_new_contract",
                use_container_width=True):
                st.session_state.page = "dashboard"
                st.rerun()
        with col_refresh:
            if st.button("Обновить список",
                key="refresh_contracts_data",
                use_container_width=True):
                st.rerun()
    st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
    st.markdown("### Информация о вас")

```



```

st.markdown(f'***ФИО:** {user_data.get('ФИО', 'Не указано')}')
st.markdown(f'***ID арендатора:** {tenant_id}')
st.markdown(f'***Телефон:** {user_data.get('Номер_телефона', 'Не указан')}')
st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
rooms = db.get_all_rooms()
free_rooms = rooms[rooms['Статус'] == 'Свободно']
if free_rooms.empty():
    st.info("В данный момент нет свободных помещений для аренды")
    st.markdown("---")
    st.markdown("### Что можно сделать:")
    col_option1, col_option2 = st.columns(2)
    with col_option1:
        if st.button("Забронировать помещение",
                      key="book_room_option",
                      use_container_width=True):
            st.info("Обратитесь к менеджеру для бронирования помещения")
    with col_option2:
        if st.button("Связаться с менеджером",
                      key="contact_manager_option",
                      use_container_width=True):
            st.info("Менеджер по телефону: +7 (495) 123-45-67")
        if st.button("Вернуться в личный кабинет",
                      key="back_to_dashboard_from_empty",
                      use_container_width=True):
            st.session_state.page = "dashboard"
            st.rerun()
    return
st.markdown("### Доступные помещения для аренды")
for _, room in free_rooms.iterrows():
    with st.container():
        col1, col2, col3 = st.columns([2, 1, 1])
        with col1:
            st.markdown(f'***Помещение №{room['N_помещения']}***')
            st.markdown(f'Этаж: {room['Этаж']}')
            st.markdown(f'Площадь: {room['Площадь']} м²')
        with col2:
            rent_cost = room.get('Стоимость_аренды', 0)
            st.markdown(f'***Стоимость:**')
            st.markdown(f'***{int(rent_cost):,} Р/мес***')
        with col3:
            if st.button("Выбрать",
                          key=f"select_room_{room['N_помещения']}",
                          use_container_width=True):
                st.session_state.selected_room = room['N_помещения']
                st.session_state.selected_room_info = room
                st.rerun()
    st.markdown("---")
if 'selected_room' in st.session_state:
    room_info = st.session_state.selected_room_info
    st.markdown("### Оформление договора")
    st.markdown(f'***Выбранное помещение:** №{room_info['N_помещения']}')
    st.markdown(f'***Этаж:** {room_info['Этаж']}')

```

```

st.markdown(f"**Площадь:** {room_info['Площадь']} м²")
st.markdown(f"**Стоимость аренды:** {int(room_info.get('Стоимость_аренды', 0))}, {
Р/мес}")
with st.form("new_contract_form"):
    col1, col2 = st.columns(2)
    with col1:
        start_date = st.date_input(
            "Дата начала договора",
            value=datetime.now().date(),
            min_value=datetime.now().date()
        )
    with col2:
        end_date = st.date_input(
            "Дата окончания договора",
            value=datetime.now().date() + timedelta(days=365),
            min_value=datetime.now().date() + timedelta(days=30))
    if end_date <= start_date:
        st.error("Дата окончания должна быть позже даты начала")
    st.markdown("### Дополнительные условия")
    terms = st.text_area(
        "Дополнительные условия (если есть)",
        height=100,
        placeholder="Укажите дополнительные условия договора..."
    )
    col_submit, col_cancel = st.columns(2)
    with col_submit:
        submit_button = st.form_submit_button("Заключить договор",
                                                type="primary",
                                                use_container_width=True)
    with col_cancel:
        cancel_button = st.form_submit_button("Отмена",
                                                use_container_width=True)
    if cancel_button:
        del st.session_state.selected_room
        del st.session_state.selected_room_info
        st.rerun()
    if submit_button:
        if end_date <= start_date:
            st.error("Исправьте даты договора")
        else:
            employees = db.get_all_employees()
            managers = employees[
                (employees['Должность'].str.contains('Менеджер')) &
                (employees['Статус'] == 'Активен')
            ]
            if managers.empty:
                st.error("Нет доступных менеджеров для оформления договора")
                return
            manager_id = managers.iloc[0]['ID_сотрудника']
            with st.spinner("Оформление договора..."):
                success = db.add_contract(
                    tenant_id=tenant_id,

```

```

        room_id=room_info['N_помещения'],
        employee_id=manager_id,
        start_date=start_date,
        end_date=end_date
    )
    if success:
        st.success("Договор успешно оформлен!")
        st.info("С вами свяжется менеджер для подписания документов")
        del st.session_state.selected_room
        del st.session_state.selected_room_info
        time.sleep(3)
        st.session_state.page = "tenant_contracts"
        st.rerun()
    else:
        st.error("Ошибка при оформлении договора. Попробуйте позже или обратитесь к менеджеру.")
except Exception as e:
    st.error(f"Ошибка загрузки данных: {str(e)}")
import traceback
traceback.print_exc()
def sidebar():
    """Боковая панель навигации"""
    with st.sidebar:
        st.markdown("""
        <div style="
            background: linear-gradient(135deg, #2c3e50 0%, #3498db 100%);
            padding: 25px 15px;
            border-radius: 8px;
            text-align: center;
            margin-bottom: 25px;
            box-shadow: 0 4px 6px rgba(0,0,0,0.1);
        ">
        <h2 style="color: white; margin: 0; font-size: 24px; font-weight: 700;">
            Торговый Центр
        </h2>
        <p style="color: rgba(255,255,255,0.9); margin: 5px 0 0 0; font-size: 14px;">
            Система управления арендой
        </p>
        </div>
        """, unsafe_allow_html=True)
    if not st.session_state.logged_in:
        st.markdown('<p class="menu-category">Общедоступные разделы</p>',
            unsafe_allow_html=True)
        if st.button("Карта ТЦ",
            use_container_width=True,
            key="public_map"):
            st.session_state.page = "map"
        if st.button("Отзывы",
            use_container_width=True,
            key="public_reviews"):
            st.session_state.page = "reviews"
        if st.button("Оставить отзыв",

```

```

        use_container_width=True,
        key="public_add_review"):
    st.session_state.page = "add_review"
st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
if st.button("Вход в систему",
            use_container_width=True,
            key="public_login",
            type="primary"):
    st.session_state.page = "login"

elif st.session_state.logged_in:
    if st.session_state.user_type in ['employee', 'tenant'] and st.session_state.user_data:
        user_data = st.session_state.user_data
        if st.session_state.user_type == 'employee':
            position = user_data.get('Должность', 'Сотрудник')
            user_id = user_data.get('ID_сотрудника', '—')
            status = user_data.get('Статус', 'Активен')
        else:
            position = "Арендатор"
            user_id = user_data.get('ID_арендатора', '—')
            status = "Арендатор"
        st.markdown(f"""
<div style="
    background: linear-gradient(135deg, #f8f9fa 0%, #e9ecef 100%);
    padding: 20px;
    border-radius: 8px;
    margin-bottom: 25px;
    border-left: 4px solid #2c3e50;
    box-shadow: 0 3px 5px rgba(0,0,0,0.05);
">
    <div style="
        display: flex;
        align-items: center;
        margin-bottom: 15px;
    ">
        <div style="
            background: linear-gradient(135deg, #2c3e50 0%, #3498db 100%);
            color: white;
            width: 45px;
            height: 45px;
            border-radius: 50%;
            display: flex;
            align-items: center;
            justify-content: center;
            font-weight: bold;
            font-size: 20px;
            margin-right: 15px;
            box-shadow: 0 3px 6px rgba(0,0,0,0.1);
        ">
            {user_data.get('ФИО', 'П')[0]}
        </div>
    </div>
</div>

```

```

        <p style="margin: 0; font-weight: 700; color: #2c3e50; font-size: 16px;">
            {user_data.get('ФИО', 'Пользователь')}
        </p>
        <p style="margin: 3px 0 0 0; font-size: 13px; color: #7f8c8d;">
            {position}
        </p>
    </div>
</div>
<div style="
    display: flex;
    justify-content: space-between;
    margin-top: 10px;
    font-size: 12px;
    color: #6c757d;
">
    <div>
        <strong>ID:</strong> {user_id}
    </div>
    <div>
        <strong>Статус:</strong> {status}
    </div>
</div>
</div>
""", unsafe_allow_html=True)
if st.session_state.user_type == 'employee' and user_data.get('Должность') ==
'Менеджер по аренде':
    # Основное меню для менеджеров
    st.markdown('<p class="menu-category">Основное меню</p>',
unsafe_allow_html=True)
    if st.button("Главная",
        use_container_width=True,
        key="sidebar_dashboard"):
        st.session_state.page = "dashboard"
    if st.button("Помещения",
        use_container_width=True,
        key="sidebar_rooms"):
        st.session_state.page = "rooms"
    if st.button("Договоры",
        use_container_width=True,
        key="sidebar_contracts"):
        st.session_state.page = "contracts"
    if st.button("Бронирования",
        use_container_width=True,
        key="sidebar_bookings"):
        st.session_state.page = "bookings"

    if st.button("Заявки на ремонт",
        use_container_width=True,
        key="sidebar_repairs"):
        st.session_state.page = "repairs"
    st.markdown('<p class="menu-category">Контрагенты</p>',
unsafe_allow_html=True)

```

```

        if st.button("Арендаторы",
                    use_container_width=True,
                    key="sidebar_tenants"):
            st.session_state.page = "tenants"

        if st.button("Сотрудники",
                    use_container_width=True,
                    key="sidebar_employees"):
            st.session_state.page = "employees"
        st.markdown('<p class="menu-category">Торговля</p>', unsafe_allow_html=True)
        if st.button("Торговые точки",
                    use_container_width=True,
                    key="sidebar_shops"):
            st.session_state.page = "shops"

        if st.button("Оплаты",
                    use_container_width=True,
                    key="sidebar_payments"):
            st.session_state.page = "payments"
    elif st.session_state.user_type == 'tenant':
        st.markdown('<p class="menu-category">Личный кабинет</p>',
                    unsafe_allow_html=True)
        if st.button("Главная",
                    use_container_width=True,
                    key="tenant_dashboard"):
            st.session_state.page = "dashboard"
        st.markdown('<p class="menu-category">Мои объекты</p>',
                    unsafe_allow_html=True)
        if st.button("Мои торговые точки",
                    use_container_width=True,
                    key="tenant_sidebar_shops"):
            st.session_state.page = "tenant_shops"
        if st.button("Мои договоры",
                    use_container_width=True,
                    key="tenant_sidebar_contracts"):
            st.session_state.page = "tenant_contracts"
        if st.button("Мои бронирования",
                    use_container_width=True,
                    key="tenant_sidebar_bookings"):
            st.session_state.page = "tenant_bookings"
        if st.button("Создать бронирование",
                    use_container_width=True,
                    key="tenant_sidebar_new_booking"):
            st.session_state.page = "create_booking"
        st.markdown('<p class="menu-category">Платежи и управление</p>',
                    unsafe_allow_html=True)
        if st.button("Мои платежи",
                    use_container_width=True,
                    key="tenant_sidebar_payments"):
            st.session_state.page = "tenant_payments"
        if st.button("Расторжение договора",
                    use_container_width=True,

```

```

        key="tenant_sidebar_terminate"):
            st.session_state.page = "tenant_terminate"
    else:
        st.markdown('<p class="menu-category">Доступные разделы</p>',
unsafe_allow_html=True)

        if st.button("Главная",
            use_container_width=True,
            key="other_dashboard"):
            st.session_state.page = "dashboard"

        if st.button("Карта ТЦ",
            use_container_width=True,
            key="other_map"):
            st.session_state.page = "map"

        if st.button("Отзывы",
            use_container_width=True,
            key="other_reviews"):
            st.session_state.page = "reviews"

        if st.button("Оставить отзыв",
            use_container_width=True,
            key="other_add_review"):
            st.session_state.page = "add_review"

        st.markdown('<div style="margin: 30px 0 20px 0; height: 1px; background: linear-
gradient(90deg, transparent, #e0e6ed, transparent);"></div>', unsafe_allow_html=True)
        if st.button("Выйти из системы",
            key="sidebar_logout",
            use_container_width=True):
            st.session_state.logged_in = False
            st.session_state.user_data = None
            st.session_state.user_type = None
            st.session_state.page = "login"
            st.session_state.edit_mode = {}
            st.session_state.show_add_form = {}
            st.success("Вы успешно вышли из системы")
            time.sleep(1.5)
            st.rerun()

    else:
        st.markdown("""
<div style="
background: linear-gradient(135deg, #f8f9fa 0%, #e9ecef 100%);
padding: 30px 20px;
border-radius: 8px;
text-align: center;
border: 2px solid #e0e6ed;
margin-top: 20px;
">
<div style="

```

```

width: 60px;
height: 60px;
background: linear-gradient(135deg, #2c3e50 0%, #3498db 100%);
color: white;
border-radius: 50%;
display: flex;
align-items: center;
justify-content: center;
margin: 0 auto 15px auto;
font-size: 24px;
font-weight: bold;
box-shadow: 0 4px 6px rgba(0,0,0,0.1);
">

</div>
<h4 style="color: #2c3e50; margin: 0 0 5px 0;">Требуется авторизация</h4>
<p style="color: #6c757d; font-size: 14px; margin: 0;">
    Войдите в систему для доступа к функциям
</p>
</div>
""", unsafe_allow_html=True)

```

```

def tenant_bookings_page():
    """Просмотр бронирований арендатора"""
    st.markdown('<div class="section-header">Мои бронирования</div>',
unsafe_allow_html=True)
    if st.session_state.user_type != 'tenant':
        st.error("Доступно только для арендаторов")
        return
    try:
        user_data = st.session_state.user_data
        tenant_id = user_data.get('ID_арендатора')
        tenant_name = user_data.get('ФИО', 'Арендатор')
        st.markdown(f"### Здравствуйте, {tenant_name}!")
        st.markdown(f"**ID арендатора:** {tenant_id}")
        col1, col2 = st.columns(2)
        with col1:
            if st.button("← Вернуться в личный кабинет",
                        key="back_to_dashboard_bookings",
                        use_container_width=True):
                st.session_state.page = "dashboard"
                st.rerun()
        with col2:
            if st.button("Создать новое бронирование",
                        key="create_new_booking",
                        use_container_width=True):
                st.session_state.page = "create_booking"
                st.rerun()

        st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
        bookings = db.get_tenant_bookings(tenant_id)
        if bookings.empty:

```



```

st.info("У вас нет активных бронирований")
st.markdown("---")
st.markdown("### Хотите забронировать помещение?")
col_empty1, col_empty2 = st.columns(2)
with col_empty1:
    if st.button("Посмотреть свободные помещения",
                  key="view_free_rooms_bookings",
                  use_container_width=True):
        rooms = db.get_all_rooms()
        free_rooms = rooms[rooms['Статус'] == 'Свободно']
        if not free_rooms.empty:
            st.markdown("### Свободные помещения")
            st.dataframe(free_rooms[['N_помещения', 'Этаж', 'Площадь',
                                    'Стоимость_аренды', 'Статус']],
                          use_container_width=True)
        else:
            st.info("В данный момент нет свободных помещений")
with col_empty2:
    if st.button("Связаться с менеджером",
                  key="contact_manager_bookings",
                  use_container_width=True):
        st.info("Обратитесь к менеджеру по телефону: +7 (495) 123-45-67")
return
st.markdown("### Ваши бронирования")
status_options = ["Все"] + list(bookings['Статус_брони'].unique())
status_filter = st.selectbox("Фильтр по статусу", status_options)
if status_filter != "Все":
    filtered_bookings = bookings[bookings['Статус_брони'] == status_filter]
else:
    filtered_bookings = bookings
if filtered_bookings.empty:
    st.info(f"Нет бронирований со статусом '{status_filter}'")
else:
    for idx, booking in filtered_bookings.iterrows():
        with st.container():
            col1, col2, col3 = st.columns([2, 2, 1])
            with col1:
                st.markdown(f"***Бронирование №{booking['ID_брони']}***")
                st.markdown(f"***Помещение:** №{booking['N_помещения']}")
                st.markdown(f"***Этаж:** {booking['Этаж']}")
                st.markdown(f"***Площадь:** {booking['Площадь']} м²")
            with col2:
                booking_date = booking['Дата_бронирования']
                expiry_date = booking['Дата_истечения']
                if hasattr(booking_date, 'strftime'):
                    booking_date_str = booking_date.strftime('%d.%m.%Y')
                else:
                    booking_date_str = str(booking_date)
                if hasattr(expiry_date, 'strftime'):
                    expiry_date_str = expiry_date.strftime('%d.%m.%Y')
                else:
                    expiry_date_str = str(expiry_date)

```

```

st.markdown(f"***Дата брони:** {booking_date_str}")
st.markdown(f"***Действует до:** {expiry_date_str}")
current_date = datetime.now().date()
if hasattr(expiry_date, 'date'):
    expiry_date_date = expiry_date.date()
    days_left = (expiry_date_date - current_date).days
    if days_left < 0:
        st.error("Истекло")
    elif days_left == 0:
        st.warning("Истекает сегодня")
    else:
        st.info(f"Осталось дней: {days_left}")
with col3:
    rent_cost = booking.get('Стоимость_аренды', 0)
    st.markdown(f"***Стоимость:**")
    st.markdown(f"***{int(rent_cost):,} Р/мес**")
    status = booking.get('Статус_брони', 'Неизвестно')
    if status == 'Активно':
        st.success("Активно")
    elif status == 'Неактивно':
        st.error("Неактивно")
    else:
        st.info(status)
st.markdown("---")
st.markdown("### Статистика бронирований")
total_bookings = len(bookings)
active_bookings = len(bookings[bookings['Статус_брони'] == 'Активно'])
inactive_bookings = len(bookings[bookings['Статус_брони'] == 'Неактивно'])
col_stat1, col_stat2, col_stat3 = st.columns(3)
with col_stat1:
    st.markdown(f"
    <div class='stat-box'>
      <div class='stat-label'>Всего бронирований</div>
      <div class='stat-value'>{total_bookings}</div>
    </div>
    ", unsafe_allow_html=True)
with col_stat2:
    st.markdown(f"
    <div class='stat-box'>
      <div class='stat-label'>Активных</div>
      <div class='stat-value'>{active_bookings}</div>
    </div>
    ", unsafe_allow_html=True)
with col_stat3:
    st.markdown(f"
    <div class='stat-box'>
      <div class='stat-label'>Неактивных</div>
      <div class='stat-value'>{inactive_bookings}</div>
    </div>
    ", unsafe_allow_html=True)
st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
st.markdown("### Управление бронированиями")

```

```

col_action1, col_action2 = st.columns(2)
with col_action1:
    if st.button("Обновить список бронирований",
                  key="refresh_bookings",
                  use_container_width=True):
        st.rerun()
with col_action2:
    if st.button("Создать новое бронирование",
                  key="new_booking_action",
                  use_container_width=True):
        st.session_state.page = "create_booking"
        st.rerun()
except Exception as e:
    st.error(f"Ошибка загрузки данных: {str(e)}")
def tenant_terminate_page():
    """Расторжение договора арендатором"""
    if 'user_type' not in st.session_state or st.session_state.user_type != 'tenant':
        st.error("Доступно только для арендаторов")
        return
    try:
        user_data = st.session_state.get('user_data', {})
        tenant_id = user_data.get('ID_арендатора')
        tenant_name = user_data.get('ФИО', 'Арендатор')
        if not tenant_id:
            st.error("Ошибка: не удалось определить ID арендатора")
            return

        st.title("Расторжение договора")
        if st.button("Назад"):
            st.session_state.page = "tenant_contracts"
            st.rerun()
        st.markdown("---")
        active_contracts_df = db.get_tenant_active_contracts(tenant_id)
        if active_contracts_df.empty:
            st.warning("У вас нет активных договоров для расторжения")
            if st.button("Посмотреть все договоры"):
                st.session_state.page = "tenant_contracts"
                st.rerun()
            return
        st.subheader("Выберите договор для расторжения")
        contract_options = []
        for _, contract in active_contracts_df.iterrows():
            contract_id = contract['N_договора']
            shop_name = contract['Название_магазина']
            room = contract['N_помещения']
            option_text = f"Договор №{contract_id} - {shop_name} (помещение №{room})"
            contract_options.append(option_text)
        selected_option = st.selectbox("Договор", contract_options)
        if not selected_option:
            return
        selected_contract_num = int(selected_option.split("№")[1].split(" ")[0])

```

```

contract_info = active_contracts_df[active_contracts_df['N_договора'] ==
selected_contract_num].iloc[0]
st.markdown("---")
st.markdown(f"### Договор №{selected_contract_num}")
st.markdown(f "***Магазин:** {contract_info.get('Название_магазина', '—')}")
st.markdown(f "***Помещение:** №{contract_info.get('N_помещения', '—')},
{contract_info.get('Этаж', '—')} этаж")
st.markdown(f "***Площадь:** {contract_info.get('Площадь', 0)} м²")
st.markdown(f "***Стоимость:** {int(contract_info.get('Тариф', 0))},} Р/мес")
st.markdown(f "***Период:** {contract_info.get('Дата_оформления_формат', '—')} –
{contract_info.get('Дата_окончания_формат', '—')}")
st.markdown("---")
st.warning(f"***Внимание:** Договор будет немедленно расторгнут. Это действие
необратимо.")
col1, col2 = st.columns(2)
with col1:
    if st.button("Да, расторгнуть договор", type="primary", use_container_width=True):
        try:
            success = db.terminate_contract(contract_id=selected_contract_num)
            if success:
                st.success(f"Договор №{selected_contract_num} успешно расторгнут!")
                st.info("Перенаправляем на страницу договоров...")
                import time
                time.sleep(2)
                st.session_state.page = "tenant_contracts"
                st.rerun()
            else:
                st.error("Не удалось расторгнуть договор")
        except Exception as e:
            st.error(f"Ошибка: {str(e)}")
    with col2:
        if st.button("Отмена", use_container_width=True):
            st.session_state.page = "tenant_contracts"
            st.rerun()
except Exception as e:
    st.error(f"Ошибка при загрузке данных: {str(e)}")
    if st.button("Вернуться к договорам"):
        st.session_state.page = "tenant_contracts"
        st.rerun()
def create_booking_page():
    """Создание нового бронирования (максимум на 1 день)"""
    st.markdown('<div class="section-header">Создание нового бронирования</div>',
unsafe_allow_html=True)
    if st.session_state.user_type != 'tenant':
        st.error("Доступно только для арендаторов")
        return
    try:
        user_data = st.session_state.user_data
        tenant_id = user_data.get('ID_арендатора')
        tenant_name = user_data.get('ФИО', 'Арендатор')
        if st.button("← Назад к бронированиям",
            key="back_to_bookings",

```

```

        use_container_width=True):
    st.session_state.page = "tenant_bookings"
    st.rerun()
st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
st.info("""
**Правила бронирования:**
- Бронирование возможно только на 1 день
- Минимальная дата начала - сегодня
- Бронирование автоматически снимается на следующий день
- После создания бронирования с вами свяжется менеджер для уточнения деталей
договора
""")
rooms = db.get_all_rooms()
free_rooms = rooms[rooms['Статус'] == 'Свободно']
if free_rooms.empty():
    st.error("В данный момент нет свободных помещений для бронирования")
    st.info("Пожалуйста, свяжитесь с менеджером для уточнения доступности
помещений")
    return
with st.form("create_booking_form"):
    st.markdown("### Выберите помещение для бронирования")
    room_options = []
    room_info_dict = {}
    for _, room in free_rooms.iterrows():
        room_id = room['N_помещения']
        room_info = f'№{room_id} (Этаж {room["Этаж"]}, {room["Площадь"]}м²,
{room["Стоимость_аренды"]}₽/мес)"
        room_options.append(room_info)
        room_info_dict[room_info] = {
            'id': room_id,
            'floor': room['Этаж'],
            'area': room['Площадь'],
            'cost': room['Стоимость_аренды']}
    selected_room = st.selectbox("Помещение", room_options)

    today = datetime.now().date()
    start_date = st.date_input(
        "Дата бронирования",
        value=today,
        min_value=today,
        max_value=today + timedelta(days=30),
        help="Бронирование будет действовать только в выбранный день"
    )
    end_date = start_date + timedelta(days=1)
    st.markdown("""
---
**Важная информация:**
1. После создания бронирования с вами свяжется менеджер в течение 24 часов
2. Вы сможете обсудить с менеджером все детали будущего договора
3. Бронирование резервирует помещение на 1 день для подготовки документов
4. Для заключения долгосрочного договора потребуется личная встреча с
менеджером

```

```

        """
col_submit, col_cancel = st.columns(2)
with col_submit:
    submitted = st.form_submit_button("Забронировать помещение",
                                       use_container_width=True,
                                       type="primary")
with col_cancel:
    if st.form_submit_button("Отмена", use_container_width=True):
        st.session_state.page = "tenant_bookings"
        st.rerun()
if submitted:
    if not selected_room:
        st.error("Выберите помещение для бронирования")
    else:
        room_data = room_info_dict[selected_room]
        room_id = room_data['id']
        with st.spinner("Создание бронирования..."):
            success = db.add_booking(
                room_id=room_id,
                tenant_id=tenant_id,
                booking_date=start_date,
                expiry_date=end_date,
                status="Активно"
            )
            if success:
                st.success(f"Помещение успешно забронировано!")
                st.info(f"""
                **С вами свяжется менеджер в течение 24 часов для уточнения деталей
                договора.**
                """)
                time.sleep(3)
                st.session_state.page = "tenant_bookings"
                st.rerun()
            else:
                st.error("Ошибка при создании бронирования. Пожалуйста, попробуйте
                позже или свяжитесь с менеджером.")
        except Exception as e:
            st.error(f"Ошибка создания бронирования: {str(e)}")
def tenant_shop_reviews_page():
    """Отзывы на конкретную торговую точку"""
    if 'selected_shop_id' not in st.session_state or 'selected_shop_name' not in st.session_state:
        st.error("Торговая точка не выбрана")
        if st.button("Вернуться к моим точкам"):
            st.session_state.page = "tenant_shops"
            st.rerun()
    return
shop_id = st.session_state.selected_shop_id
shop_name = st.session_state.selected_shop_name
st.markdown(f'<div class="section-header">Отзывы: {shop_name}</div>',
unsafe_allow_html=True)
if st.button("← Назад к торговым точкам",
            key="back_to_shops_from_reviews",

```

```

        use_container_width=True):
    st.session_state.page = "tenant_shops"
    st.rerun()
try:
    all_reviews = db.get_all_reviews()
    if all_reviews.empty():
        st.info("Пока нет отзывов на эту торговую точку")
        return
    shop_reviews = all_reviews[all_reviews['N_торговой_точки'] == shop_id]
    if shop_reviews.empty():
        st.info("Пока нет отзывов на эту торговую точку")
        return
    if 'оценка' in shop_reviews.columns:
        avg_rating = shop_reviews['оценка'].mean()
        total_reviews = len(shop_reviews)
        col1, col2, col3 = st.columns(3)
        with col1:
            st.markdown(f"""
                <div class="stat-box">
                    <div class="stat-label">Общий рейтинг</div>
                    <div class="stat-value">{avg_rating:.1f}/5</div>
                </div>
                """, unsafe_allow_html=True)
        with col2:
            st.markdown(f"""
                <div class="stat-box">
                    <div class="stat-label">Всего отзывов</div>
                    <div class="stat-value">{total_reviews}</div>
                </div>
                """, unsafe_allow_html=True)
        with col3:
            five_star = len(shop_reviews[shop_reviews['оценка'] == 5])
            st.markdown(f"""
                <div class="stat-box">
                    <div class="stat-label">5 звезд</div>
                    <div class="stat-value">{five_star}</div>
                </div>
                """, unsafe_allow_html=True)
        st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
        col_filter1, col_filter2 = st.columns(2)
        with col_filter1:
            rating_filter = st.selectbox("Фильтр по оценке",
                                         ["Все", "5 звезд", "4 звезды", "3 звезды", "2 звезды", "1 звезда"])
        with col_filter2:
            sort_option = st.selectbox("Сортировка",
                                       ["Сначала новые", "Сначала старые", "Высокие оценки", "Низкие
оценки"])
        filtered_reviews = shop_reviews.copy()
        if rating_filter != "Все":
            rating_value = int(rating_filter[0])
            filtered_reviews = filtered_reviews[filtered_reviews['оценка'] == rating_value]
        if sort_option == "Сначала новые":

```

```

        filtered_reviews = filtered_reviews.sort_values('дата_отзыва', ascending=False)
    elif sort_option == "Сначала старые":
        filtered_reviews = filtered_reviews.sort_values('дата_отзыва', ascending=True)
    elif sort_option == "Высокие оценки":
        filtered_reviews = filtered_reviews.sort_values('оценка', ascending=False)
    elif sort_option == "Низкие оценки":
        filtered_reviews = filtered_reviews.sort_values('оценка', ascending=True)
    if not filtered_reviews.empty:
        st.markdown(f"### Отзывы ({len(filtered_reviews)})")
        for idx, review in filtered_reviews.iterrows():
            author = str(review.get('автор', 'Аноним'))
            rating = int(review.get('оценка', 0))
            description = str(review.get('описание', ''))
            date_str = review.get('дата_отзыва', '')
            if date_str:
                try:
                    if isinstance(date_str, str):
                        date_obj = datetime.strptime(date_str, '%Y-%m-%d')
                        date_str = date_obj.strftime('%d.%m.%Y')
                    elif hasattr(date_str, 'strftime'):
                        date_str = date_str.strftime('%d.%m.%Y')
                except:
                    date_str = str(date_str)
            rating_color = "#28a745" if rating >= 4 else "#ffc107" if rating == 3 else "#dc3545"
            stars = "★" * rating + "☆" * (5 - rating)
            with st.container():
                col1, col2 = st.columns([1, 4])
                with col1:
                    avatar = author[0].upper() if author else 'A'
                    st.markdown(f"""
                    <div style="
                    width: 50px;
                    height: 50px;
                    background: linear-gradient(135deg, #2c3e50, #3498db);
                    color: white;
                    border-radius: 50%;
                    display: flex;
                    align-items: center;
                    justify-content: center;
                    font-weight: bold;
                    font-size: 20px;
                    margin: 0 auto;
                    ">
                    {avatar}
                    </div>
                    """, unsafe_allow_html=True)
                with col2:
                    st.markdown(f"***{author}***")
                    st.markdown(f"*{date_str}* • <span style='color: {rating_color}; font-weight:
                    bold;'>{rating}/5 ★</span>", unsafe_allow_html=True)
                    st.markdown(f"<div style='color: #FFD700; font-size: 18px; margin: 5px
                    0;'>{stars}</div>", unsafe_allow_html=True)

```



```

        if description and description != 'nan' and description != 'None':
            st.markdown(f"<div style='padding: 15px; background: #f8f9fa; border-radius: 8px; margin: 5px 0;'>{description}</div>", unsafe_allow_html=True)
            st.markdown("---")
        else:
            st.info("Нет отзывов, соответствующих выбранному фильтру")
    except Exception as e:
        st.error(f"Ошибка загрузки отзывов: {str(e)}")

def main():
    """Главная функция"""
    sidebar()
    page_handlers = {
        "login": login_page,
        "dashboard": dashboard_page,
        "rooms": rooms_page,
        "contracts": contracts_page,
        "bookings": bookings_page,
        "repairs": repairs_page,
        "employees": employees_page,
        "shops": shops_page,
        "tenants": tenants_page,
        "payments": payments_page,
        "map": map_page,
        "reviews": reviews_page,
        "add_review": add_review_page,
        "tenant_shops": tenant_shops_page,
        "tenant_contracts": tenant_contracts_page,
        "tenant_payments": tenant_payments_page,
        "tenant_terminate": tenant_terminate_page,
        "tenant_new_contract": tenant_new_contract_page,
        "tenant_shop_reviews": tenant_shop_reviews_page,
        "tenant_shop_payments": tenant_shop_payments_page,
        "tenant_bookings": tenant_bookings_page,
        "create_booking": create_booking_page
    }
    handler = page_handlers.get(st.session_state.page, login_page)
    handler()
    st.markdown("""
<div style="
    text-align: center;
    padding: 20px;
    margin-top: 40px;
    background-color: #f8f9fa;
    border-radius: 5px;
    border-top: 2px solid #2c3e50;
">
    <h4 style="color: #2c3e50; margin-bottom: 10px;">
        Проект по дисциплине "Управление данными"
    </h4>
    <p style="margin: 5px 0; color: #555;">
        Студента 3 курса МГТУ "СТАНКИН"
    
```

```

</p>
<p style="margin: 5px 0; font-weight: bold; color: #2c3e50;">
    Баранова Василия
</p>
<p style="margin: 5px 0; color: #777; font-size: 14px;">
    © 2025 год
</p>
</div>
"""', unsafe_allow_html=True)
st.markdown("""
<script>
function updateStars(rating) {
    const stars = document.querySelectorAll('.star');
    stars.forEach((star, index) => {
        if (index < rating) {
            star.textContent = '★';
            star.style.color = '#FFD700';
        } else {
            star.textContent = '☆';
            star.style.color = '#E0E6ED';}}});
window.onload = function() {
    const slider = document.querySelector('input[type="range"]');
    if (slider) {
        const initialRating = parseInt(slider.value);
        updateStars(initialRating);
        slider.addEventListener('input', function() {
            updateStars(parseInt(this.value));});});
</script>
"""', unsafe_allow_html=True)
if __name__ == "__main__":
    main()

```

Файл database.py

```

import pyodbc
import pandas as pd
from datetime import datetime, timedelta

class DatabaseManager:
    def __init__(self):
        self.server = '10.211.55.2,1433'
        self.database = 'Курсовая_работа'
        self.username = 'sa'
        self.password = 'VasiliyBaranov2005@'
        self.connection = None

    def get_connection(self):
        try:
            if self.connection is None:
                conn_str = (
                    f"DRIVER={{ODBC Driver 17 for SQL Server}};"
                    f"SERVER={self.server};"
                    f"DATABASE={self.database};"

```

```

        f"UID={self.username};"
        f"PWD={self.password};"
        "TrustServerCertificate=yes"
    )
    self.connection = pyodbc.connect(conn_str)
    print("Подключение к базе успешно")
    return self.connection
except Exception as e:
    print(f"Ошибка подключения: {e}")
    return None

def execute_query(self, query, params=None, fetch=False):
    conn = None
    try:
        conn = self.get_connection()
        if conn is None:
            print("Нет подключения к базе данных")
            return pd.DataFrame() if fetch else False

        cursor = conn.cursor()

        if params:
            cursor.execute(query, params)
        else:
            cursor.execute(query)

        if fetch:
            if cursor.description:
                columns = [column[0] for column in cursor.description]
                results = cursor.fetchall()
                if results:
                    df = pd.DataFrame.from_records(results, columns=columns)
                    print(f"Запрос выполнен успешно. Получено {len(df)} строк")
                    return df
                print("Запрос выполнен, но данных не найдено")
                return pd.DataFrame()
            else:
                conn.commit()
                print("Запрос выполнен успешно (без получения данных)")
                return True
        except Exception as e:
            print(f"Ошибка запроса: {e}")
            print(f"Запрос: {query}")
            print(f"Параметры: {params}")
            if conn:
                conn.rollback()
            return pd.DataFrame() if fetch else False

def check_user_login(self, phone_number):
    try:
        phone_clean = ".join(filter(str.isdigit, str(phone_number)))

```

```

if len(phone_clean) == 11:
    if phone_clean.startswith('7') or phone_clean.startswith('8'):
        phone_clean = phone_clean[1:]

print(f'Ищем пользователя по номеру: {phone_clean}')

query = """
SELECT
    ID_сотрудника,
    ФИО,
    Должность,
    Номер_телефона,
    Статус
FROM Сотрудник
WHERE Номер_телефона LIKE ? OR Номер_телефона LIKE ?
"""

params = [f'%{phone_clean}%', f'+7{phone_clean}%']
result = self.execute_query(query, params, fetch=True)

if not result.empty():
    employee_data = result.iloc[0]

    if employee_data['Статус'] == 'Активен':
        print(f'Найден активный сотрудник: {employee_data['ФИО']}')

        employee_info = {
            'ID_сотрудника': int(employee_data['ID_сотрудника']),
            'ФИО': str(employee_data['ФИО']),
            'Должность': str(employee_data['Должность']),
            'Номер_телефона': str(employee_data['Номер_телефона']),
            'Статус': str(employee_data['Статус']),
            'user_type': 'employee'
        }
        return True, employee_info, None
    else:
        print(f'Найден неактивный сотрудник: {employee_data['ФИО']}')
        return False, None, f"Сотрудник {employee_data['ФИО']} не активен"

query_tenant = """
SELECT
    ID_арендатора,
    ФИО,
    Номер_телефона
FROM Арендатор
WHERE Номер_телефона LIKE ? OR Номер_телефона LIKE ?
"""

tenant_result = self.execute_query(query_tenant, params, fetch=True)
if not tenant_result.empty():
    tenant_data = tenant_result.iloc[0]
    tenant_info = {

```

```

        'ID_арендатора': int(tenant_data['ID_арендатора']),
        'ФИО': str(tenant_data['ФИО']),
        'Номер_телефона': str(tenant_data['Номер_телефона']),
        'user_type': 'tenant'}
    print(f"Найден арендатор: {tenant_data['ФИО']}")
    return True, tenant_info, None
    print(f"Пользователь с номером {phone_number} не найден")
    return False, None, "Пользователь с таким номером телефона не найден"
except Exception as e:
    print(f"Ошибка при проверке: {str(e)}")
    return False, None, f"Ошибка при проверке пользователя: {str(e)}"
def get_all_rooms(self):
    print("Получение всех помещений...")
    query = "SELECT * FROM Помещение ORDER BY Этаж, N_помещения"
    return self.execute_query(query, fetch=True)
def add_room(self, room_number, floor, area, rent_cost, status):
    try:
        print(f"Добавление помещения №{room_number}")
        query = """
            INSERT INTO Помещение (N_помещения, Этаж, Площадь, Стоимость_аренды,
Статус)
            VALUES (?, ?, ?, ?, ?)
            """
        params = [int(room_number), int(floor), float(area), float(rent_cost), status]
        result = self.execute_query(query, params, fetch=False)
        if result:
            print(f"Помещение №{room_number} успешно добавлено")
        else:
            print(f"Ошибка при добавлении помещения №{room_number}")
        return result
    except Exception as e:
        print(f"Ошибка добавления помещения: {e}")
        return False
def get_all_contracts(self):
    print("Получение всех договоров...")
    query = """
    SELECT
        Договор.N_договора,
        Договор.ID_арендатора,
        Договор.N_помещения,
        Договор.ID_сотрудника,
        Договор.Дата_оформления,
        Договор.Дата_окончания,
        Договор.Тариф,
        Арендатор.ФИО as Арендатор_ФИО,
        Помещение.Этаж,
        Помещение.Площадь,
        Помещение.Стоимость_аренды
    FROM Договор
    LEFT JOIN Арендатор ON Договор.ID_арендатора = Арендатор.ID_арендатора
    LEFT JOIN Помещение ON Договор.N_помещения = Помещение.N_помещения
    ORDER BY Договор.Дата_оформления DESC
    """

```

```

"""
return self.execute_query(query, fetch=True)
def add_shop(self, room_id, category, rent_cost, shop_name):
    """Добавить торговую точку"""
    try:
        print(f"Добавление торговой точки в помещение {room_id}")
        max_query = "SELECT MAX(N_торговой_точки) as max_id FROM Торговая_точка"
        max_result = self.execute_query(max_query, fetch=True)
        new_id = 1
        if not max_result.empty and max_result.iloc[0]['max_id'] is not None:
            new_id = int(max_result.iloc[0]['max_id']) + 1
        room_check = "SELECT N_помещения FROM Помещение WHERE N_помещения =
?"
        room_result = self.execute_query(room_check, [int(room_id)], fetch=True)
        if room_result.empty:
            print(f"Помещение с ID {room_id} не найдено")
            return False
        query = """
        INSERT INTO Торговая_точка (
            N_торговой_точки,
            N_помещения,
            категория_торговой_точки,
            стоимость_аренды,
            Название_магазина,
            Статус_договора
        ) VALUES (?, ?, ?, ?, ?, N'Активен')
        """
        params = [
            int(new_id),
            int(room_id),
            str(category),
            float(rent_cost),
            str(shop_name)
        ]
        result = self.execute_query(query, params, fetch=False)
        if result:
            print(f"Торговая точка №{new_id} успешно добавлена")
        else:
            print(f"Ошибка при добавлении торговой точки")
        return result
    except Exception as e:
        print(f"Ошибка добавления торговой точки: {e}")
        import traceback
        traceback.print_exc()
        return False
def add_contract(self, tenant_id, room_id, employee_id, start_date, end_date,
category="Другое", shop_name=None):
    """Создание договора аренды с автоматическим созданием торговой точки"""
    try:
        print(f"Добавление договора для арендатора {tenant_id}, помещение {room_id}")

```

```

room_query = "SELECT Стоимость_аренды, Статус FROM Помещение WHERE
N_помещения = ?"
room_result = self.execute_query(room_query, [int(room_id)], fetch=True)
if room_result.empty:
    print(f"Помещение с ID {room_id} не найдено")
    return False
tariff = room_result.iloc[0]['Стоимость_аренды']
current_status = room_result.iloc[0]['Статус']
if current_status != 'Свободно':
    print(f"Помещение №{room_id} не свободно. Текущий статус: {current_status}")
    return False
if tariff <= 0:
    print(f"Стоимость аренды помещения {room_id} должна быть больше 0")
    return False
max_query = "SELECT MAX(N_договора) as max_id FROM Договор"
max_result = self.execute_query(max_query, fetch=True)
new_contract_id = 1
if not max_result.empty and max_result.iloc[0]['max_id'] is not None:
    new_contract_id = int(max_result.iloc[0]['max_id']) + 1
start_date_str = start_date.strftime("%Y-%m-%d")
end_date_str = end_date.strftime("%Y-%m-%d")
tariff_float = float(tariff)
tenant_check = "SELECT ID_арендатора, ФИО FROM Арендатор WHERE
ID_арендатора = ?"
tenant_result = self.execute_query(tenant_check, [int(tenant_id)], fetch=True)
if tenant_result.empty:
    print(f"Арендатор с ID {tenant_id} не найден")
    return False
tenant_name = tenant_result.iloc[0]['ФИО']
print(f"Арендатор найден: {tenant_name}")
employee_check = "SELECT ID_сотрудника, ФИО FROM Сотрудник WHERE
ID_сотрудника = ?"
employee_result = self.execute_query(employee_check, [int(employee_id)], fetch=True)
if employee_result.empty:
    print(f"Сотрудник с ID {employee_id} не найден")
    return False
employee_name = employee_result.iloc[0]['ФИО']
print(f"Сотрудник найден: {employee_name}")
contract_query = """
INSERT INTO Договор (N_договора, ID_арендатора, N_помещения,
ID_сотрудника, Дата_оформления, Дата_окончания, Тариф)
VALUES (?, ?, ?, ?, ?, ?, ?)
"""
contract_params = [
    int(new_contract_id),
    int(tenant_id),
    int(room_id),
    int(employee_id),
    start_date_str,
    end_date_str,
    tariff_float
]

```

```

contract_result = self.execute_query(contract_query, contract_params, fetch=False)
if not contract_result:
    print(f"Ошибка при добавлении договора")
    return False
print(f"Договор №{new_contract_id} успешно добавлен")
update_room_query = "UPDATE Помещение SET Статус = N'Занятое' WHERE
N_помещения = ?"
room_update_result = self.execute_query(update_room_query, [int(room_id)],
fetch=False)
if room_update_result:
    print(f"Статус помещения №{room_id} обновлен на 'Занятое'")
else:
    print(f"Не удалось обновить статус помещения №{room_id}")
if shop_name is None:
    shop_name = f"Магазин {tenant_name} (договор №{new_contract_id})"
shop_check_query = "SELECT N_торговой_точки FROM Торговая_точка WHERE
N_помещения = ?"
shop_check_result = self.execute_query(shop_check_query, [int(room_id)], fetch=True)
if not shop_check_result.empty:
    print(f"Для помещения №{room_id} уже существует торговая точка. Пропускаем
создание.")
    return new_contract_id
max_shop_query = "SELECT MAX(N_торговой_точки) as max_id FROM
Торговая_точка"
max_shop_result = self.execute_query(max_shop_query, fetch=True)
new_shop_id = new_contract_id
shop_id_check_query = "SELECT N_торговой_точки FROM Торговая_точка
WHERE N_торговой_точки = ?"
shop_id_check_result = self.execute_query(shop_id_check_query, [int(new_shop_id)],
fetch=True)
if not shop_id_check_result.empty:
    if not max_shop_result.empty and max_shop_result.iloc[0]['max_id'] is not None:
        new_shop_id = int(max_shop_result.iloc[0]['max_id']) + 1
    else:
        new_shop_id = 1
shop_query = """
INSERT INTO Торговая_точка (N_торговой_точки, N_помещения,
категория_торговой_точки, стоимость_аренды, Название_магазина, Статус_договора)
VALUES (?, ?, ?, ?, ?, ?)
"""
shop_params = [
    int(new_shop_id),
    int(room_id),
    str(category),
    float(tariff_float),
    str(shop_name),
    'Активен'
]
shop_result = self.execute_query(shop_query, shop_params, fetch=False)
if shop_result:
    print(f"Торговая точка '{shop_name}' (ID: {new_shop_id}) успешно создана")
    print(f" • Связана с договором №{new_contract_id}")

```



```

        print(f" • Категория: {category}")
        print(f" • Стоимость аренды: {tariff_float} Р/мес")
    else:
        print(f"Ошибка при создании торговой точки для договора №{new_contract_id}")
    return new_contract_id
except Exception as e:
    print(f"Ошибка добавления договора: {e}")
    traceback.print_exc()
    return False
def get_all_bookings(self):
    """Получить все бронирования"""
    try:
        print("Получение всех бронирований...")
        query = """
        SELECT
            Бронирование.ID_брони,
            Бронирование.N_помещения,
            Бронирование.ID_арендатора,
            Бронирование.Дата_бронирования,
            Бронирование.Дата_истечения,
            Бронирование.Статус_брони,
            Помещение.Этаж,
            Помещение.Площадь,
            Помещение.Стоимость_аренды,
            Арендатор.ФИО as Арендатор_ФИО
        FROM Бронирование
        LEFT JOIN Арендатор ON Бронирование.ID_арендатора =
Арендатор.ID_арендатора
        LEFT JOIN Помещение ON Бронирование.N_помещения =
Помещение.N_помещения
        ORDER BY Бронирование.ID_брони DESC
        """
        result = self.execute_query(query, fetch=True)
        return result
    except Exception as e:
        print(f"Ошибка получения бронирований: {e}")
        return pd.DataFrame()
def add_booking(self, room_id, tenant_id, booking_date, expiry_date, status):
    """Добавить бронирование"""
    try:
        print(f"Добавление бронирования для помещения {room_id}")
        max_query = "SELECT MAX(ID_брони) as max_id FROM Бронирование"
        max_result = self.execute_query(max_query, fetch=True)
        new_id = 1
        if not max_result.empty and max_result.iloc[0]['max_id'] is not None:
            new_id = int(max_result.iloc[0]['max_id']) + 1
        booking_date_str = booking_date.strftime("%Y-%m-%d")
        expiry_date_str = expiry_date.strftime("%Y-%m-%d")
        query = """
        INSERT INTO Бронирование (ID_брони, N_помещения, ID_арендатора,
Дата_бронирования, Дата_истечения, Статус_брони)
        VALUES (?, ?, ?, ?, ?, ?)
        """

```

```

        """
        params = [int(new_id), int(room_id), int(tenant_id), booking_date_str, expiry_date_str,
status]

        result = self.execute_query(query, params, fetch=False)

        if result:
            print(f"Бронирование №{new_id} успешно добавлено")

            update_query = "UPDATE Помещение SET Статус = N'Забронировано' WHERE
N_помещения = ?"
            update_result = self.execute_query(update_query, [int(room_id)], fetch=False)
            if update_result:
                print(f"Статус помещения {room_id} обновлен на 'Забронировано'")
            else:
                print(f"Не удалось обновить статус помещения {room_id}")
        else:
            print(f"Ошибка при добавлении бронирования")

        return result

    except Exception as e:
        print(f"Ошибка добавления бронирования: {e}")
        import traceback
        traceback.print_exc()
        return False

def update_booking_status(self, booking_id, new_status):
    """Обновляет статус бронирования"""
    try:
        booking_id_int = int(booking_id) if booking_id is not None else None
        new_status_str = str(new_status).strip()

        query = "UPDATE Бронирование SET Статус_брони = ? WHERE ID_брони = ?"
        params = (new_status_str, booking_id_int)

        print(f"DEBUG: Executing query: {query} with params: {params}")
        result = self.execute_query(query, params, fetch=False)
        return result

    except Exception as e:
        print(f"Ошибка при обновлении статуса бронирования: {e}")
        import traceback
        traceback.print_exc()
        return False

def get_all_repairs(self):
    try:
        print("Получение всех заявок на ремонт...")

        query = """
        SELECT

```

```

        N_заявки,
        N_помещения,
        описание_проблемы,
        дата_подачи,
        Статус
    FROM Заявка_на_ремонт
    ORDER BY N_заявки DESC
    """

    result = self.execute_query(query, fetch=True)

    if not result.empty:
        print(f"Получено {len(result)} заявок на ремонт")
    else:
        print("В таблице Заявка_на_ремонт нет данных")

    return result

except Exception as e:
    print(f"Ошибка получения заявок на ремонт: {e}")
    import traceback
    traceback.print_exc()
    return pd.DataFrame()

def add_repair(self, room_id, problem_description, status):
    try:
        print(f"Добавление заявки на ремонт для помещения {room_id}")

        max_query = "SELECT MAX(N_заявки) as max_id FROM Заявка_на_ремонт"
        max_result = self.execute_query(max_query, fetch=True)

        new_id = 1
        if not max_result.empty and max_result.iloc[0]['max_id'] is not None:
            new_id = int(max_result.iloc[0]['max_id']) + 1

        request_date = datetime.now().strftime('%Y-%m-%d')

        query = """
            INSERT INTO Заявка_на_ремонт (N_заявки, N_помещения, описание_проблемы,
            дата_подачи, Статус)
            VALUES (?, ?, ?, ?, ?)
            """

        params = [
            int(new_id),
            int(room_id),
            str(problem_description),
            request_date,
            str(status)
        ]

        result = self.execute_query(query, params, fetch=False)

```

```

    if result:
        print(f"Заявка на ремонт №{new_id} успешно добавлена")
    else:
        print(f"Ошибка при добавлении заявки на ремонт")

    return result

except Exception as e:
    print(f"Ошибка добавления заявки на ремонт: {e}")
    import traceback
    traceback.print_exc()
    return False

def get_all_employees(self):
    try:
        print("Получение всех сотрудников...")
        query = "SELECT * FROM Сотрудник ORDER BY ФИО"
        result = self.execute_query(query, fetch=True)

        if not result.empty:
            print(f"Получено {len(result)} сотрудников")
        else:
            print("Таблица сотрудников пуста")

        return result
    except Exception as e:
        print(f"Ошибка получения сотрудников: {e}")
        return pd.DataFrame()

def add_employee(self, full_name, position, phone, status):
    try:
        print(f"Добавление сотрудника: {full_name}")

        max_query = "SELECT MAX(ID_сотрудника) as max_id FROM Сотрудник"
        max_result = self.execute_query(max_query, fetch=True)

        new_id = 1
        if not max_result.empty and max_result.iloc[0]['max_id'] is not None:
            new_id = int(max_result.iloc[0]['max_id']) + 1

        query = """
        INSERT INTO Сотрудник (ID_сотрудника, ФИО, Должность, Номер_телефона,
Статус)
        VALUES (?, ?, ?, ?, ?)
        """
        params = [int(new_id), full_name, position, phone, status]

        result = self.execute_query(query, params, fetch=False)
        if result:
            print(f"Сотрудник {full_name} (ID: {new_id}) успешно добавлен")
        else:
            print(f"Ошибка при добавлении сотрудника {full_name}")
    
```

```

        return result
    except Exception as e:
        print(f"Ошибка добавления сотрудника: {e}")
        return False

def update_employee_status(self, employee_id, new_status):
    try:
        print(f"Обновление статуса сотрудника ID={employee_id} на '{new_status}'")

        check_query = "SELECT ФИО FROM Сотрудник WHERE ID_сотрудника = ?"
        check_result = self.execute_query(check_query, [int(employee_id)], fetch=True)

        if check_result.empty:
            print(f"Сотрудник с ID={employee_id} не найден")
            return False

        employee_name = check_result.iloc[0]['ФИО']

        query = """
            UPDATE Сотрудник
            SET Статус = ?
            WHERE ID_сотрудника = ?
            """
        params = [new_status, int(employee_id)]

        result = self.execute_query(query, params, fetch=False)

        if result:
            print(f"Статус сотрудника {employee_name} (ID: {employee_id}) успешно изменен на '{new_status}'")
        else:
            print(f"Ошибка при изменении статуса сотрудника")
            return result
    except Exception as e:
        print(f"Ошибка изменения статуса сотрудника: {e}")
        return False

def get_all_tenants(self):
    print("Получение всех арендаторов...")
    query = "SELECT * FROM Арендатор ORDER BY ФИО"
    return self.execute_query(query, fetch=True)

def add_tenant(self, full_name, phone):
    try:
        print(f"Добавление арендатора: {full_name}")

        max_query = "SELECT MAX(ID_арендатора) as max_id FROM Арендатор"
        max_result = self.execute_query(max_query, fetch=True)

        new_id = 1
        if not max_result.empty and max_result.iloc[0]['max_id'] is not None:
            new_id = int(max_result.iloc[0]['max_id']) + 1

```

```

query = """
INSERT INTO Арендатор (ID_арендатора, ФИО, Номер_телефона)
VALUES (?, ?, ?)
"""

params = [int(new_id), full_name, phone]

result = self.execute_query(query, params, fetch=False)
if result:
    print(f"Арендатор {full_name} (ID: {new_id}) успешно добавлен")
else:
    print(f"Ошибка при добавлении арендатора {full_name}")
return result
except Exception as e:
    print(f"Ошибка добавления арендатора: {e}")
    return False

def get_all_payments(self):
    try:
        print("Получение всех оплат...")

        query = """
SELECT
    TRY_CAST(Оплата_аренды_и_ком_услуг.N_операции AS INT) as
N_операции,
    TRY_CAST(Оплата_аренды_и_ком_услуг.N_договора AS INT) as N_договора,
    TRY_CAST(Оплата_аренды_и_ком_услуг.дата_оплаты AS DATE) as
дата_оплаты,
    TRY_CAST(Оплата_аренды_и_ком_услуг.сумма AS DECIMAL(10,2)) as
Сумма,
    COALESCE(CAST(Арендатор.ФИО AS NVARCHAR(200)), 'Неизвестно') as
Арендатор_ФИО
FROM Оплата_аренды_и_ком_услуг
LEFT JOIN Договор ON TRY_CAST(Оплата_аренды_и_ком_услуг.N_договора
AS INT) = Договор.N_договора
LEFT JOIN Арендатор ON Договор.ID_арендатора = Арендатор.ID_арендатора
ORDER BY TRY_CAST(Оплата_аренды_и_ком_услуг.N_операции AS INT)
DESC
"""

        result = self.execute_query(query, fetch=True)

        if not result.empty:
            print(f"Получено {len(result)} оплат")

        return result

    except Exception as e:
        print(f"Ошибка получения оплат: {e}")
        import traceback
        traceback.print_exc()
        return pd.DataFrame()

```

```

def add_payment(self, contract_id, payment_date, amount):
    try:
        print(f"Добавление оплаты для договора {contract_id}")

        contract_check = """
        SELECT N_договора, ID_арендатора
        FROM Договор
        WHERE N_договора = ?
        """
        contract_result = self.execute_query(contract_check, [int(contract_id)], fetch=True)

        if contract_result.empty:
            print(f"Договор №{contract_id} не найден")
            return False

        print(f"Договор №{contract_id} найден")

        max_query = "SELECT MAX(TRY_CAST(N_операции AS INT)) as max_id FROM
Оплата_аренды_и_ком_услуг"
        max_result = self.execute_query(max_query, fetch=True)

        new_id = 1
        if not max_result.empty and max_result.iloc[0]['max_id'] is not None:
            new_id = int(max_result.iloc[0]['max_id']) + 1

        payment_date_str = payment_date.strftime('%Y-%m-%d')

        query = """
        INSERT INTO Оплата_аренды_и_ком_услуг (N_операции, N_договора,
дата_оплаты, сумма)
        VALUES (?, ?, ?, ?)
        """

        params = [
            int(new_id),
            int(contract_id),
            payment_date_str,
            float(amount)
        ]

        result = self.execute_query(query, params, fetch=False)

        if result:
            print(f"Оплата №{new_id} успешно добавлена для договора №{contract_id}")
        else:
            print(f"Ошибка при добавлении оплаты для договора №{contract_id}")

        return result

    except Exception as e:
        print(f"Ошибка добавления оплаты: {e}")

```

```

import traceback
traceback.print_exc()
return False

def get_all_shops(self):
    query = """
    SELECT
        N_торговой_точки,
        N_помещения,
        категория_торговой_точки,
        стоимость_аренды,
        Название_магазина,
        Статус_договора
    FROM Торговая_точка
    """
    return self.execute_query(query, fetch=True)

def get_active_contracts_for_termination(self):
    """Получить активные договоры для расторжения"""
    try:
        print("Получение активных договоров для расторжения...")
        query = """
        SELECT
            д.N_договора,
            д.N_помещения,
            д.Дата_оформления,
            д.Дата_окончания,
            д.Тариф,
            а.ФИО as Арендатор_ФИО,
            п.Этаж,
            п.Площадь,
            тт.Название_магазина,
            тт.Статус_договора
        FROM Договор д
        LEFT JOIN Арендатор а ON д.ID_арендатора = а.ID_арендатора
        LEFT JOIN Помещение п ON д.N_помещения = п.N_помещения
        LEFT JOIN Торговая_точка тт ON д.N_помещения = тт.N_помещения
        WHERE д.Дата_окончания > GETDATE()
            AND (тт.Статус_договора IS NULL OR тт.Статус_договора = N'Активен')
        ORDER BY д.N_договора
        """
        return self.execute_query(query, fetch=True)
    except Exception as e:
        print(f"Ошибка получения активных договоров: {e}")
        return pd.DataFrame()

def get_tenant_payments(self, tenant_id):
    """Получить платежи арендатора"""
    try:
        print(f"Получение платежей арендатора ID: {tenant_id}")
        query = """
        SELECT

```



```

        о.N_операции,
        о.N_договора,
        о.дата_оплаты,
        о.сумма,
        д.N_помещения,
        п.Этаж,
        Арендатор.ФИО as Арендатор_ФИО
    FROM Оплата_аренды_и_ком_услуг о
    JOIN Договор д ON о.N_договора = д.N_договора
    LEFT JOIN Помещение п ON д.N_помещения = п.N_помещения
    LEFT JOIN Арендатор ON д.ID_арендатора = Арендатор.ID_арендатора
    WHERE д.ID_арендатора = ?
    ORDER BY о.дата_оплаты DESC
    """
    result = self.execute_query(query, [int(tenant_id)], fetch=True)

    if not result.empty:
        print(f"Найдено {len(result)} платежей для арендатора {tenant_id}")
    else:
        print(f"Платежи для арендатора {tenant_id} не найдены")

    return result
except Exception as e:
    print(f"Ошибка получения платежей арендатора: {e}")
    return pd.DataFrame()

def get_tenant_bookings(self, tenant_id):
    """Получить бронирования арендатора"""
    try:
        print(f"Получение бронирований арендатора ID: {tenant_id}")
        query = """
            SELECT
                Бронирование.ID_брони,
                Бронирование.N_помещения,
                Бронирование.ID_арендатора,
                Бронирование.Дата_бронирования,
                Бронирование.Дата_истечения,
                Бронирование.Статус_брони,
                Помещение.Этаж,
                Помещение.Площадь,
                Помещение.Стоимость_аренды,
                Арендатор.ФИО as Арендатор_ФИО
            FROM Бронирование
            LEFT JOIN Арендатор ON Бронирование.ID_арендатора =
Арендатор.ID_арендатора
            LEFT JOIN Помещение ON Бронирование.N_помещения =
Помещение.N_помещения
            WHERE Бронирование.ID_арендатора = ?
            ORDER BY Бронирование.Дата_бронирования DESC
            """
        result = self.execute_query(query, [int(tenant_id)], fetch=True)

```

```

    if not result.empty:
        print(f"Найдено {len(result)} бронирований для арендатора {tenant_id}")
    else:
        print(f"Бронирования для арендатора {tenant_id} не найдены")

    return result

except Exception as e:
    print(f"Ошибка получения бронирований арендатора: {e}")
    return pd.DataFrame()
def get_all_reviews(self):
    try:
        print("Получение всех отзывов...")
        query = """
        SELECT
            от.N_отзыва,
            от.N_торговой_точки,
            от.описание,
            от.дата_отзыва,
            от.автор,
            от.оценка,
            тт.Название_магазина
        FROM Отзыв от
        JOIN Торговая_точка тт ON от.N_торговой_точки = тт.N_торговой_точки
        ORDER BY от.дата_отзыва DESC
        """
        return self.execute_query(query, fetch=True)
    except Exception as e:
        print(f"Ошибка получения отзывов: {e}")
        return pd.DataFrame()
def add_review(self, shop_id, description, author, rating):
    try:
        print(f"Добавление отзыва для торговой точки ID: {shop_id}")
        max_query = "SELECT MAX(N_отзыва) as max_id FROM Отзыв"
        max_result = self.execute_query(max_query, fetch=True)
        new_id = 1
        if not max_result.empty and max_result.iloc[0]['max_id'] is not None:
            new_id = int(max_result.iloc[0]['max_id']) + 1

        review_date = datetime.now().strftime('%Y-%m-%d')
        query = """
        INSERT INTO Отзыв (N_отзыва, N_торговой_точки, описание, дата_отзыва,
автор, оценка)
        VALUES (?, ?, ?, ?, ?, ?)
        """
        params = [
            int(new_id),
            int(shop_id),
            description,
            review_date,
            author,
            rating
        ]

```

```

        int(rating)
    ]
    result = self.execute_query(query, params, fetch=False)
    if result:
        print(f"Отзыв №{new_id} успешно добавлен")
    else:
        print(f"Ошибка при добавлении отзыва")
    return result
except Exception as e:
    print(f"Ошибка добавления отзыва: {e}")
    return False
def get_shops_for_reviews(self):
    try:
        print("Получение торговых точек для отзывов...")
        query = """
        SELECT
            N_торговой_точки,
            Название_магазина
        FROM Торговая_точка
        WHERE Статус_договора = N'Активен'
        ORDER BY Название_магазина
        """
        return self.execute_query(query, fetch=True)
    except Exception as e:
        print(f"Ошибка получения торговых точек для отзывов: {e}")
        return pd.DataFrame()
def get_all_floors(self, floor):
    try:
        print(f"Получение торговых точек на {floor} этаже...")
        query = """
        SELECT
            тт.N_торговой_точки,
            тт.Название_магазина,
            тт.категория_торговой_точки,
            тт.стоимость_аренды,
            п.Этаж,
            п.N_помещения
        FROM Торговая_точка тт
        INNER JOIN Помещение п ON тт.N_помещения = п.N_помещения
        WHERE тт.Статус_договора = N'Активен'
            AND п.Этаж = ?
        ORDER BY тт.категория_торговой_точки, тт.Название_магазина
        """
        return self.execute_query(query, [floor], fetch=True)
    except Exception as e:
        print(f"Ошибка получения торговых точек по этажу {floor}: {e}")
        return pd.DataFrame()
def check_room_availability(self, room_id):
    """Проверить доступность помещения"""
    try:
        print(f"Проверка доступности помещения №{room_id}")

```

```

query = """
SELECT Статус FROM Помещение WHERE N_помещения = ?
"""

result = self.execute_query(query, [int(room_id)], fetch=True)
if not result.empty:
    status = result.iloc[0]['Статус']
    is_available = status == 'Свободно'
    print(f"Помещение №{room_id}: статус '{status}', доступно: {is_available}")
    return is_available, status
else:
    print(f"Помещение №{room_id} не найдено")
    return False, "Не найдено"

except Exception as e:
    print(f"Ошибка проверки доступности помещения: {e}")
    return False, "Ошибка"

def close_connection(self):
    try:
        if self.connection:
            self.connection.close()
            self.connection = None
            print("Подключение к базе данных закрыто")
    except Exception as e:
        print(f"Ошибка при закрытии подключения: {e}")

def get_tenant_all_contracts(self, tenant_id):
    """Получить все договоры арендатора"""
    try:
        print(f"Получение всех договоров арендатора ID: {tenant_id}")

        query = """
        SELECT
            д.N_договора,
            д.ID_арендатора,
            д.N_помещения,
            д.Дата_оформления,
            д.Дата_окончания,
            д.Дата_расторжения,
            д.Тариф,
            п.Этаж,
            п.Площадь,
            тт.Название_магазина,
            тт.категория_торговой_точки,
        CASE
            WHEN д.Дата_расторжения IS NOT NULL THEN 'Расторгнут'
            WHEN д.Дата_окончания < GETDATE() THEN 'Истек'
            ELSE 'Активен'
        END as Статус
        FROM Договор д
        LEFT JOIN Помещение п ON д.N_помещения = п.N_помещения
        LEFT JOIN Торговая_точка тт ON д.N_помещения = тт.N_помещения
        WHERE д.ID_арендатора = ?
        ORDER BY д.Дата_оформления DESC
        """

```

```

result = self.execute_query(query, [int(tenant_id)], fetch=True)
if not result.empty:
    print(f"Найдено {len(result)} договоров для арендатора {tenant_id}")
else:
    print(f"Договоры для арендатора {tenant_id} не найдены")

return result

except Exception as e:
    print(f"Ошибка получения всех договоров арендатора: {e}")
    return pd.DataFrame()
def get_tenant_terminated_contracts(self, tenant_id):
    """Получить расторгнутые договоры арендатора"""
    try:
        print(f"Получение расторгнутых договоров арендатора ID: {tenant_id}")
        query = """
        SELECT
            д.N_договора,
            д.ID_арендатора,
            д.N_помещения,
            д.Дата_оформления,
            д.Дата_окончания,
            д.Дата_расторжения,
            д.Тариф,
            а.ФИО as Арендатор_ФИО,
            п.Этаж,
            п.Площадь,
            тт.Название_магазина,
            тт.категория_торговой_точки,
            'Расторгнут' as Статус,
            FORMAT(д.Дата_оформления, 'dd.MM.yyyy') as Дата_оформления_формат,
            FORMAT(д.Дата_окончания, 'dd.MM.yyyy') as Дата_окончания_формат,
            FORMAT(д.Дата_расторжения, 'dd.MM.yyyy') as Дата_расторжения_формат
        FROM Договор д
        LEFT JOIN Арендатор а ON д.ID_арендатора = а.ID_арендатора
        LEFT JOIN Помещение п ON д.N_помещения = п.N_помещения
        LEFT JOIN Торговая_точка тт ON д.N_помещения = тт.N_помещения
        WHERE д.ID_арендатора = ?
            AND д.Дата_расторжения IS NOT NULL
            ORDER BY д.Дата_расторжения DESC
        """
        result = self.execute_query(query, [int(tenant_id)], fetch=True)
        if not result.empty:
            print(f"Найдено {len(result)} расторгнутых договоров для арендатора {tenant_id}")
            return result
        else:
            print(f"Расторгнутые договоры для арендатора {tenant_id} не найдены")
            return pd.DataFrame()
    except Exception as e:
        print(f"Ошибка получения расторгнутых договоров арендатора: {e}")
        return pd.DataFrame()

```

```

def get_tenant_contracts(self, tenant_id):
    """Получить договоры арендатора (альтернативное имя для совместимости)"""
    return self.get_all_tenant_contracts(tenant_id)
def get_tenant_active_contracts(self, tenant_id):
    """Получить активные договоры арендатора - ТОЧНЫЙ ЗАПРОС"""
    try:
        print(f"Получение активных договоров арендатора ID: {tenant_id}")
        query = """
        SELECT
            д.N_договора,
            д.ID_арендатора,
            д.N_помещения,
            д.Дата_оформления,
            д.Дата_окончания,
            д.Тариф,
            п.Этаж,
            п.Площадь,
            п.Статус as Статус_помещения,
            тт.Название_магазина,
            тт.категория_торговой_точки,
            тт.N_торговой_точки,
            тт.Статус_договора as Статус_торговой_точки,
            'Активен' as Статус,
            FORMAT(д.Дата_оформления, 'dd.MM.yyyy') as Дата_оформления_формат,
            FORMAT(д.Дата_окончания, 'dd.MM.yyyy') as Дата_окончания_формат
        FROM Договор д
        LEFT JOIN Помещение п ON д.N_помещения = п.N_помещения
        LEFT JOIN Торговая_точка тт ON д.N_договора = тт.N_торговой_точки
        WHERE д.ID_арендатора = ?
            AND д.Дата_окончания > GETDATE()
            AND (тт.Статус_договора IS NULL OR тт.Статус_договора = N'Активен')
            AND д.N_договора NOT IN (
                SELECT N_договора
                FROM Договор
                WHERE ID_арендатора = ?
                AND Дата_окончания <= GETDATE()
            )
        ORDER BY д.N_договора
        """
        result = self.execute_query(query, [int(tenant_id), int(tenant_id)], fetch=True)
        if not result.empty():
            print(f"Найдено {len(result)} активных договоров для арендатора {tenant_id}")
            contract_ids = result['N_договора'].tolist()
            print(f"Договоры: {contract_ids}")
        else:
            print(f"Активные договоры для арендатора {tenant_id} не найдены")
        return result
    except Exception as e:
        print(f"Ошибка получения активных договоров арендатора: {e}")
        import traceback
        traceback.print_exc()
        return pd.DataFrame()

```

```

def get_tenant_active_contracts_debug(self, tenant_id):
    """Метод для отладки - возвращает сырые данные о статусе договора"""
    try:
        print(f"DEBUG: Получение отладочной информации для арендатора ID: {tenant_id}")
        query = """
        SELECT
            д.N_договора,
            д.N_помещения,
            тт.Название_магазина,
            тт.Статус_договора,
            LEN(ISNULL(тт.Статус_договора, '')) as Длина_статуса,
            UNICODE(LEFT(ISNULL(тт.Статус_договора, ''), 1)) as Код_первого_символа,
            CAST(тт.Статус_договора AS VARBINARY(100)) as
Статус_договора_бинарный,
            д.Дата_окончания,
            CASE
                WHEN тт.Статус_договора = N'Расторгнут' THEN 'Расторгнут'
                WHEN д.Дата_окончания < GETDATE() THEN 'Истек'
                ELSE 'Активен'
            END as Статус_вычисленный
        FROM Договор д
        LEFT JOIN Торговая_точка тт ON д.N_помещения = тт.N_помещения
        WHERE д.ID_арендатора = ?
        ORDER BY д.N_договора
        """
        result = self.execute_query(query, [int(tenant_id)], fetch=True)
        if not result.empty():
            print(f"DEBUG: Найдено {len(result)} договоров для отладки")
            for _, row in result.iterrows():
                status = row['Статус_договора']
                print(f" Договор №{row['N_договора']}: Статус='{status}'
(длина={row['Длина_статуса']}, код={row['Код_первого_символа']})")
            return result
        except Exception as e:
            print(f"DEBUG: Ошибка отладки: {e}")
            return pd.DataFrame()

def get_tenant_active_contracts(self, tenant_id):
    """Получить активные договоры арендатора - ИСПРАВЛЕННЫЙ ЗАПРОС"""
    try:
        print(f"Получение активных договоров арендатора ID: {tenant_id}")
        query = """
        SELECT
            д.N_договора,
            д.ID_арендатора,
            д.N_помещения,
            д.Дата_оформления,
            д.Дата_окончания,
            д.Тариф,
            п.Этаж,
            п.Площадь,
            п.Статус as Статус_помещения,

```

```

        тт.Название_магазина,
        тт.категория_торговой_точки,
        тт.N_торговой_точки,
        тт.Статус_договора as Статус_торговой_точки,
        'Активен' as Статус,
        FORMAT(д.Дата_оформления, 'dd.MM.yyyy') as Дата_оформления_формат,
        FORMAT(д.Дата_окончания, 'dd.MM.yyyy') as Дата_окончания_формат
    FROM Договор д
    LEFT JOIN Помещение п ON д.N_помещения = п.N_помещения
    LEFT JOIN Торговая_точка тт ON д.N_договора = тт.N_торговой_точки --
ИЗМЕНЕНИЕ: связь по номеру договора
    WHERE д.ID_арендатора = ?
        AND д.Дата_окончания > GETDATE()
        AND (тт.Статус_договора IS NULL OR тт.Статус_договора = N'Активен')
    ORDER BY д.Дата_окончания DESC
    """

    result = self.execute_query(query, [int(tenant_id)], fetch=True)
    if not result.empty:
        print(f'Найдено {len(result)} активных договоров для арендатора {tenant_id}')
    else:
        print(f'Активные договоры для арендатора {tenant_id} не найдены')
    return result

except Exception as e:
    print(f'Ошибка получения активных договоров арендатора: {e}')
    import traceback
    traceback.print_exc()
    return pd.DataFrame()

def get_all_tenant_contracts(self, tenant_id):
    """Получить все договоры арендатора - ИСПРАВЛЕННЫЙ"""
    try:
        print(f'Получение всех договоров арендатора ID: {tenant_id}')
        query = """
        SELECT
            д.N_договора,
            д.ID_арендатора,
            д.N_помещения,
            д.Дата_оформления,
            д.Дата_окончания,
            д.Тариф,
            п.Этаж,
            п.Площадь,
            тт.Название_магазина,
            тт.категория_торговой_точки,
            тт.Статус_договора,
            CASE
                WHEN тт.Статус_договора = N'Расторгнут' THEN N'Расторгнут'
                WHEN д.Дата_окончания < GETDATE() THEN N'Истек'
                ELSE N'Активен'
            END as Статус,
            FORMAT(д.Дата_оформления, 'dd.MM.yyyy') as Дата_оформления_формат,
            FORMAT(д.Дата_окончания, 'dd.MM.yyyy') as Дата_окончания_формат
        """

```



```

        FROM Договор д
        LEFT JOIN Помещение п ON д.N_помещения = п.N_помещения
        LEFT JOIN Торговая_точка тт ON д.N_договора = тт.N_торговой_точки --
    ИЗМЕНЕНИЕ: связь по номеру договора
    WHERE д.ID_арендатора = ?
    ORDER BY д.Дата_оформления DESC
    """
    result = self.execute_query(query, [int(tenant_id)], fetch=True)

    if not result.empty:
        print(f"Найдено {len(result)} договоров для арендатора {tenant_id}")
    else:
        print(f"Договоры для арендатора {tenant_id} не найдены")
    return result
except Exception as e:
    print(f"Ошибка получения всех договоров арендатора: {e}")
    return pd.DataFrame()
def terminate_contract(self, contract_id):
    """Расторгнуть договор"""
    try:
        print(f"Расторжение договора ID: {contract_id}")
        check_query = """
            SELECT д.N_договора, д.N_помещения, д.Дата_окончания
            FROM Договор д
            WHERE д.N_договора = ?
        """
        check_result = self.execute_query(check_query, [int(contract_id)], fetch=True)
        if check_result.empty:
            print(f"Договор №{contract_id} не найден.")
            return False
        room_id = check_result.iloc[0]['N_помещения']
        yesterday = datetime.now().date() - timedelta(days=1)
        update_contract_query = """
            UPDATE Договор SET Дата_окончания = ? WHERE N_договора = ?
        """
        contract_result = self.execute_query(
            update_contract_query,
            [yesterday, int(contract_id)],
            fetch=False)
        if not contract_result:
            print(f"Ошибка обновления даты окончания договора №{contract_id}")
            return False
        print(f"1. Дата окончания договора №{contract_id} изменена на {yesterday}")
        update_room_query = """
            UPDATE Помещение SET Статус = N'Свободно' WHERE N_помещения = ?
        """
        room_result = self.execute_query(update_room_query, [int(room_id)], fetch=False)
        if not room_result:
            print(f"Ошибка освобождения помещения №{room_id}")
            return False

        print(f"2. Помещение №{room_id} освобождено (Статус: Свободно)")
    
```

```

update_shop_query = """
    UPDATE Торговая_точка
    SET Статус_договора = N'Расторгнут'
    WHERE N_торговой_точки = ?
"""

shop_result = self.execute_query(update_shop_query, [int(contract_id)], fetch=False)
if not shop_result:
    print(f"3. Внимание: Не удалось обновить статус торговой точки для договора №{contract_id}")
    print("    Возможно, торговая точка не существует для этого договора")
else:
    print(f"3. Статус торговой точки для договора №{contract_id} изменен на 'Расторгнут'")
    print(f"    Договор №{contract_id} успешно расторгнут!")
    print(f"    • Дата окончания изменена на вчерашнюю дату ({yesterday})")
    print(f"    • Помещение №{room_id} освобождено")
    print(f"    • Статус торговой точки изменен на 'Расторгнут'")
    return True
except Exception as e:
    print(f"Ошибка расторжения договора: {e}")
    import traceback
    traceback.print_exc()
    return False

def add_shop(self, room_id, category, rent_cost, shop_name, contract_id=None,
status='Активен'):
    try:
        print(f"Добавление торговой точки для помещения {room_id}")
        if contract_id is not None:
            shop_id = contract_id
            print(f"Для торговой точки будет использован ID договора: {shop_id}")
        else:
            max_query = "SELECT MAX(N_торговой_точки) as max_id FROM
Торговая_точка"
            max_result = self.execute_query(max_query, fetch=True)
            shop_id = 1
            if not max_result.empty and max_result.iloc[0]['max_id'] is not None:
                shop_id = int(max_result.iloc[0]['max_id']) + 1
            query = """
                INSERT INTO Торговая_точка (N_торговой_точки, N_помещения,
категория_торговой_точки, стоимость_аренды, Название_магазина, Статус_договора)
                VALUES (?, ?, ?, ?, ?, ?)
            """
            params = [
                int(shop_id),
                int(room_id),
                str(category),
                float(rent_cost),
                str(shop_name),
                str(status)
            ]
            result = self.execute_query(query, params, fetch=False)

```

```

if result:
    print(f"Торговая точка '{shop_name}' (ID: {shop_id}) успешно добавлена")
    return shop_id
else:
    print(f"Ошибка при добавлении торговой точки")
    return False
except Exception as e:
    print(f"Ошибка добавления торговой точки: {e}")
    return False
def get_tenant_shops(self, tenant_id):
    """Получить торговые точки арендатора"""
    try:
        print(f"Получение торговых точек арендатора ID: {tenant_id}")
        query = """
            SELECT тт.N_торговой_точки, тт.Название_магазина,
            тт.категория_торговой_точки, тт.стоимость_аренды, тт.N_помещения,
            тт.Статус_договора, д.N_договора, д.Дата_оформления, д.Дата_окончания, д.Тариф,
            п.Этаж, п.Площадь, п.Статус as Статус_помещения,
            CASE
                WHEN д.Дата_окончания > GETDATE() AND тт.Статус_договора =
                N'Активен' THEN N'Активен'
                WHEN д.Дата_окончания < GETDATE() THEN N'Истек'
                WHEN тт.Статус_договора = N'Расторгнут' THEN N'Расторгнут'
                ELSE N'Неизвестно'
            END as Статус,
            FORMAT(д.Дата_оформления, 'dd.MM.yyyy') as Дата_оформления_формат,
            FORMAT(д.Дата_окончания, 'dd.MM.yyyy') as Дата_окончания_формат
            FROM Договор д
            LEFT JOIN Торговая_точка тт ON д.N_договора = тт.N_торговой_точки
            LEFT JOIN Помещение п ON д.N_помещения = п.N_помещения WHERE
            д.ID_арендатора = ?
            ORDER BY д.Дата_оформления DESC
            """

        result = self.execute_query(query, [int(tenant_id)], fetch=True)

        if not result.empty():
            print(f"Найдено {len(result)} торговых точек для арендатора {tenant_id}")
        else:
            print(f"Торговые точки для арендатора {tenant_id} не найдены")
        return result
    except Exception as e:
        print(f"Ошибка получения торговых точек арендатора: {e}")
        return pd.DataFrame()
def get_all_floors_list(self):
    """Получить список всех этажей"""
    try:
        print("Получение списка всех этажей...")
        query = """
            SELECT DISTINCT Этаж FROM Помещение
            ORDER BY Этаж
            """

```

```

result = self.execute_query(query, fetch=True)
if not result.empty:
    floors = result['Этаж'].tolist()
    print(f"Найдено этажей: {floors}")
    return floors
else:
    print("Этажи не найдены")
    return []
except Exception as e:
    print(f"Ошибка получения списка этажей: {e}")
    return []
def get_floors_with_shops(self):
    """Получить список этажей с активными торговыми точками"""
    try:
        print("Получение этажей с активными торговыми точками...")
        query = """
        SELECT DISTINCT п.Этаж FROM Торговая_точка тт
        INNER JOIN Помещение п ON тт.N_помещения = п.N_помещения WHERE
        тт.Статус_договора = N'Активен'
        ORDER BY п.Этаж
        """
        result = self.execute_query(query, fetch=True)
        if not result.empty:
            floors = result['Этаж'].tolist()
            print(f"Этажи с активными торговыми точками: {floors}")
            return floors
        else:
            print("Активных торговых точек не найдено")
            return []
    except Exception as e:
        print(f"Ошибка получения этажей с торговыми точками: {e}")
        return []
def get_active_shops_by_floor(self, floor):
    """Получить активные торговые точки на указанном этаже"""
    try:
        print(f"Получение активных торговых точек на {floor} этаже...")
        query = """
        SELECT тт.N_торговой_точки, тт.Название_магазина,
        тт.категория_торговой_точки, тт.стоимость_аренды, п.Этаж, п.N_помещения,
        д.N_договора, а.ФИО as Арендатор_ФИО FROM Торговая_точка тт
        INNER JOIN Помещение п ON тт.N_помещения = п.N_помещения
        LEFT JOIN Договор д ON тт.N_торговой_точки = д.N_договора
        LEFT JOIN Арендатор а ON д.ID_арендатора = а.ID_арендатора
        WHERE тт.Статус_договора = N'Активен' AND п.Этаж = ?
        ORDER BY тт.Название_магазина
        """
        return self.execute_query(query, [floor], fetch=True)
    except Exception as e:
        print(f"Ошибка получения торговых точек по этажу {floor}: {e}")
        return pd.DataFrame()

```